



ENHANCING SOLAR POWER ECONOMICS FOR HOUSEHOLDS



JUNE 12, 2024

Analysis

Background: Often, solar panel owners sell excess electricity to the grid when generating from solar panels more than is being used domestically, the grid buys electricity off solar panel owners for a much lower price than it sells it to them. The national grid sells electricity at a price of an average of 27p per KWh in the past year. The national grid buys excess electricity off solar panel owners at a price that ranges between 1p per KWh to 24p per KWh. These prices range based on seasons and time of day or night and are rough averages/ ranges.

The problem: Solar panel owners end up selling excess electricity to the grid one day at a rate of 12p per KWh for example and buying it back for 27p per KWh the next day, this is obviously extremely cost inefficient and could lose solar panel owners lots of money over the course of a year, especially in the summer. I believe this can be solved computationally provided the computer has all the data it needs, for example it would need the current cost of electricity from the grid, the cost of electricity from the grid at different times and seasons and the predicted weather. If it has all data needed it should be able to carry out a set of calculations and make decisions on what to do with electricity generated at given moments, for example when to use battery's charge.

Intentions of solutions: Save solar panel owners money by creating an algorithm that uses the weather predictions for the next day to change how much solar panel users charge the battery to during off-peak hours. This will save solar panel owners time by automatically making these changes to energy system instead of making the user manually check the weather and change the charge battery will be charged to. In summary I will make an application that makes changes to user's energy plan based on their solar panels and weather for next day to save user time and money.

Why it is needed: My program aims to ensure solar panel owners buy less energy from the grid during peak hours, buy as much as possible during off-peak hours and put it into battery and sell less to them by not charging battery too much. This should make solar panels a more viable option for households when considering cost of them and will hopefully increase the demand for solar panels domestically by making them more economically viable (by saving a bit extra every week the user will break even on their costs quicker). My algorithm should also level out the demand for electricity because if more people are using electricity at night and less at day, the demand for electricity in the day will decrease and therefore so should peak rates, making electricity cheaper.

Who: My program will be directed at solar panel owners on a domestic level (house owners with solar panels on the roof) but there is no reason other solar panel owners can't use them (e.g. business owners with solar panels).

The next step: Research further the client's needs and wants and how the problem can be solved.

Stakeholder/stakeholders

Stakeholder 1

Name: Dave Wallace

Why he is a stakeholder: Dave owns a large set of solar panels on the roof of his garage, a battery and an electric car. He is an ideal user for the algorithm as there are many decisions that the algorithm could/should take to be as cost effective as possible. Dave's current tariff includes a 56.28p standing charge per day a peak unit rate of 31.237p per KWh and an off-peak unit rate of 10.374p per KWh.

Availability: I will try to ask for the user's opinion at major stages of the process, after each iteration and before each decision to make the algorithm as user friendly as possible and as beneficial as possible.



Stakeholder 2

Name: Andrew Sainsbury

Why he is a stakeholder: His business (Taste connection/nactarome) has a large factory with many solar panels on the roof and a smaller building with solar panels as you can see on the right so I think he could heavily benefit from my algorithm as it could save him lots of money. Andrew is not a high priority stakeholder as the algorithm is mainly directed at household savings but there is no reason why he couldn't also use my algorithm, in fact, the more solar panels and battery storage the user has the more money the user can save.

Availability: I will try to get feedback from Andrew to help set criteria and usability features and hopefully have feedback at the end to see how successful my algorithm was.



Stakeholder user needs and wants

Interview with David

Me, David

What solution are you currently using to fix your problem?

Right now, I am manually adjusting when to use battery electricity, solar panel electricity, when to buy from the grid, when to charge my car and I often forget to charge the battery during off-peak hours. I need to check the weather for tomorrow every night and change how much to charge the battery to.

Would you be interested in an app/algorithm that did all of this for you and at a maximum economic efficiency?

Yes, that sounds like it would save a lot of time and money.

And what would you like from this program?

I would like it to be able to use predictions of weather where I live along with my current battery percentage and my peak and off-peak times to decide how much to charge battery to over night, I would like to buy less electricity during my peak rate, sell less electricity back to the grid for very little price and buy more electricity during my off-peak rate to charge my battery.

Are there any other specific needs or wants you have?

Yes, I would like the algorithm to be user-friendly, easily accessible and clear with what is currently going on with my electricity. I would also like it to be able to predict how much energy is demanded for the next day based on temperature and season.

So, an application that displays the energy plan in a user-friendly way would be ideal?

Yes, I think that would be very useful.

What specific hardware do you currently use for solar panels and battery?

I use a givenenergy battery, a JA Solar 395W Mono MBB Percium Half-Cell All Black Short Frame solar panel and an EV charger.

Is there any other hardware or software you currently use for electricity management?

I use givenenergy app which lets me monitor and manage my energy system, however I have to make changes to it manually which is inconvenient.

What are your tariff peak hours and rates?

Anytime from 5am to midnight is peak rate of 31.273p per KWh and my off-peak rate is 10.374p per KWh.

How much do you sell electricity back to the grid for?

Pretty much zero.

Interview with Andrew

What software/hardware do you use alongside your solar panels?

I use givenenergy to control my energy flow, batteries and view energy details.

And what would you like from the algorithm?

I would like to save money in any way possible, including being able to input into an application that I'm away so that the algorithm knows that I will use less energy and make changes.

User needs and wants summary/ user requirements

- Algorithm that optimises how and when electricity is used and how much to charge battery during off-peak hours to save as much energy as possible
- Application works side by side with users givenenergy app
- Easy to understand what algorithm is doing with electricity at any given time
- Be time saving and convenient to user
- Saves user money
- Completely automated

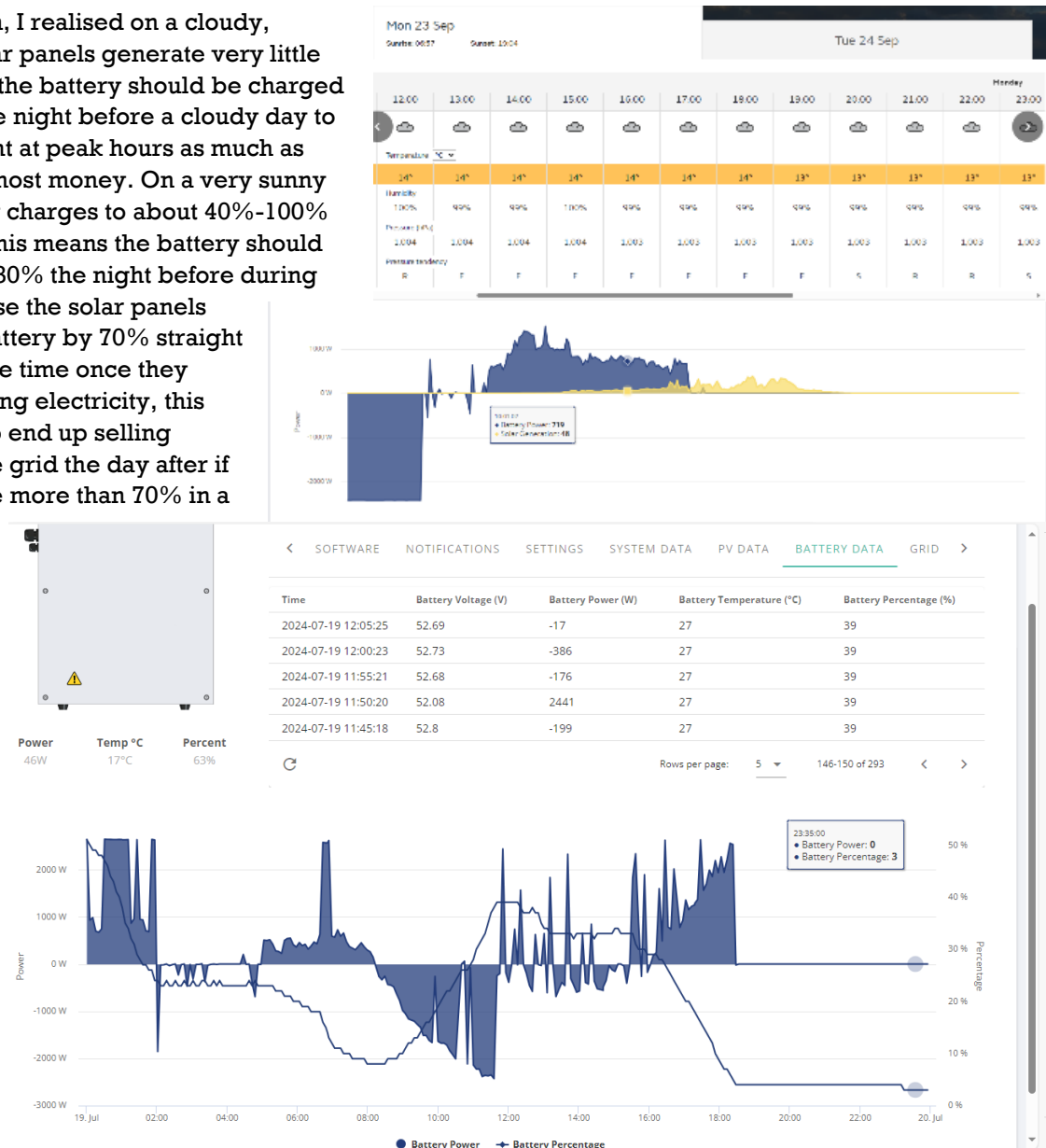
Research

Further knowledge/details

Solar panels are made from photovoltaic cells that convert the sun's energy into electricity. This is done because of the photo-electric effect, photons from the sun's light hit solar panels giving electrons enough energy to flow through circuit. The main variable that determines how much electricity a solar panel produces is the amount of sunlight it receives (and not what I thought which was UV level) which means overcast days will produce little electricity and clear days will produce lots. After further research, I realised that if the user were to buy electricity during off-

peak times and store it in their battery, they can save a lot of money (20p per KWh). If the weather predicts a clear, sunny day the next day, the battery should start the day with no/less charge, therefore it should have been used up the day before. The peak times in the UK on weekdays are usually 4 to 7pm/7 to 9pm, on weekends and the off peak times in the UK fall between 11pm and 8am, the rates and peak/off peak times all depend on the type of metre and tariff the user owns this doesn't mean that the user has to input their tariff as my algorithm functions using their givenenergy portal which already contains the users tariff details. My main client's peak hours are from 5am to midnight at a rate of 31.273p and an off-peak rate of 10.374p per KWh. My client currently uses givenenergy to control his energy system.

After further research, I realised on a cloudy, overcast day, the solar panels generate very little power and therefore the battery should be charged overnight to 100% the night before a cloudy day to reduce energy bought at peak hours as much as possible to save the most money. On a very sunny clear day, the battery charges to about 40%-100% during the summer, this means the battery should be charged to about 30% the night before during off peak hours because the solar panels will not charge the battery by 70% straight away, it will take some time once they have started generating electricity, this may cause the user to end up selling electricity back to the grid the day after if solar panels generate more than 70% in a short amount of time which can cause a 10p per KWh unnecessary expenditure but that is better than not charging the battery enough the night before and having to import at peak rate as this causes a 20p per KWh unnecessary expenditure. The algorithm should never use battery at off peak rates as this saves no money.



How to calculate how much solar energy will be generated: to find the hours of peak sunlight use the chart on right which calculates the average peak sunlight hours in a day for certain months in Nympsfield (where client lives), for solar panels of client's angle from north and tilt angle from the ground. In the first 9 days of the month should calculate hours of peak sunlight by doing 3 times that month's peak sunlight hours for that month plus the peak sunlight hours of last month times 2 all over 5 to create a more realistic value, do a similar calculation for last 9 days of the month. Multiply hours of peak sun hours (depends on the months) by 1-cloud coverage times by solar panel wattage by number of solar panels to calculate how many KWhs will be generated the next day. On 07/10/24 the next day's solar energy generated will be 12 times 2.426 times 295 times 0.06 which equals 0.69 KWhs. On 09/10/24 the generation for next day will be 2.426 times 12 times 395 times 0.24 which is 2.8 KWhs. This can be calculated by my algorithm at 23:00 the day before so that the battery can be charged accordingly, for example if the solar panels are going to generate 110% or more of battery's storage, don't bother charging the battery the night before. The battery should aim to end each day with 4% so that as little electricity is imported from the grid as possible. The battery has a capacity of 14.75 KWhs and on average during the summer Dave uses about 25-35 KWhs during

Your annual average:

2.94 peak sun hours per day

Your monthly averages:

- January: 0.73
- February: 1.64
- March: 2.87
- April: 4.14
- May: 4.56
- June: 5.08
- July: 4.82
- August: 3.89
- September: 3.67
- October: 2
- November: 1.15
- December: 0.72

October 14-20 th (2)	Monday	Tuesday	Wednesday	Thursday	Friday	Saturday	Sunday
Weather	0.45 cloud coverage (CC)	0.8 CC	0.35 CC	0.05 CC	0.5 CC	0.6 CC	0.1 CC
Predicted energy for today	5.214 KWhs	1.896 KWhs	6.162 KWhs	9.006 KWhs	4.740 KWhs	3.792 KWhs	8.532 KWhs
Battery% After off peak charging in the AM	74.7	97.1	68.2	48.9	77.9	84.3	52.2

the summer and more like 35-45 KWhs in the winter. Therefore, the percentage to charge battery to the day before where percentage day before $\leq 100\%$ can be calculated by doing predicted solar energy production dived by 14.75 KWhs times this all by 100, take this away from 100% this is how much percentage to charge the battery to during off peak hours. Add on a 10% to whatever is the result because of the energy used before the solar panels start generating.

The highest rates of energy production by solar panels are generally between 10:00 and 16:00.

The user uses an average of 1.6 KWhs before the solar panels start generating energy.

This is a table to demonstrate what changes my algorithm should make in a week to save money.

To predict the next day's energy demand should use today's energy consumption along with today's temperature and tomorrow's temperature to predict tomorrow's energy demand. If the temperature is less tomorrow there will be more energy demand. Calculation (Tomorrow's average temp-Today's temperature out)/today's temperature * 1/3 today's energy consumption + today's energy consumption. After researching the givenenergy API further I have realised that I cannot access the weather through givenenergy, so I need to use a different API called solarcast to access weather.

Existing systems

Solar diverter

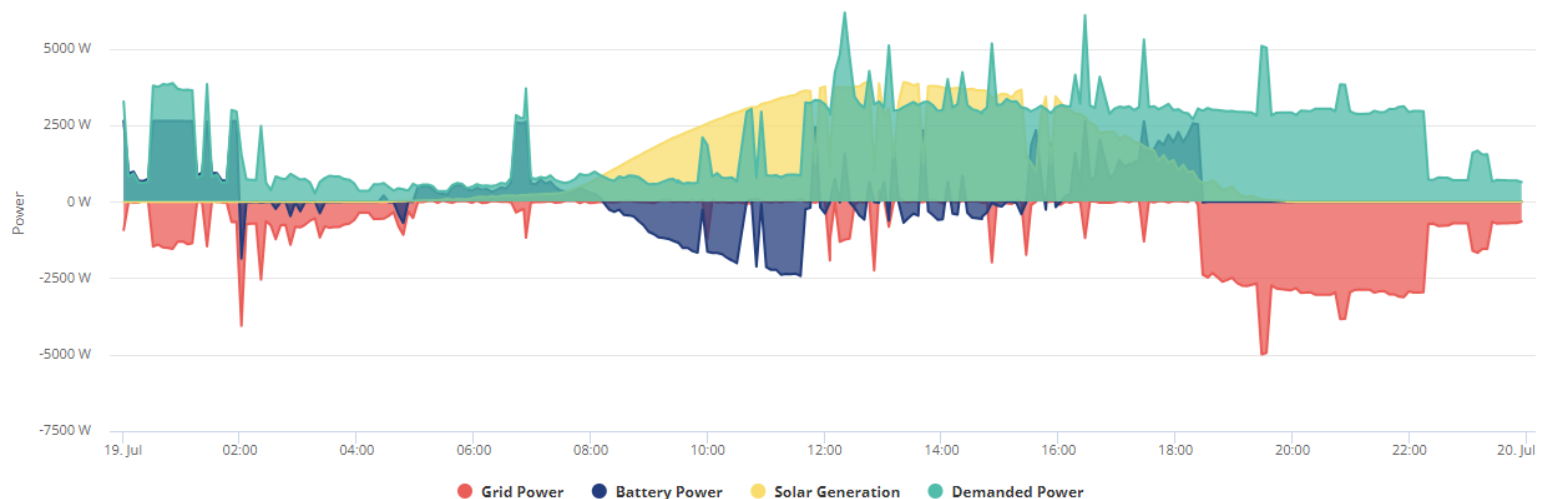
A solar diverter is a device used alongside with solar panels to optimise energy usage for a household. Its purpose is to direct excess electricity generated by the solar panels towards a specific load or appliance, usually to heat water, instead of exporting it back to the grid for a cheap price. This maximises the overall efficiency of the solar panels for the homeowner. When the solar panels generate more electricity than is currently being used within the home, the solar diverter detects this surplus and automatically diverts the excess energy to a designated appliance. These generally cost from £200-300 and according to David Lukeman they can pay for themselves within two years. It can't be used to heat water if the property uses a combi boiler.

Negatives of this product is a solar diverter does not predict when householders will be producing excess or at what times, they also do not take into account what hours are peak rate or off-peak for user. My system allows for both of these variables which means it is likely to save the client more money than a solar diverter. My system should also work no matter the hardware used by the client but will be dependent on them using givenenergy.

Solar buyback

Solar buyback is also referred to as solar equity release, and you may also see offers to sell your feed-in tariff. Companies offer a lump sum to householders who already have solar PV installed, in exchange for receiving the remainder of their feed-in tariff. As with free solar panel schemes, these companies are keen to cash in on the feed-in tariff.

A negative of this is that companies buy it off users for an extremely low prices compared to what homeowners buy electricity off the national grid this means homeowners essentially buy electricity at one point and sell it later for a lower price. My



system/algorithm plans to avoid doing this by calculating and predicting when to use and when and how much to charge battery percentage, when to use grid electricity based on weather, tariff rates and times and percentage in battery left to avoid selling excess electricity to the grid as much as possible and avoid buying from the grid as well.

Voltage optimiser

Generally, power from the National Grid is supplied at a higher voltage than is required. Although the official normal supply voltage in the UK is 230V, the actual voltage supplied by the National Grid fluctuates around an average of 242V.

Most appliances are rated at 220V. Therefore, appliances are being supplied with more electricity than necessary to operate. This results in both energy and financial waste as well as extra wear and tear on your appliances.

By installing a voltage optimiser, you can reduce the voltage received by your home or business. Voltage optimisers are installed in series with your electricity supply, between the distribution transformer and the main low-voltage distribution board. When the electricity passes through your Voltage Optimiser, the supply voltage will be reduced to the required amount. Any excess voltage being supplied will be simply returned back to the grid and not used on-site. By rejecting and returning the surplus to the grid, your electrical equipment will benefit from an optimised power supply and you will save money.

A component that my algorithm will have that a voltage optimiser doesn't have is it will take into account usage and tariff money on rates and times. My product will not do the job of a voltage optimiser but will serve the same purpose (saving electricity). My product would also be able to work with a voltage optimiser for maximum savings.

Givenergy

Givenergy are a company that produces a battery, solar inverter and an app. David (my main stakeholder) uses the givenergy app to manage and monitor their energy system and also use a givenergy battery as stated in the client interview.

The givenergy app works alongside an inverter, generation source, battery and an EV charger, this should make it very easy to link battery and other hardware by just connecting my algorithm to the app. The web portal offers an API to implement custom automations which will make implementing my algorithm much easier.

Ideas and approaches

I would like to create an application that works side by side with the client's app givenergy to automatically make changes to the user's energy system. My application should calculate the predicted energy produced the next day by solar panels and use this to decide what percentage to charge the battery to during off-peak hours so that there is room in the battery if there will be excess electricity being generated to avoid selling back to the grid and avoid buying from the grid as much as possible. The user does not need to make any changes or input any data if they already use the givenergy app as the algorithm will automatically make changes to the energy plan if there is no user input but they may like to, for example if the algorithm has decided that the battery

will be charged to 80% over night but the user knows that most people in the house will be out all day and not use the energy they may decide to change it to 78%.

Proposed solution

Features	Explanation	Justification
Automated	User inputs data when setting up, from then on algorithm is in control unless user deems so.	Save user time by not making user manually change what to charge the battery to based on the next days weather.
Estimates demanded energy	Estimates how much energy will be needed the next day.	Is needed in the battery percentage to charge overnight calculation.
User Interface	Shows user what is planned and why for the next day/current day.	For easy usability and so the user can make changes if they feel it is needed.
Calander on user app	So, the user can input when they are away.	So that the algorithm does not buy as much energy when user is away during off-peak hours because it likely won't be needed.
Input area for user to input their identification	So, the application can connect to the user's givenenergy website by inputting API key and Solar Panels ID.	So it is easy for anyone to download application and connect to their givenenergy.

Identifying possible limitations

A limitation I may experience is creating a user interface that can be used alongside the algorithm as it must be automatic and may be difficult to implement the automatic aspect and the API code into the App. This is because to be automatic, the code must run at certain times without user telling it to this means code must be stored somewhere where it can be implemented that it must run at certain times. This will be hard if there are multiple users as I will need to create a separate algorithm without a GUI that access the GUI's database to access API keys and other details and then iterates through every user in the list running the same program with each user's details.

Another limitation may be the accuracy of the algorithm and weather recordings, is hard to predict how much electricity will be produced by solar panels which may mean not much money is saved. This is because the cloud coverage predictions for the next day can vary greatly which will affect my energy predictions.

Another limitation is the usefulness of the algorithm during the winter when there is little sun and there is not much energy being produced by the solar panels so there is not much that can be done to save money. During the winter there are also much shorter days which will obviously reduce the amount of energy that is generated as there are less hours to do so.

The last limitation that I see is that my program is completely geared around givenenergy solar panel inverter users. This means people without givenenergy will not be able to use my program and cannot access savings. This will reduce my "target market" majorly and in the future, I could make more versions for different inverter suppliers. Providing the different suppliers offer API control, hopefully only method will need to be added and or changed.

Computational methods

My solution is computationally solvable because it has many logical steps to calculating how much to charge the users battery to at off-peak times to save the user the most money. My algorithm is also going to be finding patterns in data which can be used to self-improve and increase savings for user based on previous data. The algorithm will also eliminate human error in the calculations of best plans to implement to save the most money.

I will use abstraction to take away the unnecessary parts of problem, e.g. whether it will rain today and leave in necessary parts of the problem e.g. when peak rate is charged and will it be cloudy or clear today, this will make the problem easier to solve.

Concurrency should be used when diverting electricity from solar panels to both the battery and household usage on a clear day when solar panels are producing more energy than needed to avoid selling the excess to grid and give electricity a place to go.

Frequently used data, e.g. number of solar panels and their wattage should be cached for quick access as these will be used in every calculation to find out how much energy solar panels will produce the next day.

I think my algorithm will use backtracking as it will need to analyse how effective the energy predictions are and make changes in the future to the calculation based on results to allow for more accurate predictions and therefore save the user more money, to do this I should use selection and iteration modules to alter the calculator for certain months. It should also use backtracking to find the energy used the day before on givenenergy so that the demanded energy for the next day can be estimated.

I will decompose the problem down into smaller more solvable problems for example I will tackle the calculations of different situations individually, for example calculate the predicted energy for tomorrow then calculate how much to charge battery to the night before and what percentage to add on to this depending on early morning usage and overall usage for that day.

I will use an API to get data out of the givenenergy portal and input data into the givenenergy portal (i.e. make decisions) such as getting the cloud coverage for the next day out of givenenergy, calculating the next days predicted solar panel energy with this, how much energy was used before solar panels start to generate on that day (output from givenenergy) and calculate what percentage to charge the battery to during the off-peak hours and input this into givenenergy. The givenenergy portal provides an easy guide on how to use their API.

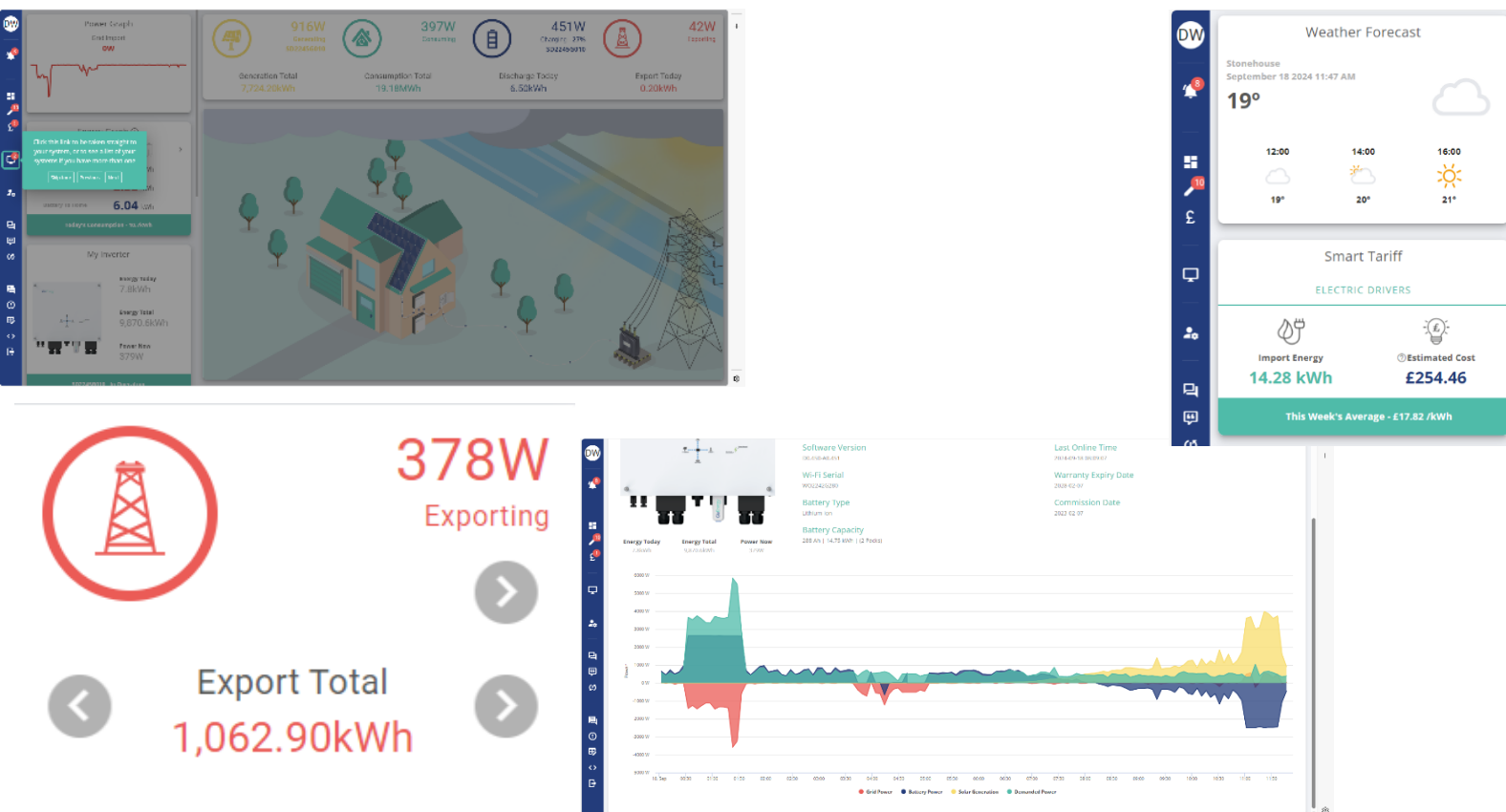
Hardware and software

Hardware

There are not many hardware requirements as the givenenergy portal allows for easy implementation of automation software. Thanks to the AlarmManager class in visual studios I do not need any raspberry pi or QNAP NASs and can make the algorithm automatic only using visual studios, although I suspect I may need one of these if the volume of users becomes too much for my computer alone.

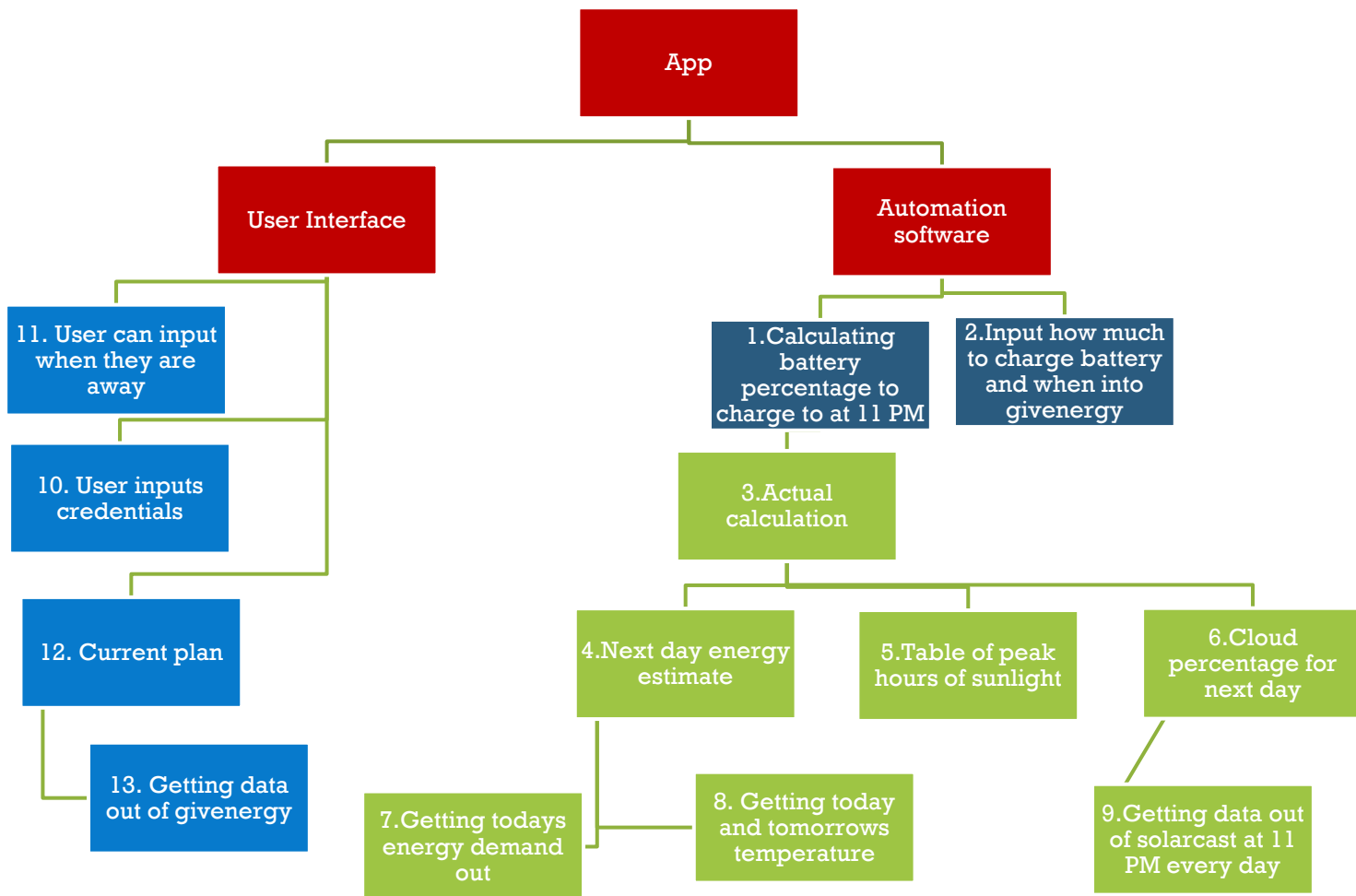
Software

I should use Python as the givenenergy API supports it and is a language supported by visual studios. The IDE used should be Visual studios as I am well experienced with this, and this IDE supports the use of tkinter which is the library I will use to help build my GUI. I will need to use Davids givenenergy API to fetch parameters from the givenenergy portal such as battery percentage, the givenenergy portal contains its own weather but it is not accessible through the API so I will need to use a different API called solarcast. Accessing the givenenergy's API should be straightforward because givenenergy have released the API to the public, there is a guide on the givenenergy website on how to use the givenenergy API. To design the program and implement it, I need access to David's givenenergy portal. After looking at the givenenergy portal on Davids account, I can see that the problem in detail, David has exported a lot of energy to the grid and during mid day on the 18/09/24 is exporting 378W, this may have been avoided if he had more percentage left in his battery, which is what my algorithm would have planned for and would have used energy from the battery at times of 12:00-1:30 instead of using grid energy because algorithm can see it will be sunny the next day and would generate more than can use and would therefore save some money. For any databses needed I should use an excel file. To upload and convert my code into a website I should use "Python Anywhere" which converts python code into HTML and CSS.



No.	Success Criteria	Justification
1	Be automatic , once set up, user does not need to interact with givenenergy, the algorithm should make changes to how much to charge battery to during off-peak hours on its own.	To be convenient for the user, the user needs to trust that the algorithm is saving them money without their input or/and them constantly checking it is working.
2	Reduce energy sold by user back to the grid , especially in the summer when the sun is more intense and the user is producing more energy than they are using which leads to excess energy sold to grid, should accomplish this by ensuring battery is only charged enough at start of the day to get through demanded energy before solar panels start generating energy. If there is battery left over from the day that is not predicted to be needed the next day use it in the user's washing machine and dishwasher energy peak during off-peak hours.	The grid buys energy from homeowners for a much lower price than the price that the grid sells it back to them, therefore by reducing the energy sold the grid and storing it for later use the user spends less on the higher rates charged by the grid as they need to buy less.
3	Reduce energy bought from the grid , buys all/most energy during off-peak hours in midnight to 5AM hours, use this energy on washing machines and dishwashers during these times as well.	This means the user saves on average 20p per KWh which will save the user lots of money.
4	User should be able to input when they are away and the algorithm should aim to buy no energy from the grid to keep up with low demand from an empty house. The algorithm should do this by storing solar energy in battery and use battery/solar energy at any point there is demanded energy because the solar panels should be generating more than the house is using apart from maybe in winter months.	This will cost the user the least money whilst they are away from home (hopefully cost them only the standing fees).
5	User interface.	For user to interact with.
6	User can view what is happening with their energy at any given time.	So the user can feel reassured that the algorithm is working.
7	Be reliable , consistently makes roughly accurate predictions and meets other specifications no matter the month.	So that user is confident enough to not check up on their givenenergy system for months and feel confident in the algorithm to do its job.
8	Saves user above minimum 4 pounds a month regularly, above 6 pounds a month in the summer.	This is so that the user feels it is worthwhile to implement this algorithm into their givenenergy portal.
9	Uses past data to influence the algorithms decisions (might be a bit of a stretch).	This should result in more savings as the algorithm will be able to see what did work and what didn't work by how much is saved.
10	A way to calculate how much energy has been saved via tariff rates, total energy bought off the grid, total energy produced by solar panels, the same calculations assuming no changes were made that month by the algorithm, this may be quite difficult.	So that the user can see the effects of the algorithm and the algorithm can use the data to make decisions in the future.
11	User enters their credentials into website including API key and Solar Panels ID.	So the website can access the user's givenenergy account.
12	Accessible to the public.	Anyone can access/download/use the app.
13	Estimates next day's energy production.	To predict how much to charge battery to.

Design Main



Box 1- This section

should calculate the optimum battery percentage to charge the battery to the next day by calling smaller functions to perform different tasks.

Box 3- This is the module that performs the actual calculation using all the variables found in modules 4, 5, and 6 to be used in box 1. $\text{Peak hours of Sun} \times (1 - \text{Cloud coverage}) \times \text{solar panel wattage} (395) \times \text{number of solar panels} (12)$.

Box 4- This module estimates how much demanded energy is needed for the next day based on the date tomorrow, tomorrow's average temperature and today's energy used and average temperature to be used in box 3, a change in temperature often results in more or less energy being used domestically.

Box 7- A module for getting the day's energy consumption out of givenergy including the API get function to be used in box 4.

Box 8- Getting the change in temperature can be helpful to estimate the change in energy demand, will use solarcast API to access the temperature.

Box 5- A table of the peak hours of sunlight for different days and months of the year to be used in box 3 stored as a 2D array.

Box 6- The predicted cloud percentage for the next day to be used in box 3.

Box 9- The actual API get function code that gets cloud percentage out of solarcast.

Box 2-Inputs into givenergy the value of percentage calculated in box 1 and the datetime to charge battery to in.

Box 10- User inputs their credentials into the website so the algorithm has access to their givenergy by storing the Serial Number and API key as variables to be used in the givenergy API get/POST/PUT functions. Needs lots of validation to check that all data is in the correct forms.

Box 11- User inputs when they're away so algorithm can take that into account and charge battery up less the night before to waste less energy and therefore money.

Box 12-User can see what algorithm is doing/planning on doing via givenergy API get functions.

Box 13- Actual get functions to retrieve data from givenergy.

Data Format

Data	Format	Example
date	YYYY-MM-DD	The 8 th of November 2024: 2024-11-08
datetime	YYYY-MM-DD hh:mm:ss	The 8 th of November 2024 at 9:21AM: 2024-11-08 09:21:38
Battery percentage	0-100 (integer)	Charging battery to 80%: 80
Cloud coverage	0-100 (integer)	Cloud coverage for next day is 80%: 80
Serial Number	String	David's serial number is SD2245G###

Type	Description	Examples
ISO8601-type	Type is either <i>date</i> or <i>datetime</i> See this link for more information	<i>date</i> 2022-01-01 <i>datetime</i> 2022-01-01T12:34:56Z
datetime	A generic datetime, formatted YYYY-MM-DD hh:mm:ss. This data type is typically used when fetching data from endpoints that operate based on the system or user's local time	2022-01-01 06:30:00

How to use an API

A guide for my usage of APIs later

Endpoint: The URL where the API can be accessed.

Request: The action of querying the API.

Response: The data returned by the API.

HTTP Methods: Common methods include GET (retrieve data), POST (send data), PUT (update data), and DELETE (remove data).

Authentication: Many APIs require a key or token to authenticate requests

Timed Discharge (ID: 2)

This mode will trigger your battery to cover your home's electric demand only at the times you specify

This preset requires an array (or object, where the keys are the slot numbers) of slots that contain the following fields:

- *start_time* - The time that the slot should start
- *end_time* - The time that the slot should end
- *percent_limit* - The amount that this slot will discharge down to. This is not available for all slots for all inverters

For most inverters, this preset also has an *enabled* field that can be toggled to enable or disable the preset

i The number of slots available is different for each inverter and can range between 0 and 10. Call the 'Get Settings Presets' endpoint for the relevant inverter to determine the correct fields to send

i Please note that slots are 0-indexed, meaning that the first slot is slot 0, the second slot is slot 1, and so on.

Examples of setting different slots can be seen to the right:

Timed Charge (ID: 1)

This mode will trigger your battery to charge at the times you specify

This preset requires an array (or object, where the keys are the slot numbers) of slots that contain the following fields:

- *start_time* - The time that the slot should start
- *end_time* - The time that the slot should end
- *percent_limit* - The amount that this slot will charge up to. This is not available for all slots for all inverters

For most inverters, this preset also has an *enabled* field that can be toggled to enable or disable the preset

i The number of slots available is different for each inverter and can range between 0 and 10. Call the 'Get Settings Presets' endpoint for the relevant inverter to determine the correct fields to send

i Please note that slots are 0-indexed, meaning that the first slot is slot 0, the second slot is slot 1, and so on.

This piece of code gets data out of givenergy in a rather ugly form which could be searched through to extract the data in atomic form such as battery percentage. As you can see, weather data is not and cannot be retrieved from givenergy so I will have to use a different API for getting the next day's weather out. I have decided to use solarcast as it is free and gives me location specific data to help be as accurate as possible even going as far to include solar panel tilt and orientation.

[illegible]

14

Website GUI design
Log in page

The log in page is a blue rectangular box with rounded corners. At the top, it says "Solar Companion" in white. Below this, there are four input fields: "Input your serial number", an empty field, "Input your password", and another empty field. At the bottom, there is a green button with the text "GO" in white.

The user's serial number acts as a username and is also needed for the identification and accessing user's givenenergy.

User enters password to get into their account, should be 8 or more characters

When user has entered these inputs they click go to get into their account if entered incorrectly the user must reenter either or both inputs

User must input their API key so that the algorithm can use this to access the user's givenenergy

Users email should be taken as a username, password should be set and API key should be inputted as well as solar ID (serial number) to create an account that can access users givenenergy and is secure.

Sign up page

The sign up page is a blue rectangular box with rounded corners. It has four input fields: "Input email", "Input password", "Input API key", and "Input solar ID". At the bottom, there is a green button with the text "Continue" in white.

Home page

The home page is a blue rectangular box with rounded corners. At the top, it says "Solar Companion" in white. Below this, there is a grey bar with the text "API key Input/display". Below that, there is a list of variables and their values: "Battery%:", "Grid import:", "Grid export:", "Domestic use:", "Solar generation:", "Predicted total cost of today:", "Predicted savings of today:", "Tomorrows weather:", "Tomorrows predicted total costs:", and "Tomorrows predicted savings:". At the bottom, there is a grey button with the text "Go to calander" (note the spelling error in the image).

The current variables and their values from the user's givenenergy and tomorrows predicted values for the user to understand what the algorithm is doing to try to save money

Button that goes to user's solar calendar page

Calendar page

The calendar page is a blue rectangular box with rounded corners. At the top, it says "Solar Companion" in white. Below this, there is a grid representing a calendar. Below the grid, there is a grey bar with the text "Days home is empty". At the bottom, there is a grey button with the text "Back".

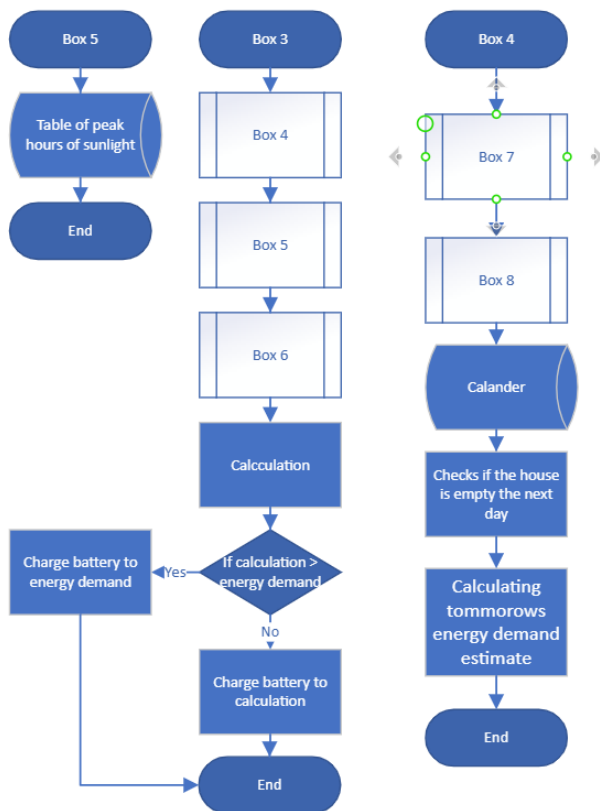
Displays user calendar where they can input and when they are away

Allows user to input when they are away to alter how much charge is put into the battery

Goes back to home screen

Flowcharts algorithm

These flowcharts are an indication of the different modules and steps the algorithm uses to save the user money. The core program stems from box 1 as it leads to all other boxes.



Box 5

This module is just for storing the table of peak solar hours for each month. To be used in later calculation.

Key variables: What month it is to be passed in as a parameter (most likely as an integer) so it can return the correct peak solar hours (will be an integer so finding range is easy or real if I decide it isn't accurate enough).

Data structures: Will contain an array of peak solar hours, position in array will correlate to (month - 1) because arrays indexes start from 0. Will contain integers.

Box 3

This module is where the actual calculation for predicted energy generation for tomorrow and what percentage to charge battery to happens.

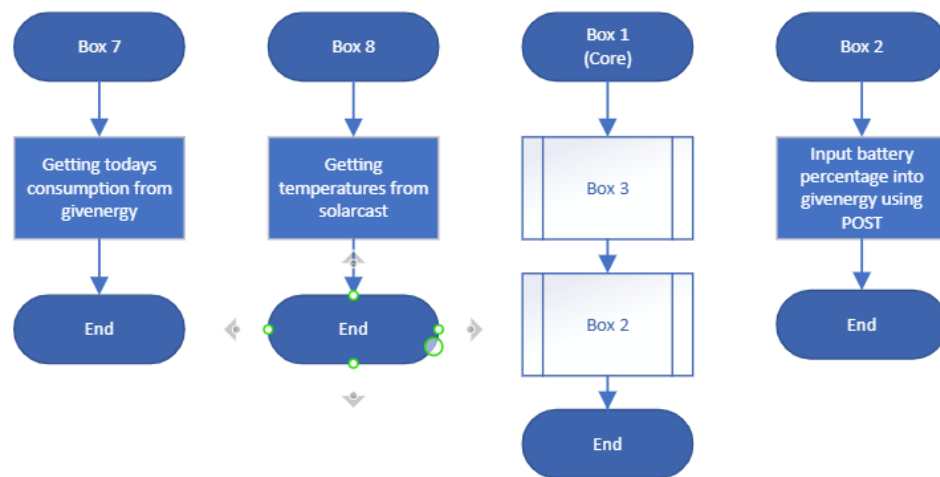
Key variables: energy demand (real number as is more accurate than integer), energy generation predicted (real number as is more accurate than integer) these two variables will be compared and if the predicted energy is larger than the energy demand, it is charged to the energy demand to leave battery free to store solar generation and if not it calculated how much battery is needed to be charged to. Final key variable is the battery percentage (integer 0 -100).

Validation: Needs to validate that battery percentage is between 0 and 100 using two simple if statements and a line that re-assigns these values to be 0 or 100 depending on which one went over.

Box 4

This module finds the predicted energy demand for tomorrow so it can be used in module three.

Key variables: Average temperature for today and tomorrow (as real numbers), demanded energy for today (as a real number) and returning predicted demanded energy for tomorrow (as a real number).



Box 7

Module 7 includes reading from givenenergy API the energy demand for today so a rough estimate for demanded energy for tomorrow can be made in module 4.

Key variables: Today's demanded energy (as a real number).

Box 8

Module 8 finds the average temperature for today and the average temperature for tomorrow as these factors affect the demanded energy, colder temperatures need more heating. Is used in module 4.

Key variables: Today's average temperature (as a real number), tomorrow's predicted average temperature (as a real number).

Box 1

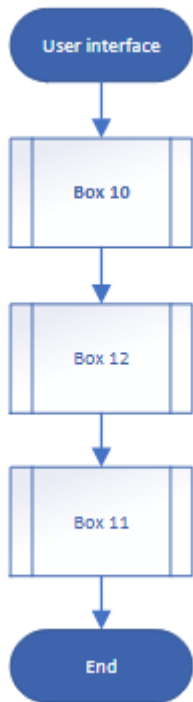
This module is responsible for calling all other modules, is the root of program. Once a battery percentage has been calculated it calls module two which will write this battery percentage into givenenergy.

Key variables: Battery percentage is returned to this module by module 3 and is passed into module 2.

Box 2

In this module the battery percentage is inputted into givenenergy.

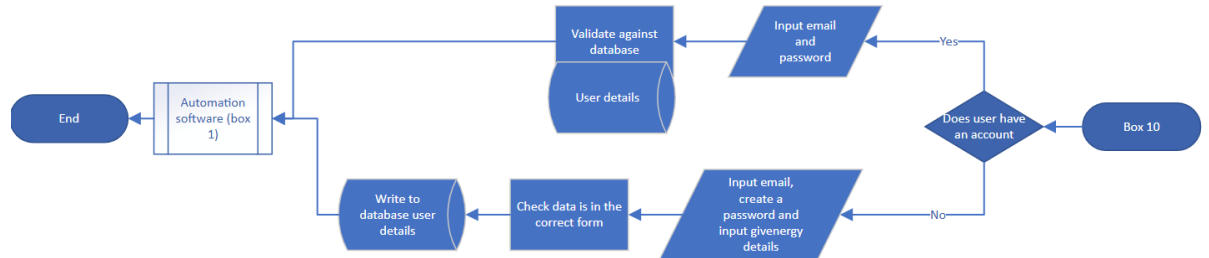
Key variables: battery percentage (as an integer).



User interface

This design defines the layout of the user interface. Uses a modular approach with each module carrying out a specific task.

Box 10



This module is necessary for allowing multiple user accounts to exist and be handles accordingly.

Key variables: user email (string data type) this is needed to act as a username and give user a domain to log in to, password (string) so that user's account can be kept secure, confirm password (string) for when signing up to software this ensures user has entered the password they think they have and didn't accidentally leave a typo in their password as this would stop them being able to sign in, API key (string) so algorithm can access user's givenery account, solar id (string) so algorithm can access the actual givenery website.

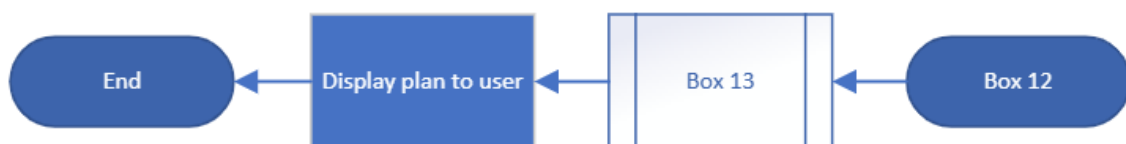
Key data structures: user details database probably will use excel, is needed to allow multiple users to log in to their accounts and retain the user details.

Validation: will need lots of validation:

When user tries to log into account, will need to search database to check if email exists in database, use a binary search and a bubble sort for finding email and check the stored password matches the value of password inputted.

When user tries to sign up the algorithm will need to check that all data inputted is in the correct form and that password matches confirm password, so user knows the password they tried to input has been inputted.

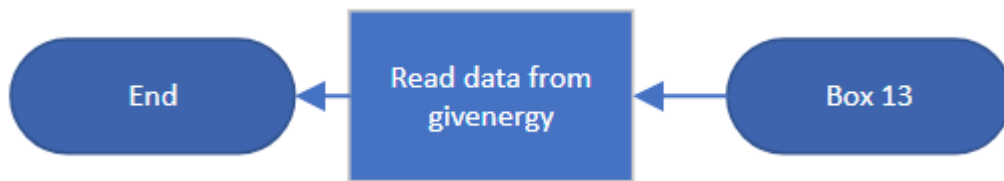
Box 12



Is needed to display users' energy plan to the user.

Key variables: Data from box 13 (likely in form of JSON as seen when experimenting with API code).

Box 13



This module will contain the code that fetches data from givenenergy, likely similar to the code in the “experimenting with API code”.

Key variables: API key (string) so algorithm can access user’s givenenergy account, solar id (string) so algorithm can access the actual givenenergy website.

Box 11



This module will contain the code for allowing user to interact with a calendar which will hopefully improve accuracy of demanded energy predictions later.

Key variables: whether user is away or home on each day, more like a field (a Boolean value that holds true for home and false for away), date that relates to whether user is home or not (a string in form YYYY-MM-DD).

Key data structures: database or 2D array that acts as a calendar, one field containing dates of the year and another field containing whether or not user home.

No.	Usability features	Justification
1	Straightforward and simple design	User can easily interpret where to find certain information and where to access different features. This will ensure they are not overwhelmed with numbers, words and images and are not put off my algorithm.
2	Different pages for different functions	So that the user can see and do all they need to for the task that they intend to do and no more. This can make it easier for user to navigate interface. For example: having separate pages for today's data, tomorrow's predictions and actions to be carried out and a calendar page for user to input when they are away.
3	Accurate and informative page titles/labels	So that the user can understand easily what different pages do, why they are needed. Accurate label names help user understand the data displayed to them. This is necessary because my algorithm will be dealing with energy in lots of different forms so will need to be clear labels so that the user doesn't get confused.
4	Graphs to display the flow of energy for that day as are shown on givenenergy portal	This is a good visualisation as to what happened to energy at certain times of the day.
5	Users can switch between pages without delay	This will increase the convenience of using the algorithm as waiting times will be reduced.

Final testing plan

During this phase I will need to test the following factors: All modules work well together and robustness/usability of program.

Specific aspects to be tested are on the next page.

Not included is that the predicted energy predictions are accurate because they take a long time to test so will be hard to test them twice. This also applies for the battery percentage testing.

Test name/purpose	Test data	Test data	Expected result
Is the login interface/sign up interface functional and can be traversed as intended. This is necessary so I can check that the user can switch between log in page and sign in page with no complications.	n/a	n/a	Can traverse in between login and sign-up page as please. Background changes size to fit size of frame. Doesn't allow for no input to be attempted. Can type in all entry boxes.
Does login page allow an email that is stored in database to be inputted with correct password and proceed accordingly. Necessary so I can check that users can log in to their created account.	Email: BOMP@NEA.co.uk	Password: 1234	Valid, displays given energy data
Does login page correctly deny access to invalid attempts. Necessary so I can check user can't accidentally log in to an account that doesn't exist by having a typo in their email.	Emails: BOMP@NEE.co.uk BOMP@NEA.co.uk No input	Password: 1234 1233 No input	Invalid. No account detected Invalid password Enter valid email
Does sign up page function as designed to with valid inputs. Necessary to check that user can create an account when they input valid details.	Email: wilf5@email.co.uk Usual API key and solar id	Password: 1234 Confirm password: 1234	Goes to welcome page, displays data, can go to prediction page which also displays data, users' details are stored in data base. Can log in with correct details
Does sign up page reject invalid inputs when email is not a real email. Necessary so can check that users can't sign up with false emails in case they need to be contracted in form of validation or resetting password.	Email: wilf@notemail Normal API key Normal solar id	Password: 1234 Confirm Password: 1234	Please enter valid email message displayed
Does sign up page reject invalid inputs. Passwords don't match. Necessary to check that when user accidentally mis enters password, algorithm lets them retry.	Email: wilf@email.co.uk Normal API key Normal solar id	Password: 1234 Confirm password: 123	Please make sure passwords match error message displayed
Does sign up page reject invalid inputs. Presence checks for API	Email: wilf@email.co.uk	Password: 1234	Doesn't let through to welcome page

key and solar id. Necessary to check user can't sign up without inputting their given energy details	No API key No solar id	Confirm password: 1234	
Does sign up page reject invalid inputs. Presence checks for email. Necessary to check that user can't create an account with no email as the email is what is used to identify account, acts as primary key.	Email: None Normal API key Normal solar id	Password: 1234 Confirm password: 1234	Doesn't let through when there is nothing in email entry box
Does sign up page reject invalid inputs. Presence checks for passwords. Necessary to check that user can't sign up without passwords as if could, this would not be secure.	Email: Wilf5@email.co.uk Normal API key Normal solar id	Password: None Confirm: None	Doesn't let user into welcome page
Can user input when they are away and can this be taken into account in energy predictions. Necessary to check is taken into account so predicted demanded energy is more accurate.	Away for the next two days	n/a	Battery is charged to less than usually would be for next two days.
Are graphs properly displayed and easy to interpret. Necessary to check all graphs are readable, there are no errors and user can see what is happening/happened in their energy plan at any given moment.	n/a	n/a	Graphs are clear, high contrast, easy to read and axes are properly named.
Robust test. Necessary to check can change window size in case the user decides to change window size, and that the algorithm won't crash.	Changing size of window quickly	n/a	Quality of picture is unaffected and is not laggy.
Robust test. Necessary to check that the algorithm won't crash if the user clicks buttons too quickly.	Spamming of buttons	n/a	There is no difference in speed to access different pages. Doesn't crash.

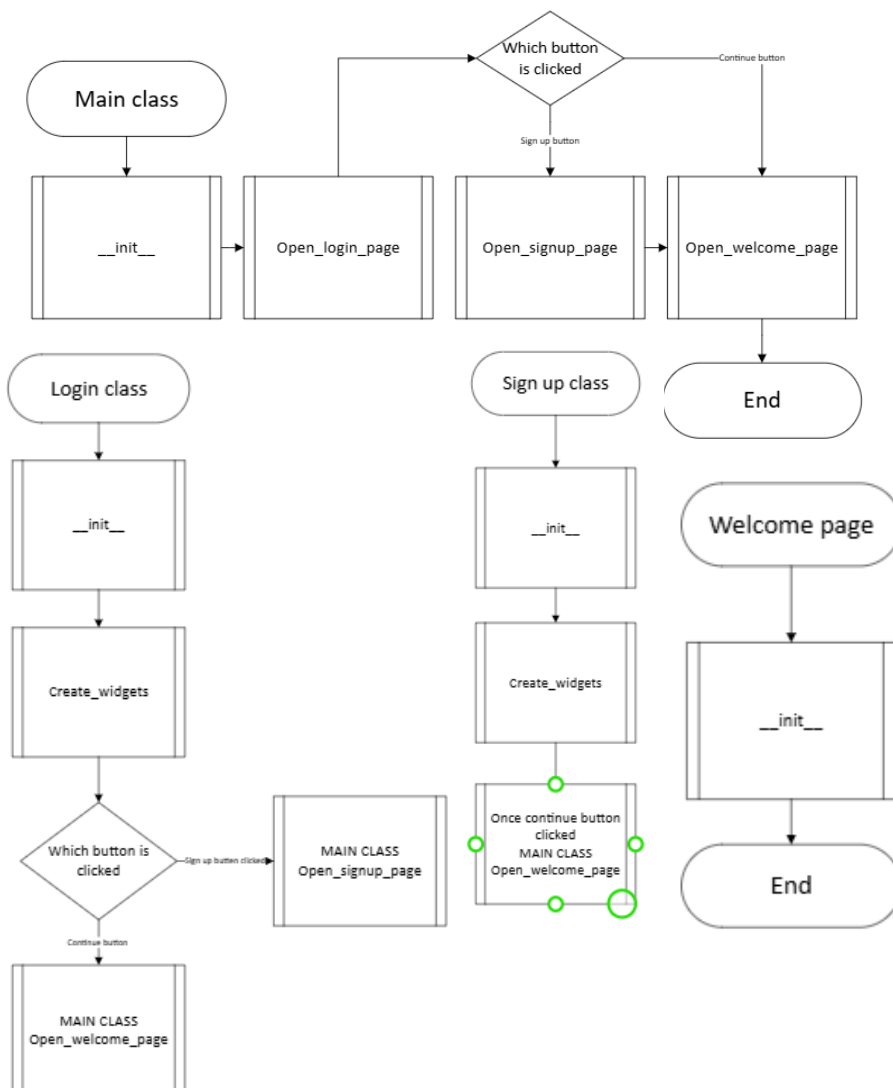
GUI iteration 1

GUI design 1

To start with I decided to design 3 pages including the first page, a log in page, a sign in page and a welcome page to lay the foundations of my GUI. I decided to program my GUI using OOP to make my code reusable when I decide to add more details and/or features, it also helps me to split up the code into modules to make the code neater and easier to understand.

I will need a main controller class that co-ordinates the transfer between pages.

I will need 4 different classes for 3 different pages and the main class.



This is a design for the controller class that co-ordinates the opening and closing of frames in between pages.

Below are designs for the rest of the classes, in the first iteration there will be extremely limited functionality, all GUI will do is switch between frames when needed. The create widgets methods create labels, entry labels and buttons for the user to interact with.

GUI Development 1

To create the main window in tkinter I need a tk.Tk() application and a mainloop() to keep the window open and responsive.

Creates the tkinter window and assigns it to a variable named root

```
112 root = tk.Tk()
113 app = Main(root)
114 root.mainloop()
115
```

Creates an instance of a Main class and passes in variable root to this instance.

Starts the tkinter main loop to keep the application running

Using these basics I programmed these classes.

```
class Main:
    def __init__(self, root):
        self.root = root
        self.current_frame = None
        self.open_login_page()

    def open_login_page(self):
        if self.current_frame:
            self.current_frame.frame.destroy()
        self.current_frame = LoginPage(self.root, self)

    def open_signup_page(self):
        if self.current_frame:
            self.current_frame.frame.destroy()
        self.current_frame = SignupPage(self.root, self)

    def open_welcome_page(self, email):
        if self.current_frame:
            self.current_frame.frame.destroy()
        self.current_frame = WelcomePage(self.root, email)
```

an instance of my SignupPage class and
open_welcome_page

Line 89 defines the class main

Special method __init__ initializes an object of the class, sets the current_frame to none and opens the login page using the open_login_page method

The open_login_page method destroys the current frame although no frames should exist at this stage (this will be needed later) and creates an instance of my LoginPage class

The open_signup_page method is nearly identical but when destroys the current frame there is likely a frame to destroy and it creates a similar case for the method

```
class LoginPage:
    def __init__(self, root, app):
        self.root = root
        self.app = app
        self.frame = tk.Frame(self.root)
        self.frame.pack()

        self.create_widgets()

    def create_widgets(self):
        # Email label and entry
        self.email_label = tk.Label(self.frame, text="Email:")
        self.email_label.grid(row=0, column=0, padx=10, pady=10)
        self.email_entry = tk.Entry(self.frame)
        self.email_entry.grid(row=0, column=1, padx=10, pady=10)

        # Password label and entry
        self.password_label = tk.Label(self.frame, text="Password:")
        self.password_label.grid(row=1, column=0, padx=10, pady=10)
        self.password_entry = tk.Entry(self.frame, show="*")
        self.password_entry.grid(row=1, column=1, padx=10, pady=10)

        # Continue button
        self.continue_button = tk.Button(self.frame, text="Continue", command=self.open_welcome_page)
        self.continue_button.grid(row=2, column=0, columnspan=2, pady=10)

        # Sign Up button
        self.signup_button = tk.Button(self.frame, text="Sign Up", command=self.open_signup_page)
        self.signup_button.grid(row=3, column=0, columnspan=2, pady=10)
```

Defines my class login page

Special method `__init__` initializes an instance of class `LoginPage`, `self.app = app` provides access to main application instance. `Self.frame = tk.Frame(self.root)` creates a frame widget which will contain other widgets, `self.frame.pack()` calls `pack()` method which is a geometry manager in Tkinter. `Self.create_widgets()` calls the `create_widgets()` method which creates the widgets that are displayed on this page.

```
        # Sign Up button
        self.signup_button = tk.Button(self.frame, text="Sign Up", command=self.open_signup_page)
        self.signup_button.grid(row=3, column=0, columnspan=2, pady=10)

    def open_welcome_page(self):
        self.app.open_welcome_page(self.email_entry.get())

    def open_signup_page(self):
        self.app.open_signup_page()
```

Extention of my class login page includes methods `open_welcome_page` and `open_signup_page`

`Self.app` refers to the main application instance passed to this class `open_welcome_page` is a method in the `app` instance and passes the users email into the class that is being opened. `Open_signup_page` refers to another method in the `app` instance and doesn't need to pass anything into this class that is being opened.

```
class SignupPage:
    def __init__(self, root, app):
        self.root = root
        self.app = app
        self.frame = tk.Frame(self.root)
        self.frame.pack()

        self.create_widgets()

    def create_widgets(self):
        email_label = tk.Label(self.frame, text="Email:")
        email_label.grid(row=0, column=0, padx=10, pady=10)
        email_entry = tk.Entry(self.frame)
        email_entry.grid(row=0, column=1, padx=10, pady=10)

        password_label = tk.Label(self.frame, text="Password:")
        password_label.grid(row=1, column=0, padx=10, pady=10)
        password_entry = tk.Entry(self.frame, show="*")
        password_entry.grid(row=1, column=1, padx=10, pady=10)

        confirm_password_label = tk.Label(self.frame, text="Confirm Password:")
        confirm_password_label.grid(row=2, column=0, padx=10, pady=10)
        confirm_password_entry = tk.Entry(self.frame, show="*")
        confirm_password_entry.grid(row=2, column=1, padx=10, pady=10)

        api_key_label = tk.Label(self.frame, text="API Key:")
        api_key_label.grid(row=3, column=0, padx=10, pady=10)
        api_key_entry = tk.Entry(self.frame)
        api_key_entry.grid(row=3, column=1, padx=10, pady=10)

        solar_id_label = tk.Label(self.frame, text="Solar ID:")
        solar_id_label.grid(row=4, column=0, padx=10, pady=10)
        solar_id_entry = tk.Entry(self.frame)
        solar_id_entry.grid(row=4, column=1, padx=10, pady=10)

        continue_button = tk.Button(self.frame, text="Continue", command=lambda: self.app.open_welcome_page(email_entry.get()))
        continue_button.grid(row=5, column=0, colspan=2, padx=10, pady=10)
```

Defines my class SignupPage

Special method `__init__` initializes an instance of my class, the rest is a very similar design to my LoginPage class except with a few more widgets for things like API key and Solar ID that will be needed later. The difference in continue button is due to the fact that in the SignupPage class the continue button passes a parameter into the welcomepage class and lambda allows more flexibility when passing in parameters

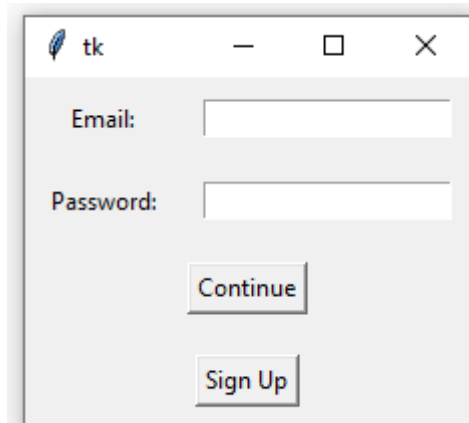
```
class WelcomePage:
    def __init__(self, root, email):
        self.root = root
        self.frame = tk.Frame(self.root)
        self.frame.pack()

        welcome_label = tk.Label(self.frame, text=f"Welcome, {email}!")
        welcome_label.pack(padx=20, pady=20)
```

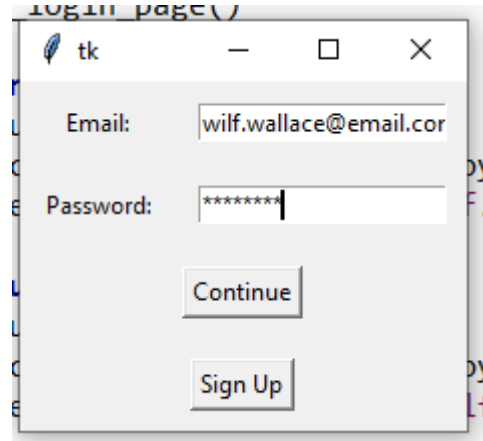
Similar layout to all my other classes, at the moment doesn't need to be able to call any of my main class methods from here but will in the future when add more frames/pages to code so should also have app passed into it so I will update it in the next iteration.

GUI Testing 1

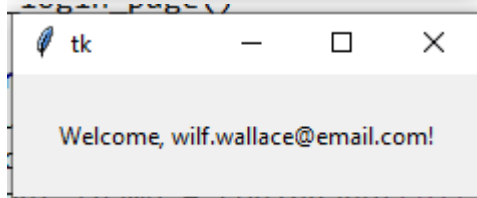
Login route- works well, next iteration should include a show password button. Doesn't look very interesting so should add a background image in next iteration. Doesn't actually check if password matches email, should fix in the next iteration.



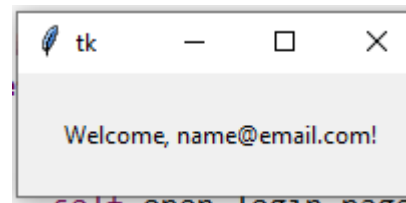
A screenshot of a Tkinter window titled 'tk'. It contains two text input fields: 'Email:' and 'Password:'. Below the 'Password:' field are two buttons: 'Continue' and 'Sign Up'.



A screenshot of the same Tkinter window. The 'Email:' field now contains 'wilf.wallace@email.com' and the 'Password:' field contains '*****'. The 'Continue' and 'Sign Up' buttons are still present.

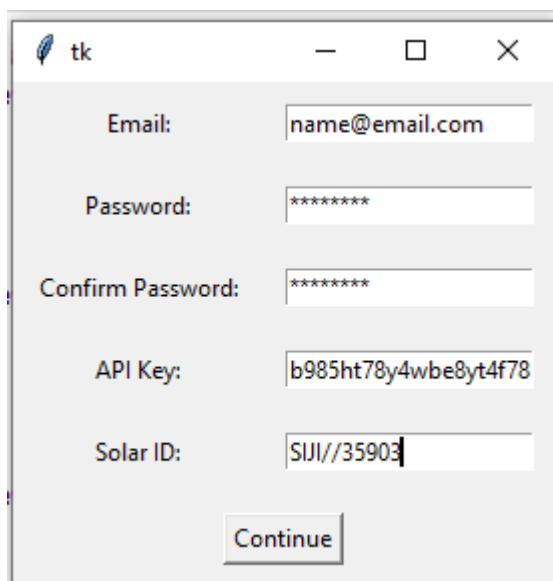


A screenshot of the Tkinter window showing a welcome message: 'Welcome, wilf.wallace@email.com!'.



A screenshot of the Tkinter window showing a welcome message: 'Welcome, name@email.com!'.

Signup route- similar observations, quite boring, doesn't actually store any details user inputs yet. Should fix in the next iteration.



A screenshot of a Tkinter window titled 'tk'. It contains five text input fields: 'Email:' (name@email.com), 'Password:' (*****), 'Confirm Password:' (*****), 'API Key:' (b985ht78y4wbe8yt4f78), and 'Solar ID:' (SIJI//35903). Below the 'Solar ID:' field is a 'Continue' button.

GUI Evaluation 1

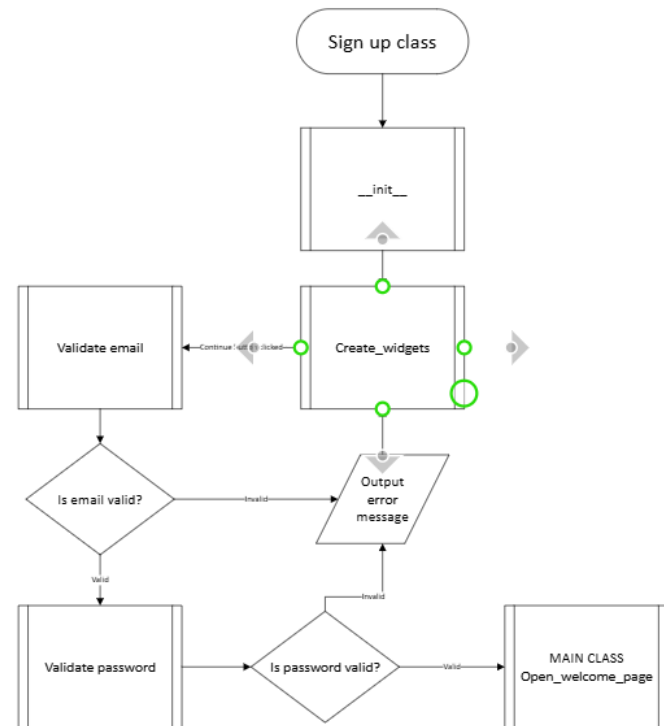
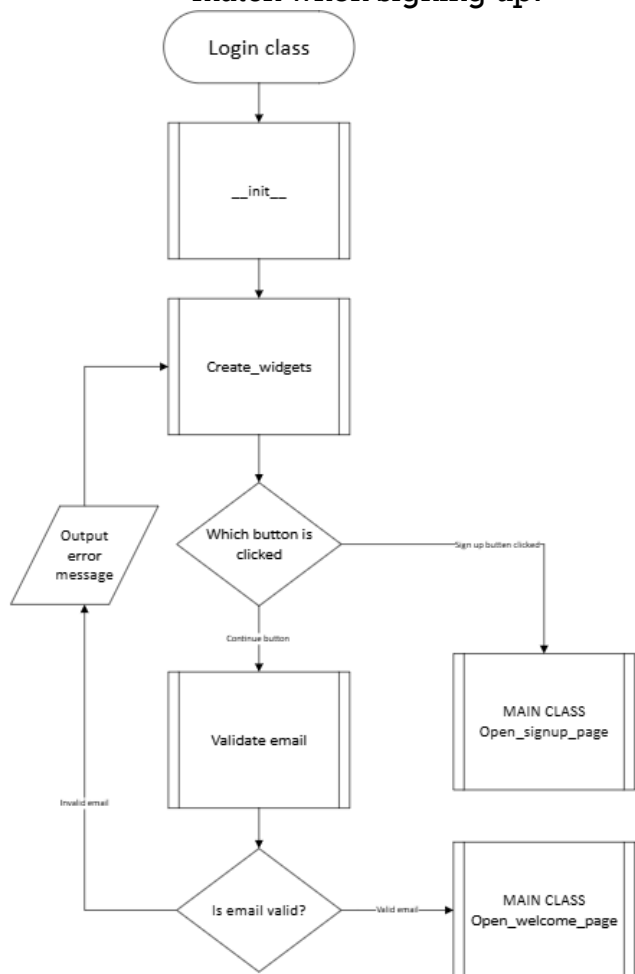
My code works well at this basic level but does not actually store any data or validate any of the data the user inputs so in my next iteration I should set up validating functions.

GUI iteration 2

GUI design 2

This iteration will aim to ensure user has entered a valid email address when logging in and signing up. Will also ensure password and confirm password match when signing up.

These flow charts demonstrate the changes that will be made to the flow of the algorithm. Whenever an entity is invalid the user gets to re enter the entity in the create widgets method.



Test plan

Data is to be validated, so I need a test plan.

Test #	Test name/purpose	Expected result
1	The @ in users' email is in a correct position, there is letters in between @ and the first dot and there are letters after the first dot and there are letters before the @. Test that a valid email is accepted in sign-up page. Test: COMP@NEA.co.uk	When valid email with correct @ and dot positioning is inputted, user is accepted from sign-up page into the welcome page.

2	Check that email that has no letters before the @ is rejected in sign-up page. Test: @NEA.co.uk	Reject, error message is displayed, user can retry to enter a new email.
3	Check that @ needs dots after it to be accepted in sign-up page. Test: COMP.NEA@couk	Reject, error message is displayed, user can retry to enter a new email.
4	Check that when passwords match, the user can proceed to welcome page. Test: 1234, 1234	Accepted, proceed to the welcome page.
5	Check that rejects passwords that don't match. Test: Comp, CComp	Rejected, error message, user can retry entering new passwords.
6	Check that rejects passwords that don't match, not just capitals. Test: Comp, Nea!	Rejected, error message, user can retry entering new passwords.

GUI development 2

```

A GUI (1).py
C: > Users > wilfw > Downloads > A GUI (1).py > ...
5 class LoginPage:
33
34     def validate_email(self, email):
35         if "@" in email and "." in email:
36             at_index = email.index("@")
37             dot_index = email.rindex(".")
38             if at_index < dot_index and at_index > 0 and dot_index < len(email) - 1:
39                 return True
40             return False

```

Added function to class LoginPage to check that not only "@" and "." are in the users email when trying to login but they are in a valid position, e.g. @ is not at start, @ is before a dot and the dot is not at the end of the email.

If email is valid, it returns value True to be used when called later and False if not valid.

```

A GUI (1).py
C: > Users > wilfw > Downloads > A GUI (1).py > LoginPage > create_widgets
5 class LoginPage:
42     def open_welcome_page(self):
43         email = self.email_entry.get()
44         if self.validate_email(email):
45             self.app.open_welcome_page(self, email)
46         else:
47             messagebox.showerror("Invalid Email", "Please enter a valid email address.")
48
49     def open_signup_page(self):
50         self.app.open_signup_page()

```

Validate_email is used in the open_welcome_page function and if true opens the welcome page and if false returns a message box that asks the user to enter a valid email address.

```
A GUI (1).py X
C: > Users > wilfw > Downloads > A GUI (1).py > LoginPage > open_welcome_page
52 class SignupPage:
53
89     def validate_email(self, email):
90         if "@" in email and "." in email:
91             at_index = email.index("@")
92             dot_index = email.rindex(".")
93             if at_index < dot_index and at_index > 0 and dot_index < len(email) - 1:
94                 return True
95             return False
96
```

I added in the same function for validating an email address into my signupPage class.

```
A GUI (1).py X
C: > Users > wilfw > Downloads > A GUI (1).py > SignupPage > open
52 class SignupPage:
53
97     def validate_password(self, password, confirmp):
98         if password == confirmp:
99             return True
100         return False
101
```

I also added a validate password function into my class SignupPage class to ensure the users password and conform password match up.

```
A GUI (1).py X
C: > Users > wilfw > Downloads > A GUI (1).py > SignupPage > open_welcome_page
52 class SignupPage:
53
100         return False
101
102     def open_welcome_page(self):
103         email = self.email_entry.get()
104         password = self.password_entry.get()
105         confirmp = self.confirm_password_entry.get()
106         if self.validate_email(email) and self.validate_password(password, confirmp):
107             self.app.open_welcome_page(self, email)
108         else:
109             if self.validate_email(email):
110                 messagebox.showerror("Invalid Email", "Please enter a valid email address.")
111             elif self.validate_password(password, confirmp):
112                 messagebox.showerror("Passwords don't match", "Please make sure passwords match.")
113
```

My function open_welcome_page in my SignupPage class needed a lot of modifying to call the two new functions and to use get methods to retrieve the password, conform password and email from the instance of this class. Then I needed to use some variation of if statements to check if the password entry matched the confirm password entry and the email was valid and if not figure out which one is untrue and produce an error message to inform the user.

Test #	Test name/purpose	Test data Email	Test data Password	Test data Confirm password	Expected result	Actual result
1	@ positioning, V	COMP@NEA.co.uk	1234	n/a	V->WP	V->WP
2	@ positioning, IV	@NEA.co.uk	1234	n/a	IV->MB	IV->MB
3	@/. positioning, IV	COMP.NEA@couk	1234	n/a	IV->MB	IV->MB
4	Passwords matching V	COMP@NEA.co.uk	1234	1234	V->WP	V->WP
5	Passwords matching IV	COMP@NEA.co.uk	Comp	COmp	IV->MB	IV-> wrong message box
6	Passwords matching IV	COMP@NEA.co.uk	Comp	Nea!	IV->MB	IV-> wrong MB

GUI testing 2

For this testing iteration I should check that my validation functions hold up and user can't enter invalid values but can enter valid values. During this iteration I am only testing that the emails are valid and passwords have to match up in the sign up page.

MB = message box WP = welcome page V = valid IV =invalid

Test #1

Test #2

Test #3

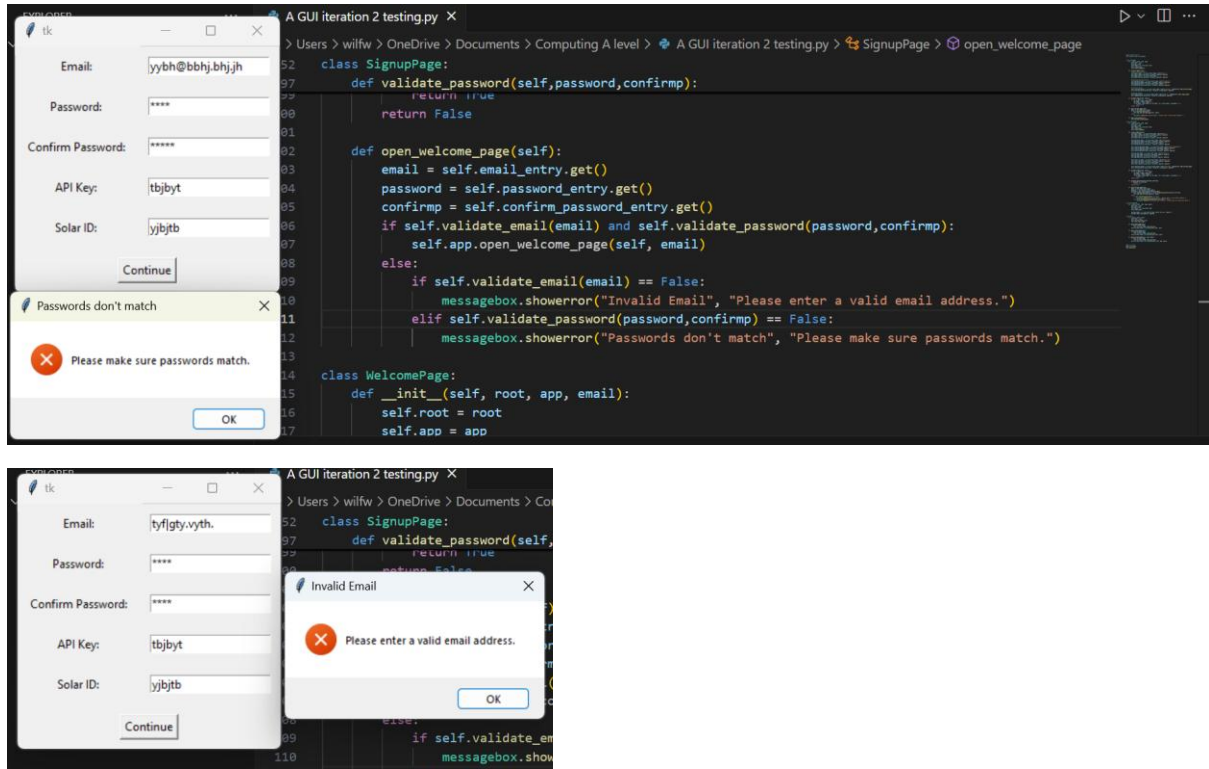
Test #4

Test #5

Test #6

Debugging GUI 2

Fixing problems that caused test 5 and 6 to fail this was a quick fix as I just had to add ==False to end of if statement to flip why and when message boxes occur and as you can see has fixed the problem.



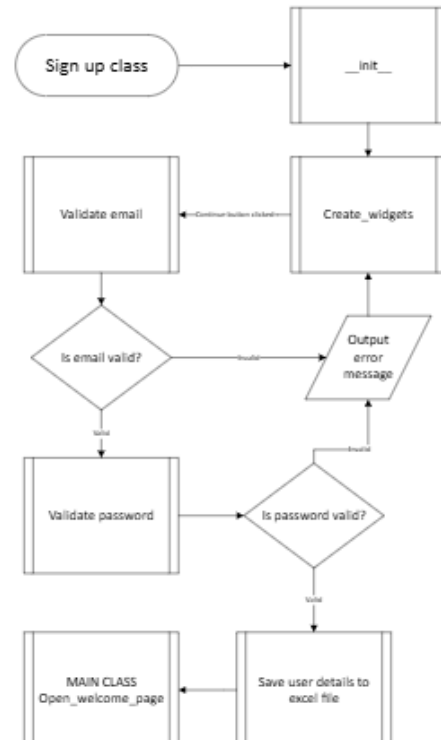
Evaluation 2

The validation methods now work effectively and make sure no one can input invalid emails or accidentally input wrong password when signing up. The next iteration should loom to store user details and not allow people without accounts to log in.

GUI iteration 3

GUI design 3

I will be using an excel file acting as a database to store the user's details and will link it to my program which will require the pandas library. I will also need the os library to read from and write to files. This doesn't require much change to the overall flow of the algorithm but will require me to learn how to write to an excel file.

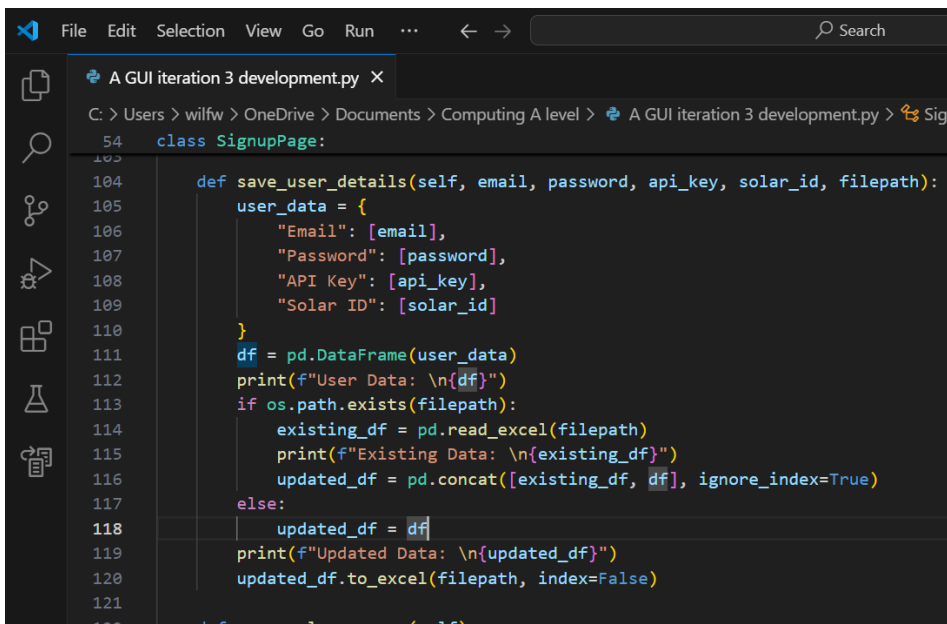


Test plan

Need to test sign-up page and that it writes inputs to database.

Test #	Test name/purpose	Expected result
1	Tests for when correct email and matching passwords are inputted. Test data: Test4@nea.co.uk , 1234, 1234	Welcome page is displayed, data is saved.
2	Tests for when invalid email is inputted with matching passwords. Test data: Test5nea.co.uk, 1234, 1234	Message box is displayed, data is not saved, user can try again, if valid data is then saved.
3	Tests for when valid email is entered but not matching passwords. Test data: Test6@nea.co.uk , 1234, 123	Message box is displayed, data is not saved, user can try again, if valid data is then saved.
4	Tests for when invalid email and not matching passwords are inputted. Test data: Test7@nea, 1234, 123	Message box is displayed, data is not saved, user can try again, if valid data is then saved.

GUI development 3



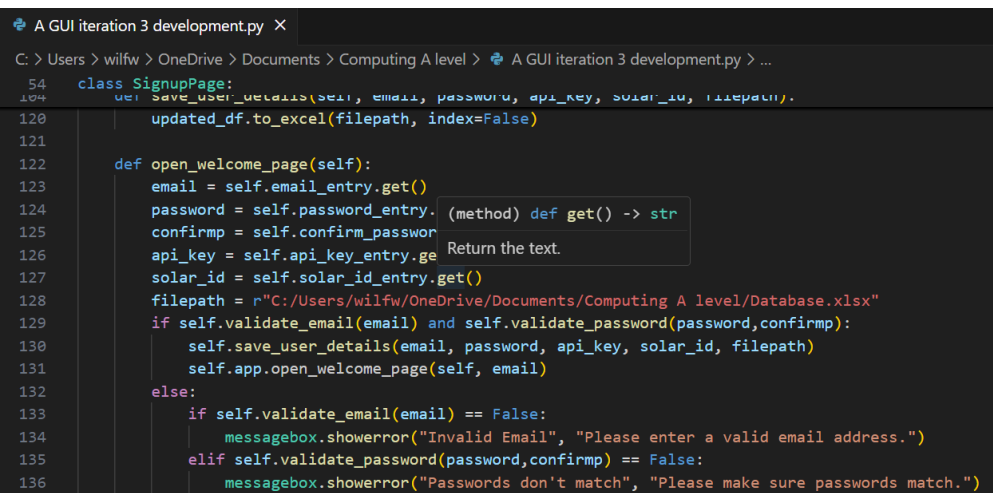
```

54 class SignupPage:
104     def save_user_details(self, email, password, api_key, solar_id, filepath):
105         user_data = {
106             "Email": [email],
107             "Password": [password],
108             "API Key": [api_key],
109             "Solar ID": [solar_id]
110         }
111         df = pd.DataFrame(user_data)
112         print(f"User Data: \n{df}")
113         if os.path.exists(filepath):
114             existing_df = pd.read_excel(filepath)
115             print(f"Existing Data: \n{existing_df}")
116             updated_df = pd.concat([existing_df, df], ignore_index=True)
117         else:
118             updated_df = df
119         print(f"Updated Data: \n{updated_df}")
120         updated_df.to_excel(filepath, index=False)
121

```

To save users sign up data to an excel file I needed another method in my signup page class that takes in all of the details user inputs apart from confirmpassword as this doesn't need to be saved. First the data is stored in a dictionary with keys that specify what section data should be put in the database. The dictionary is then converted to a pandas dataframe. Then my code

checks if the file exists and reads the existing data in the file into a dataframe then the existing data and the new data is concatenated into an "updated dataframe" to combine the rows from both dataframes. If the file doesn't exist it uses the new dataframe as the "updated dataframe". The updated dataframe is then saved to an excel file. Index=False makes sure row indices are not written to the file. The print statements were used to help debug my program and identify any logic errors during development.



```

54 class SignupPage:
104     def save_user_details(self, email, password, api_key, solar_id, filepath):
120         updated_df.to_excel(filepath, index=False)
121
122     def open_welcome_page(self):
123         email = self.email_entry.get()
124         password = self.password_entry.get()
125         confirmp = self.confirm_password_entry.get()
126         api_key = self.api_key_entry.get()
127         solar_id = self.solar_id_entry.get()
128         filepath = r"C:/Users/wilfw/OneDrive/Documents/Computing A level/Database.xlsx"
129         if self.validate_email(email) and self.validate_password(password, confirmp):
130             self.save_user_details(email, password, api_key, solar_id, filepath)
131             self.app.open_welcome_page(self, email)
132         else:
133             if self.validate_email(email) == False:
134                 messagebox.showerror("Invalid Email", "Please enter a valid email address.")
135             elif self.validate_password(password, confirmp) == False:
136                 messagebox.showerror("Passwords don't match", "Please make sure passwords match.")

```

The save data method is called in my open welcome page method, and the file path is also established here. All details are also "got"/fetched here to be passed into the save user details method. User details are only saved if email and passwords are valid.

GUI testing 3

In this test I need to test that user data gets saved when and only when passwords match and email is valid. This is a form of white box testing.

Test #	Test name/purpose	Test data Email	Test data Password	Test data Confirm password	Expected result	Actual result
1(4)	Correct email and matching passwords	Test4@nea.co.uk	1234	1234	Welcome page, saved data	Welcome page, saved data
2(5)	Invalid email and matching passwords	Test5nea.co.uk	1234	1234	Message box, no saved data, try again, saved data	Message box, no saved data, try again, saved data
3(6)	Valid email and not matching passwords	Test6@nea.co.uk	1234	123	Message box, no saved data, try again, saved data	Message box, no saved data, try again, saved data
4(7)	Invalid email and not matching passwords	Test7@nea	1234	123	Message box, no saved data, try again, saved data	Message box, no saved data, try again, saved data

Test 1

	A	B	C	D	E
1	Email	Password	API Key	Solar ID	
2	COMP@N	1234	1234	1234	
3	test3@ne	1234	1234	1234	
4					
5					
6					

	A	B	C	D	E
1	Email	Password	API Key	Solar ID	
2	COMP@N	1234	1234	1234	
3	test3@ne	1234	1234	1234	
4	test4@ne	1234	1234	1234	
5					
6					
7					

Test 2

The screenshot shows a spreadsheet with the following data:

	A	B	C	D	E
1	Email	Password	API Key	Solar ID	
2	COMP@N	1234	1234	1234	
3	test3@ne	1234	1234	1234	
4	test4@ne	1234	1234	1234	

Overlaid on the spreadsheet is a Tkinter window titled 'tk' with the following fields:

- Email: Test5@nea.co.uk
- Password: (masked with asterisks)
- Confirm Password: (masked with asterisks)
- API Key: 1234
- Solar ID: 1234
- Continue button

In the background, a code editor shows Python code for a Tkinter application, including a 'SignupPage' class and a 'Welcome' message.

The screenshot shows a spreadsheet with the following data:

	A	B	C	D	E
1	Email	Password	API Key	Solar ID	
2	COMP@N	1234	1234	1234	
3	test3@ne	1234	1234	1234	
4	test4@ne	1234	1234	1234	

Overlaid on the spreadsheet is a Tkinter window titled 'tk' with the following fields:

- Email: Test5@nea.co.uk
- Password: (masked with asterisks)
- Confirm Password: (masked with asterisks)
- API Key: 1234
- Solar ID: 1234
- Continue button

In the background, a code editor shows Python code for a Tkinter application, including a 'SignupPage' class and a 'Welcome' message.

Test 3

The screenshot shows a spreadsheet with the following data:

	A	B	C	D	E
1	Email	Password	API Key	Solar ID	
2	COMP@N	1234	1234	1234	
3	test3@ne	1234	1234	1234	
4	test4@ne	1234	1234	1234	
5	Test5@ne	1234	1234	1234	

Overlaid on the spreadsheet is a Tkinter window titled 'tk' with the following fields:

- Email: Test5@nea.co.uk
- Password: (masked with asterisks)
- Confirm Password: (masked with asterisks)
- API Key: 1234
- Solar ID: 1234
- Continue button

In the background, a code editor shows Python code for a Tkinter application, including a 'SignupPage' class and a 'Welcome' message.

Test 4

The screenshot shows a spreadsheet with the following data:

	A	B	C	D	E
1	Email	Password	API Key	Solar ID	
2	COMP@N	1234	1234	1234	
3	test3@ne	1234	1234	1234	
4	test4@ne	1234	1234	1234	
5	Test5@ne	1234	1234	1234	
6	Test6@ne	1234	1234	1234	

Overlaid on the spreadsheet is a Tkinter window titled 'tk' with the following fields:

- Email: Test7@nea.co.uk
- Password: (masked with asterisks)
- Confirm Password: (masked with asterisks)
- API Key: 1234
- Solar ID: 1234
- Continue button

In the background, a code editor shows Python code for a Tkinter application, including a 'SignupPage' class and a 'Welcome' message.

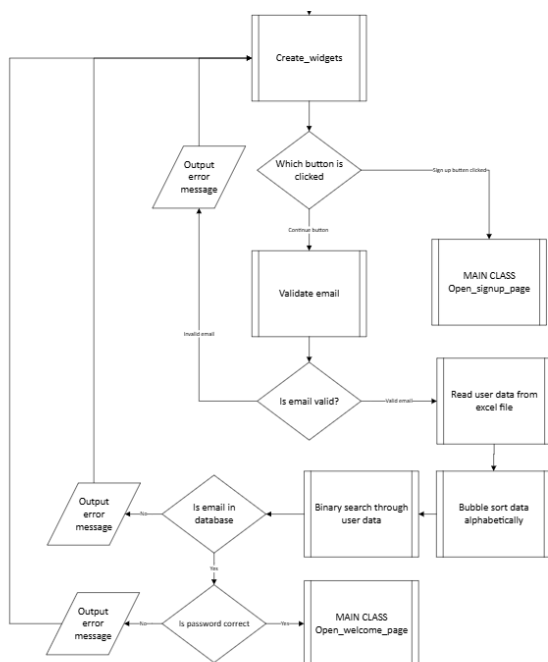
GUI evaluation 3

My sign-up page saves and stores user details effectively when excel file is not in use. The next step for my code is for users to login to accounts stored in database and be denied access when they try to use an email that is not in database or/and a password that does not correspond to email inputted.

GUI iteration 4

GUI design 4

For this I will need to be able to read from my excel file in my login page class and have a search algorithm search for inputted email and compare the inputted password to the correct password. I will use a binary search algorithm so I will need to read in my file and sort it alphabetically, this will be quicker than a linear search algorithm. My code will have to be sorted to be binary searched, to sort my code I should use a bubble sort as it is fairly efficient for small databases, as my database is not expected to be very large, not an extremely large target market.



This updated flow chart shows the new subroutines needed and the new, more complex flow of the algorithm that is needed to check users log in details against details in algorithm.

Test plan

I need to test that my algorithm can correctly read from database and make the correct decisions.

Test #	Test name/purpose	Expected result
1	Test for when correct email and password are entered as are stored in database.	Displays the welcome page to the user.
2	Test for when correct email is used from the database but a different password to the one stored in the database is used.	Displays error message, user retries with correct password, displays welcome page.
3	Tests for when an email that is not in the database is inputted.	Displays error message, user retries with correct email and password, displays welcome page.

GUI development 4

```
27
28     # Continue button
29     self.continue_button = tk.Button(self.frame, text="Continue", command=self.check_login_details)
30     self.continue_button.grid(row=2, column=0, columnspan=2, pady=10)
31
```

First, I redirected what happened when user clicks the continue button to go to method `check_login_details` where once login details are confirmed welcome page will be displayed.

```
81     def check_login_details(self):
82         email = self.email_entry.get()
83         password = self.password_entry.get()
84         filepath = r"C:/Users/wilfw/OneDrive/Documents/Computing A level/Database.xlsx"
85         print(self.read_user_data(filepath))
86         if self.validate_email(email):
87             user_data = self.read_user_data(filepath)
88             user_index = self.binary_search(user_data, email)
89             if user_index != -1 and user_data.loc[user_index, 'Password'] == password:
90                 self.app.open_welcome_page(self, email)
91             else:
92                 messagebox.showerror("Login Failed", "Incorrect email or password.")
93         else:
94             messagebox.showerror("Invalid Email", "Please enter a valid email address.")
95
```

Next, I wrote my method that checks the user login details. It gets the users email input and their password input and the filepath is defined inside this method. It then prints the user's data from the `read_user_data` which will be sorted, this helps me make sure my methods are working. If email is valid, it reads the `users_data` into a variable and passes it into a binary search along with users inputted email and saves the output to variable `user_index`. If the email is not in the database messagebox shows an error of incorrect email or password. If email is not valid message box says to enter a valid email address. If email is in database (and so will be valid) then welcome page is opened.

```
def read_user_data(self, filepath):
    if os.path.exists(filepath):
        df = pd.read_excel(filepath)
        return self.sort_user_data(df)
    else:
        return pd.DataFrame(columns=["Email", "Password", "API Key", "Solar ID"])
```

I then needed to make my method that reads the user data into a pandas data frame and returns it sorted by using my sort user data method and if the file path has changed or doesn't exist anymore it returns the collum titles with no data.

```

59     def sort_user_data(self, df):
60         n = len(df)
61         for i in range(n):
62             for j in range(0, n-i-1):
63                 if df.loc[j, 'Email'] > df.loc[j+1, 'Email']:
64                     jadd1temp = df.iloc[j+1]
65                     jtemp = df.iloc[j]
66                     df.iloc[j], df.iloc[j+1] = jadd1temp, jtemp
67         return df.reset_index(drop=True)

```

My method sort user data receives my database as a data frame. It finds the length (number of rows) of my data frame and the outer loop runs n times which represents each pass through the list. The inner loop is used to perform comparisons and swaps for that pass. It iterates from 0 to n-i-1 to ensure that inner loop does not compare elements that have already been compared. As i increases, the number of iterations inner loop does decreases as the end (highest) elements have already been sorted. The outer loop then iterates through the rest of the loop and finishes the sort.

```

69     def binary_search(self, df, email):
70         low = 0
71         high = (variable) low: int
72         while low <= high:
73             mid = (low + high) // 2
74             if df.loc[mid, 'Email'] == email:
75                 return mid
76             elif df.loc[mid, 'Email'] < email:
77                 low = mid + 1
78             else: high = mid - 1
79         return -1

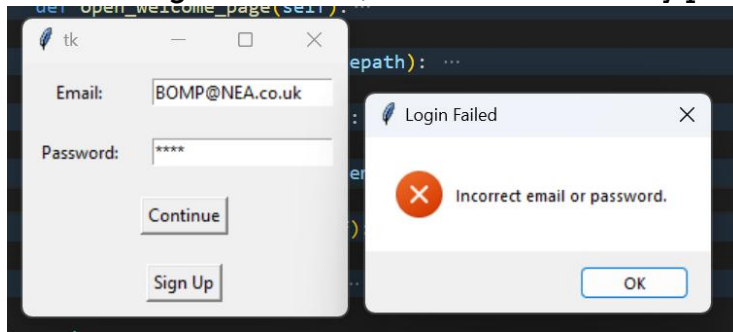
```

The next method to be made is my binary search algorithm which uses the location keyword to find the location index of emails in database and run a generic binary search to find the users email, if users email is not in database -1 is returned

if users email is in database the location of it is returned.

GUI Testing and development 4

When doing a basic test, I discovered that my program didn't actually work so I need to debug my code and find where it is going wrong.



```
7 class LoginPage:
81 def check_login_details(self):
84     filepath = r"C:/Users/wilfw/OneDrive/Documents/Computing A level/Database.xlsx"
85     print(self.read_user_data(filepath))
86     if self.validate_email(email):
87         user_data = self.read_user_data(filepath)
88         print("UserData: \n", user_data)
89         user_index = self.binary_search(user_data, email)
90         print("User Index: ", user_index)
91         if user_index != -1:
92             print("Stored Password:", user_data.loc[user_index, 'Password'])
93             print("Entered Password:", password)
94             if user_index != -1 and user_data.loc[user_index, 'Password'] == password:
95                 self.app.open_welcome_page(self, email)
96             elif user_data.loc[user_index, 'Password'] != password and user_index != -1:
97                 messagebox.showerror("Login Failed", "Incorrect password.")
98             elif user_data.loc[user_index, 'Password'] == password and user_index == -1:
99                 messagebox.showerror("Login Failed", "Incorrect email.")
100         else:
101             messagebox.showerror("Invalid Email", "Please enter a valid email address.")
102
103
104
105
106
107
108
109
110
111
112
113
114
115
116
117
118
119
120
121
122
123
124
125
126
127
128
129
130
131
132
133
134
135
136
137
138
139
140
141
142
143
144
145
146
147
148
149
150
151
152
153
154
155
156
157
158
159
160
161
162
163
164
165
166
167
168
169
170
171
172
173
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999
1000
```

This is an updated version of my code to help me debug my code with extra print statements and more precise error boxes. This shows that even though the passwords match the error box occurs and says passwords are incorrect, so the email is obviously matching, but passwords don't match. To fix this I should try either to cast the password as a string and compare or try the .strip() method.

I decided to update my code by making a variable stored password, so it is reusable and makes my code easier to debug in the future. I also added the casting method str() to convert the password stored with email to a string as the user enters the password as a string so this is where my code has been going wrong.

And after these changes my code works using BOMP@NEA.co.uk and 1234 and the .strip() method was not needed.

GUI Testing iteration 4

I will use white box testing again to test my updated log in class with a similar format to how I tested my sign-up class.

Evidence is screen recordings in folders.

Debugging GUI 4

Test #	Test name/purpose	Test data Email	Test data Password	Expected result	Actual result
1(3)	Test correct email and password	Test7@nea.co.uk	1234	Welcome page	Welcome page
2(4)	Test correct email and incorrect password	test3@nea.co.uk	12345	Error message, retry, password 1234, welcome page.	Error message, retry, password 1234, welcome page.
3(5)	Test email that isn't in database	DOMP@NEA.co.uk	1234	Error message, retry, COMP@NEA.co.uk , welcome page	No error message, retry, COMP@NEA.co.uk , welcome page

```

if user_index != -1 and stored_password == password:
    self.app.open_welcome_page(self, email)
elif stored_password != password and user_index != -1:
    messagebox.showerror("Login Failed", "Incorrect password.")
elif stored_password == password and user_index == -1:
    messagebox.showerror("Login Failed", "Incorrect email.")

```

My code is going wrong somewhere in this section, looking at it I can see the problem is if there is no email in database there is no stored password so that part of the if statement will never be valid.

After making these changes it is still not working so I follow the trace in terminal which says:

```

Traceback (most recent call last):
  File "C:\Program Files\WindowsApps\PythonSoftwareFoundation.Python.3.11.3.11.2544.0_x64__qbz5n2kfra8p0\Lib\tkinter\__init__.py", line 1967, in __call__
    return self.func(*args)
  File "C:\Users\wilfw\OneDrive\Documents\Computing A level\A GUI iteration 4 development.py", line 91, in check_login_details
    stored_password = str(user_data.loc[user_index, 'Password'])
  File "C:\Users\wilfw\AppData\Local\Packages\PythonSoftwareFoundation.Python.3.11_qbz5n2kfra8p0\LocalCache\local-packages\Python311\site-packages\pandas\core\indexing.py", line 1183, in _getitem_
    return self.obj._get_value(*key, takeable=self._takeable)
  File "C:\Users\wilfw\AppData\Local\Packages\PythonSoftwareFoundation.Python.3.11_qbz5n2kfra8p0\LocalCache\local-packages\Python311\site-packages\pandas\core\frame.py", line 4221, in _get_value
    row = self.index.get_loc(index)
  File "C:\Users\wilfw\AppData\Local\Packages\PythonSoftwareFoundation.Python.3.11_qbz5n2kfra8p0\LocalCache\local-packages\Python311\site-packages\pandas\core\indexes\range.py", line 415, in get_loc
    raise KeyError(key) from err
KeyError: -1

```

The root of these problems occurs from line 91 in my code which is.

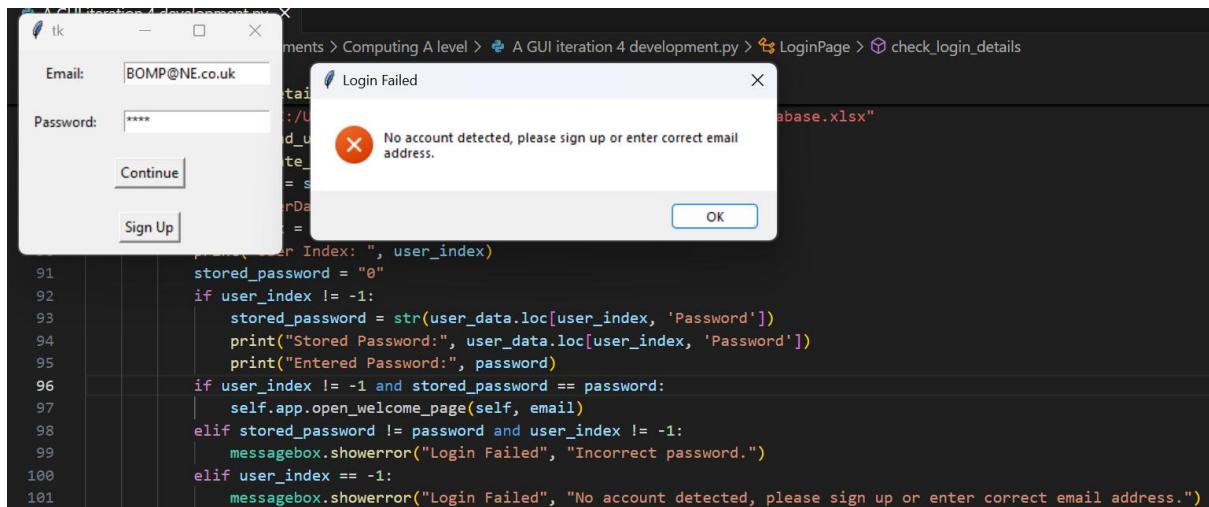
```

91 stored_password = str(user_data.loc[user_index, 'Password'])

```

Again, looking at this there is a clear error of when user_index is equal to minus one there is no data stored there and therefore has no password to find, the most obvious fix is to add an if statement of If user_index != -1: to code and have the

variable originally stored as 0. Here is a screen shot of the changes made and an image of my code producing the correct error message.



Evaluation 4

My log_in class and Sign_up class now both work well alongside my database but they are both very basic and boring, a more interesting background should be added to both of them, the labels and entrys should stay in the centre of the frame/window when window size changed and maybe a back button added to the sign up page in case user accidently clicks it and wants to go back to log in page.

GUI iteration 5

GUI design 5



In this iteration I need to create a more attractive tkinter frame and add a back button to the sign_up page class. I have decided to use this image for my background as I feel it is the perfect brightness and expresses what my website/program is about (saving money and energy using solar panels).

GUI development 5

```

7 class LoginPage:
8     def __init__(self, root, app):
9         self.root = root
10        self.app = app
11        self.frame = tk.Frame(self.root)
12        self.frame.grid(row=0, column=0, sticky="nsew")
13        self.create_widgets()
14
15        self.root.grid_rowconfigure(0, weight=1)
16        self.root.grid_columnconfigure(0, weight=1)
17
18    def create_widgets(self):
19        self.frame.grid_rowconfigure(0, weight=1)
20        self.frame.grid_rowconfigure(2, weight=1)
21        self.frame.grid_columnconfigure(0, weight=1)
22        self.frame.grid_columnconfigure(2, weight=1)
23
24class SignupPage:
25    def __init__(self, root, app):
26        self.root = root
27        self.app = app
28        self.frame = tk.Frame(self.root)
29        self.frame.grid(row=0, column=0, sticky="nsew")
30        self.create_widgets()
31
32        self.root.grid_rowconfigure(0, weight=1)
33        self.root.grid_columnconfigure(0, weight=1)
34
35    def create_widgets(self):
36        self.frame.grid_rowconfigure(0, weight=1)
37        self.frame.grid_rowconfigure(7, weight=1)
38        self.frame.grid_columnconfigure(0, weight=1)
39        self.frame.grid_columnconfigure(2, weight=1)
40
41        self.email_label = tk.Label(self.frame, text="Email:")
42        self.email_label.grid(row=1, column=0, padx=10, pady=10, sticky="e")
43        self.email_entry = tk.Entry(self.frame)
44        self.email_entry.grid(row=1, column=1, padx=10, pady=10, sticky="w")
45
46        self.password_label = tk.Label(self.frame, text="Password:")
47        self.password_label.grid(row=2, column=0, padx=10, pady=10, sticky="e")
48        self.password_entry = tk.Entry(self.frame, show="*")
49        self.password_entry.grid(row=2, column=1, padx=10, pady=10, sticky="w")
50
51class BackgroundHelper:
52    def __init__(self, image_path):
53        self.image_path = image_path
54        self.bg_image = Image.open(image_path)
55        self.bg_photo = ImageTk.PhotoImage(self.bg_image)
56
57    def apply_background(self, root):
58        canvas = tk.Canvas(root, width=self.bg_photo.width(), height=self.bg_photo.height())
59        canvas.create_image(0, 0, anchor='nw', image=self.bg_photo)
60        canvas.place(relwidth=1, relheight=1)
61
62        canvas.bind('<Configure>', self.resize_background)
63
64        root.grid_rowconfigure(0, weight=1)
65        root.grid_columnconfigure(0, weight=1)
66        return canvas
67
68    def resize_background(self, event):
69        self.bg_image = self.bg_image.resize((event.width, event.height), Image.LANCZOS)
70        self.bg_photo = ImageTk.PhotoImage(self.bg_image)
71
72        event.widget.create_image(0, 0, anchor='nw', image=self.bg_photo)
73        event.widget.image = self.bg_photo

```

The changes made to the special initialization method ensure the frame expands to fill up the space available. The changes made to the method create_widgets ensures the widgets move to occupy the centre of the frame when window is expanded. The same changes are made to the sign-up page class and welcome page class. My sign up page proved to be more hassle, my confirm password label would often stay off to the left, this was fixed by adding sticky="e" to it which sticks it to the left of the centre of the column whilst sticky="w" does the same on the right side. I had to switch from using the method pack() in my code to use the method grid() because the method pack() does not allow me to add row or column span or weight to my widgets which means my frame and widgets could not automatically adjust with the size of the window.

I then created a class that applies my background image through the use of various methods. Self.image_path= image_path means the image path can be used anywhere in the class without having to be passed in. Same goes for the other lines of initialization method. My

apply_background method creates the canvas for image to be stored on and adds the image to it using the filepath. My method resize_background is then called called inside canvas method bind alongside <Configure> which makes sure whenever canvas is resized background is also resized. Configures grid layout to make the canvas resizable. Returns canvas with resized background. Method resize_background resizes background image with new height and width of canvas. Image.LANCZOS ensures high quality resizing. Updates variable

self.bg_image with the resized image. Re-creates image on canvas to reflect new dimensions. And the last line updates the canvas widgets image to prevent it being garbage collected.

```

self.back_button= tk.Button(self.frame, text="back", command=app.open_login_page)
self.back_button.grid(row=0, column=0)

```

Creating my back button was very simple as my code is very re-usable and only took two lines of code.

GUI Testing 5





GUI Evaluation 5

My background image functions well, and my back button is unproblematic. The quality of image often decreases if change size of window, frame and canvas too much but is mostly unnoticeable. My next stage in my programs development is to actually show data from users givenenergy using API.

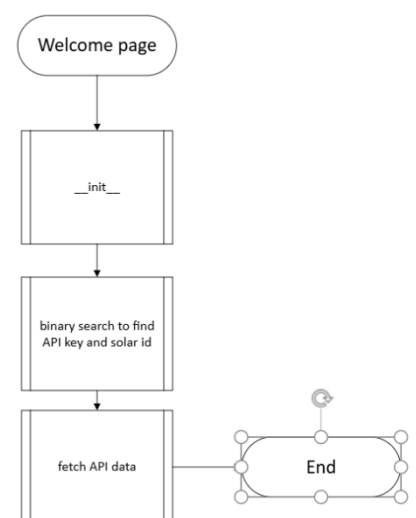
API code iteration 1

I have decided that I should focus on getting a finished prototype done before further improving GUI so that I can ask for end users' opinions on the whole program and then make changes.

API code design 1

I have already experimented with the API code previously and will now need to implement it in methods of welcomepage to extract, sort through and display in a more attractive form, calculate the optimum and then allow user to make changes to plan and input plan into givenenergy. This is obviously a lot to do so for the first iteration I will aim to get API data displayed on welcome page no matter how ugly the format.

To accomplish this, I think I will need to add methods to my welcomepage class, one method will be for fetching the API data and the other will be for finding the corresponding API key and serial number from users email in the database. This flow chart displays the new methods/sub routines that are needed. Thanks to my modular



approach, the binary search used in login class page can be used in this class to save me time. I will create widgets in ininit method currently as there are only two widgets needed. My fetch API data method will look very similar to the API code I was experimenting on in the design main stage so can be practically re-used here.

API code development 1

```
292     def open_welcome_page(self, app, email, user_data):
293         if self.current_frame:
294             self.current_frame.frame.destroy()
295             self.current_frame = WelcomePage(self.root, app, email, user_data)
```

My open_welcome_page class needed to be edited to pass in the user_data (database) into the initialisation method.

```
228 class WelcomePage:
229 >     def __init__(self, root, app, email, user_data): ...
248
249 >     def fetch_API_data(self, email, user_data):|...
263
264 >     def binary_search(self, email, df, searchfor): ...
275
```

My two new methods outlined in my welcome page class, I edited the initialisation parameters so that user_data (database) could be used throughout this class to find API key and serial number/solar ID of user, I edited my initialisation method to display a new label on screen including the data fetched from API, and edited it to call my new methods.

```
246     data_printing = tk.Label(self.frame, text=f"{self.fetch_API_data(email, user_data)}")
247     data_printing.grid(row=1, column=0, padx=40, pady=40)
248
```

Adding the new label proved to be extremely easy and required just two lines of code.

```
224 class WelcomePage:
246
247     def fetch_API_data(self, email, user_data):
248         givenenergy_key = self.binary_search(email, user_data, 'API Key')
249         serialnum = self.binary_search(email, user_data, 'Solar ID')
250         givenenergy_headers = {
251             'Authorization': f'Bearer {givenenergy_key}',
252             'Content-Type': 'application/json',
253             'Accept': 'application/json'
254         }
255         givenenergy_url = f"https://api.givenenergy.cloud/v1/inverter/{str(serialnum)}/meter-data/latest"
256         response = requests.get(givenenergy_url, headers=givenenergy_headers)
257         data = response.json()
258         return data
259
260     def binary_search(self, email, df, searchfor):
261         low = 0
262         high = len(df) - 1
263         while low <= high:
264             mid = (low + high) // 2
265             if df.loc[mid, 'Email'] == email:
266                 return df.loc[mid, searchfor]
267             elif df.loc[mid, 'Email'] < email:
268                 low = mid + 1
269             else: high = mid - 1
270         return - 1
```

My fetch_API_data method reuses lots of code used in experimenting with the APIs previously and my binary_search method is practically identical to the one used previously besides the searched for item.

API code testing 1



My code works well despite the extremely stretched frame it creates; the next step is to convert data into a nicer format. When trying to concatenate using panda, the code couldn't concatenate the data as it is in a different format as seen by these error messages.

```
self.current_frame = WelcomePage(self.root, app, email, user_data)
File "c:\Users\wilfw\OneDrive\Documents\Computing A level\A GUI iteration 4 development.py", line 242, in __init__
    text=self.fetch_API_data(email, user_data)
File "c:\Users\wilfw\OneDrive\Documents\Computing A level\A GUI iteration 4 development.py", line 258, in fetch_API_data
    return pd.concat(self.data)
File "C:\Users\wilfw\AppData\Local\Packages\PythonSoftwareFoundation.Python.3.11_qbz5n2kfra8p0\LocalCache\local-packages\Python311\site-packages\pandas\core\reshape\concat.py", line 382, in concat
    op = _Concatenator(
File "C:\Users\wilfw\AppData\Local\Packages\PythonSoftwareFoundation.Python.3.11_qbz5n2kfra8p0\LocalCache\local-packages\Python311\site-packages\pandas\core\reshape\concat.py", line 448, in __init__
    ndims = self._get_ndims(objs)
File "C:\Users\wilfw\AppData\Local\Packages\PythonSoftwareFoundation.Python.3.11_qbz5n2kfra8p0\LocalCache\local-packages\Python311\site-packages\pandas\core\reshape\concat.py", line 448, in _get_ndims
    ndims = self._get_ndims(objs)
File "C:\Users\wilfw\AppData\Local\Packages\PythonSoftwareFoundation.Python.3.11_qbz5n2kfra8p0\LocalCache\local-packages\Python311\site-packages\pandas\core\reshape\concat.py", line 489, in _get_ndims
    raise TypeError(msg)
TypeError: cannot concatenate object of type '<class 'dict'>'; only Series and DataFrame objs are valid
PS C:\Users\wilfw> & C:\Users\wilfw\AppData\Local\Microsoft\WindowsApps\python3.11.exe "c:\Users\wilfw\OneDrive\Documents\Computing A level\A GUI iteration 4 development.py"
```

API code testing evaluation 1

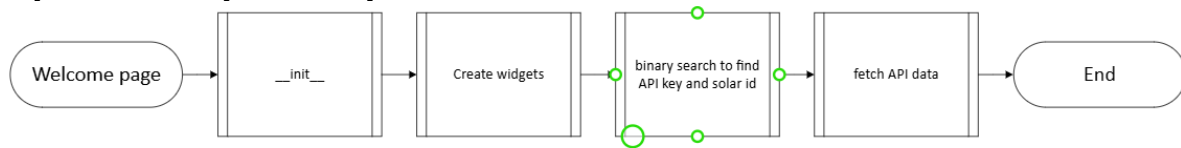
My next step is to create a series of search algorithms that search for individual pieces of data so they can be displayed in a nicer format.

API code iteration 2

API code design 2

I need to separate the long data up into easier to work with sections of data, to do this I need to learn how to use JSON data and formatting which was quite easy to learn and add a create widgets method to the class for modularity and ensuring

my code is easy on the eye.



This is the updated version of my welcome page class, and the methods included.

User interview to find out the data needed to display

Italic= Me

Bold = Dave

Which pieces of data would you like to be displayed to you when you log in to your account?

I would like to know the percentage of the battery that I started the day with and the current battery charge as this is a simple indicator as to how much energy has been used/generated, could also indicate to me if have imported energy from the grid yet (battery would be at 4%) or energy has been exported to the grid yet (battery at 100%). I think the only other variables I will need are energy generated by solar panels today, the energy imported from the grid and energy exported to the grid today for obvious reasons.

API code development 2

I initially came across error messages that told me that I couldn't extract data from my JSON format because it was in dictionary form and needed to be in string form, this was a quick fix as I needed to concatenate data into string form using `json.dumps()` concatenator. As you can see in the piece of code in the image this requires little code and is simple. My next step is to extract more data and display on GUI in labels. To make my code shorter I should try and do some of the json code inside label definitions.

```
240 welcome_label = tk.Label(self.frame, text=f"Welcome, {email}!")
241 welcome_label.grid(row=0, column=0, padx=20, pady=20)
242 text=self.fetch_API_data(email, user_data)
243 print(text)
244 data_printing = tk.Label(self.frame, text=f"{self.fetch_API_data(email, user_data)}")
245 data_printing.grid(row=1, column=0, padx=40, pady=40)
246 JSON_str = json.dumps(text)
247 data = json.loads(JSON_str)
248 time = data['data']['time']
249 print(time)
250
```

The pieces of data I included to be displayed have all been given by the user.

```
def create_widgets(self, email, user_data):
    self.content_frame = tk.Frame(self.canvas, bg='white', bd=1)
    self.content_frame.grid(row=0, column=0, sticky='nsew')

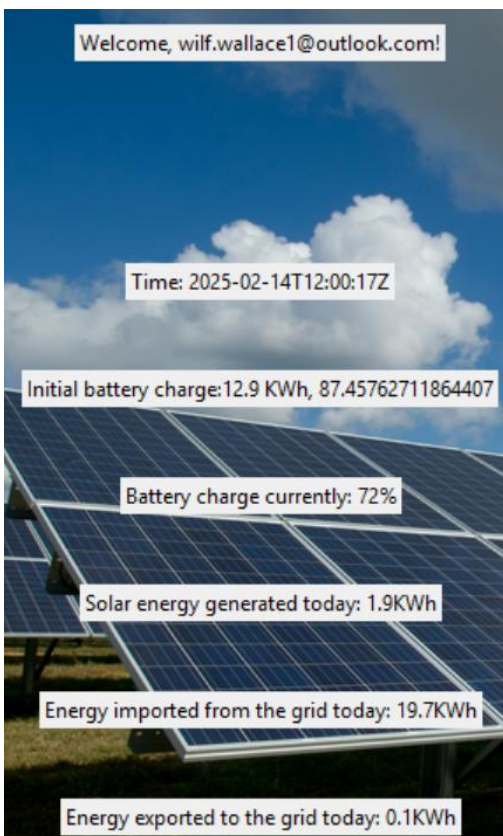
    self.frame.grid_rowconfigure(0, weight=1)
    self.frame.grid_rowconfigure(8, weight=1)
    self.frame.grid_columnconfigure(0, weight=1)
    self.frame.grid_columnconfigure(1, weight=1)
    welcome_label = tk.Label(self.frame, text=f"Welcome, {email}!")
    welcome_label.grid(row=0, column=0, padx=20, pady=20)
    serialnum = self.binary_search(email, user_data, 'Solar ID')
    meterurl = f"https://api.givenergy.cloud/v1/inverter/{str(serialnum)}/meter-data/latest"
    systemurl = f"https://api.givenergy.cloud/v1/inverter/{str(serialnum)}/system-data/latest"
    metertext=self.fetch_API_data(email, user_data, meterurl)
    print(metertext)
    meter = json.loads(json.dumps(metertext))
    systemtext = self.fetch_API_data(email, user_data, systemurl)
    print(systemtext)
    system = json.loads(json.dumps(systemtext))
    time_label = tk.Label(self.frame, text=f"Time: {meter['data']['time']}")
    time_label.grid(row=2, column=0, padx=20, pady=20)
    battery_charge_label_initial = tk.Label(self.frame, text=f"Initial battery charge: {meter['data']['today']['ac_charge']} KWh, {meter['data']['today']['battery_charge_label_initial']}")
    battery_charge_label_initial.grid(row=3, column=0, padx=20, pady=20)
    battery_charge_label_rn = tk.Label(self.frame, text=f"Battery charge currently: {system['data']['battery']['percent']}%")
    battery_charge_label_rn.grid(row=4, column=0, padx=20, pady=20)
    solar_gen_label = tk.Label(self.frame, text=f"Solar energy generated today: {meter['data']['today']['solar']} KWh")
    solar_gen_label.grid(row=5, column=0, padx=20, pady=20)
    grid_imp_label = tk.Label(self.frame, text=f"Energy imported from the grid today: {meter['data']['today']['grid']['import']} KWh")
    grid_imp_label.grid(row=6, column=0, padx=20, pady=20)
    grid_exp_label = tk.Label(self.frame, text=f"Energy exported to the grid today: {meter['data']['today']['grid']['export']} KWh")
    grid_exp_label.grid(row=7, column=0, padx=20, pady=20)

25     def __init__(self, root, app, email, user_data):
26         self.root = root
27         self.app = app
28         self.frame = tk.Frame(self.root, bg='white', bd=1)
29         self.frame.grid(row=0, column=0, sticky="nsew")
30         self.bg_helper = BackgroundHelper(r"C:/Users/wilfw/OneDrive/Documents/Computing A level/solarpanelpic(PNG).png")
31         self.canvas = self.bg_helper.apply_background(self.frame)
32         self.create_widgets(email, user_data)
```

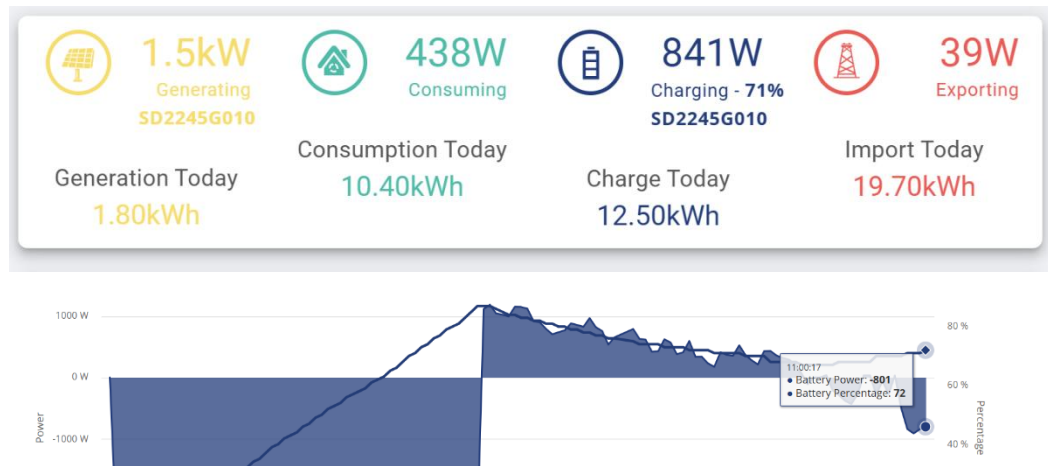
As well as splitting up initialisation method into init method and createwidgets method, the code I produced extracts all data from json format and displays it as a label on grid. I realised whilst coding that there is a different piece of data that I can get from the givenergy API called system data that gives me the current battery percentage meaning I don't have to calculate it using the battery discharge and other variables, but all other pieces of data were able to be fetched using meter-data. Once I fully understood how to concatenate json data and extract it, this was easy as it was quite repetitive of code used lots in program already. I also managed to use the same method for different urls by passing in the url as a parameter instead of defining in the url.

```
def fetch_API_data(self, email, user_data, url):
    givenergy_key = self.binary_search(email, user_data, 'API Key')
    givenergy_headers = {
        'Authorization': f'Bearer {givenergy_key}',
        'Content-Type': 'application/json',
        'Accept': 'application/json'
    }
    response = requests.get(url, headers=givenergy_headers)
    data = response.json()
    return data
```

API code testing 2



My code works very well and obviously only when user is connected to WIFI. When checking against givenrgy to ensure that correct data is presented, there are positive results with the initial charge value not being entirely accurate all the time(often one percent off).



Api code evaluation 2

I am happy with how my code works, a cool but not immediately necessary addition could be to calculate money saved by one having solar panels and two by charging battery at cheaper hours and total energy saved. Another addition that could be implemented is a refresh button to re-fetch API data so the user can see the most recent data say if they keep page open for a while and data becomes “outdated” and old. However, I think the next addition is retrieving data from the solar cast API now as this is a necessary part of the solution.

API code iteration 3

In this iteration I plan to extract the data from solar cast that will result in me having predictions for the next day’s energy predictions.

User interview

Italic: Me

Bold: Dave

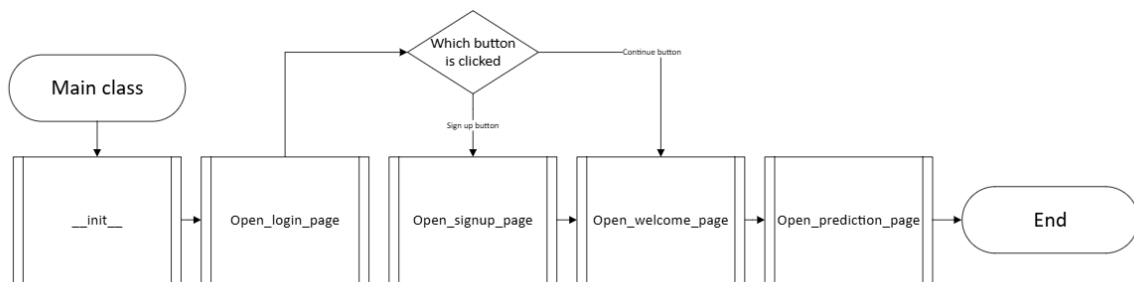
Do you think you need to be informed of the changed the algorithm is making to your energy system and if so, how much data would you like to be displayed?

I think I do need to know in case I disagree due to circumstances the algorithm can’t predict such as no-one being home and I decide to change

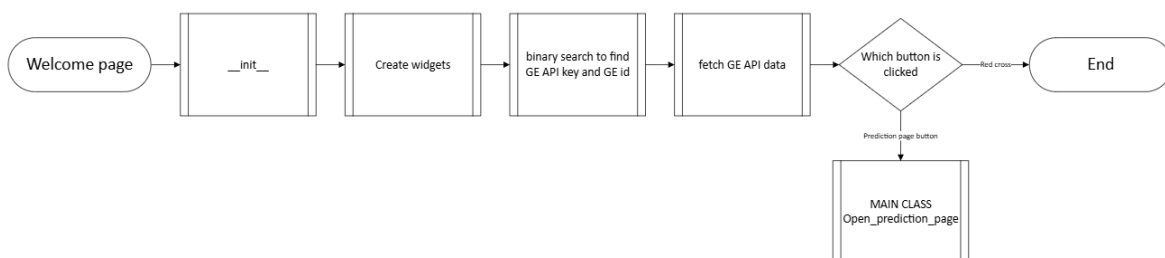
what algorithm decides. To change what the algorithm decides I think it would be helpful for me to know the algorithms “reasoning” so I think the predicted energy generated by solar panels for tomorrow should be displayed as well as battery % to be charged to during off-peak hours.

API code design 3

I think a new class should be created that displays a screen for plans with tomorrow’s energy, for example after a few software iterations it should display the battery to charge the battery to and the energy predictions for tomorrow including usage and solar generation. For this iteration it will just be solar generation being displayed in the new class. This new class will require a new method in class main, a new initialization method in new class alongside a new method create widgets, fetch API data method and somewhere in the midst convert the data into a readable form using my newfound skill with handling data in json format and an oiterative piece of code adding up each time interval. This will also require a new field to be stored in database as solarcast API key and solarcast id so data specific to users’ location and solar panels can be fetched.



This updated version of my main class displays the additional method and how it fits in with the flow of the class.



This updated version of welcome page class shows the flow of algorithm with an additional button



My new class structure will look like this, and it is likely I can re-use modules from the welcome page class because of similar inputs and outputs demanded and modular approach used. Fetch SC API data method will be very similar if not identical to code used in research section when experimenting with API code.

API code development 3

Whilst developing new methods and editing old methods I discovered lots of redundant code and mistakes left in my code from earlier iteration, for example, having open_signup_page method in login class was a waste of space as same affect could be achieved in a single line of code. Also, I found I had repeated this section of code a lot throughout my algorithm in various __init__ methods. I removed the redundant pieces of code and found my algorithm still worked.

```
self.frame = tk.Frame(self.root, bg='white', bd=1)
self.frame.grid(row=0, column=0, sticky='nsew')
```

I also found the initialization of welcome page was incorrect as variables were passed in wrong order meaning I couldn't correctly use my main class from welcome page which was discovered after trying to call open_predictions_page in main class.

```
PP_button = tk.Button(self.frame, text="Go to prediction page", command=lambda: app.open_prediction_page(email, user_data))
PP_button.grid(row=8, column=0, padx=20, pady=20)
```

The first step was calling a method in class main by using "app" to access main class. Creating buttons and/or labels have become easy but I had some issues this time because the button kept on getting "clicked" straight away without user input, this was fixed through the use of lambda which ensures command is only initiated when button is clicked by user.

```
class Main:
>     def __init__(self, root): ...
>
>     def open_login_page(self): ...
>
>     def open_signup_page(self): ...
>
>     def open_welcome_page(self, email, user_data): ...
>
>     def open_prediction_page(self, email, user_data):
>         if self.current_frame:
>             self.current_frame.frame.destroy()
>             self.current_frame = PredictionPage(self.root, self, email, user_data)
```

Then I created the method in class main and called new class, passing in all needed data for finding solarcast API details. The format of method is very similar to other methods in this class.

```
274 class PredictionPage:
275     def __init__(self, root, app, email, user_data):
276         self.root = root
277         self.app = app
278         self.frame = tk.Frame(self.root, bg='white', bd=1)
279         self.frame.grid(row=0, column=0, sticky='nsew')
280         self.bg_helper = BackgroundHelper(r"C:/Users/wilfw/OneDrive/Documents/Computing A level/solarpanelpic(PNG).png")
281         self.canvas = self.bg_helper.apply_background(self.frame)
282         self.create_widgets(email, user_data)
283
```

I then coded the initialisation method of predictions page using similar format to other class initialisations bar main, using my class background helper as usual to set the background of frame.

```
284     def create_widgets(self, email, user_data):
285         self.frame.grid_rowconfigure(0, weight=1)
286         self.frame.grid_rowconfigure(2, weight=1)
287         self.frame.grid_columnconfigure(0, weight=1)
288         self.frame.grid_columnconfigure(1, weight=1)
289         title_label = tk.Label(self.frame, text=f"Predictions for, {email}!")
290         title_label.grid(row=0, column=0, padx=20, pady=20)
291         SCid = self.binary_search(email, user_data, 'SC ID')
292         SCurl = f'https://api.solcast.com.au/rooftop_sites/{SCid}/forecasts?format=json'
293         SCdata = self.fetch_SC_data(email, user_data, SCurl)
294         total_pv_estimate = self.find_pv_total_estimate(SCdata.json())
295         #print(SCdata)
296         test_label = tk.Label(self.frame, text=str(total_pv_estimate))
297         test_label.grid(row = 1, column = 0, padx = 20, pady = 20)
```

My create widgets method looks rather messy but follows the same layout as other create widgets methods in other classes and uses other methods in class to display second label test_label that should display total_pv_estimate. The method binary search is a duplicate of binary search method in the welcome page with the exception of what is passed into the method as it is searching for solar id.

```
299     def fetch_SC_data(self, email, user_data, SCurl):
300         SCkey = self.binary_search(email, user_data, 'SC key')
301         SCheaders = {
302             'Authorization': f'Bearer {SCkey}',
303             'Content-Type': 'application/json',
304         }
305
306         response = requests.request("GET", SCurl, headers=SCheaders)
307         print(str(response))
308         return response.json()
```

This method is responsible for getting data from solarcast API using format experimented with in design main section.

```

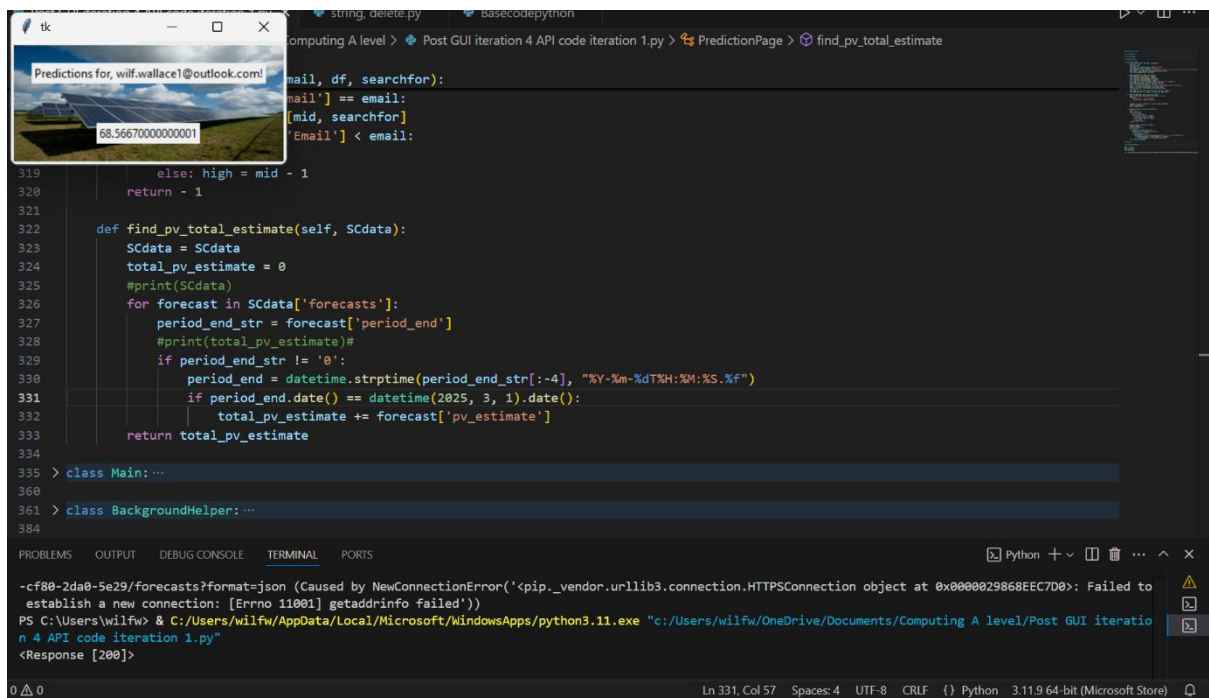
322     def find_pv_total_estimate(self, SCdata):
323         SCdata = SCdata
324         total_pv_estimate = 0
325         #print(SCdata)
326         for forecast in SCdata['forecasts']:
327             period_end_str = forecast['period_end']
328             #print(total_pv_estimate)#
329             if period_end_str != '0':
330                 period_end = datetime.strptime(period_end_str[:4], "%Y-%m-%dT%H:%M:%S.%f")
331                 if period_end.date() == datetime(2025, 3, 1).date():
332                     total_pv_estimate += forecast['pv_estimate']
333         return total_pv_estimate

```

This method uses data fetched from API in json form to create a nicer format of total pv estimated to generate the next day.

	A	B	C	D	E	F
1	Email	Password	API Key	Solar ID	SC ID	SC key
2	COMP@NEA.co.uk	1234	1234	1234		1234
3	test3@nea.co.uk	1234	1234	1234		1234
4	test4@nea.co.uk	1234	1234	1234		1234
5	Test5@nea.co.uk	1234	1234	1234		1234
6	Test6@nea.co.uk	1234	1234	1234		1234
7	Test7@nea.co.uk	1234	1234	1234		1234
8	BOMP@NEA.co.uk	1234	4444	4444		1234
9	wilf.wallace1@outlook.com		yJ0eXAI0iJKV1QilCJhbG	SD2245G010	67fb-cf80-2da0-5e29	pdfNTYcXeRO8_WI-WCxn1iqw8SCQfZO

I had to add in columns and data to database for my code to work, I will add in a place for user to input these values in the next iteration.



As you can see my code works perfectly, however the total pv estimate is completely unrealistic and is not a mistake in the code so can only be due to API not being at all accurate.

API code iteration 4

Therefore, I need to use a different API. The API I have decided to switch to is “weather API” which is an API that will give me cloud percentage so will need to do some calculations. This API is nice because it can change the location of data wanted every time it is called so I can use one key for every user which couldn’t be done with the solar cast API. This allows me to delete the additional columns added into database and saves me work of having to add input labels to enter users API details later.

Your annual average:

2.94 peak sun hours per day

Weather API research

The weather API contains lots of variables that can be used but the only needed ones are cloud coverage, for calculating predicted energy using the equation and table found in research section:

(In the first 9 days of the month should calculate hours of peak sunlight by doing 3 times that month’s peak sunlight hours for that month plus the peak sunlight hours of last month times 2 all over 5 to create a more realistic value, do a similar calculation for last 9 days of the month. Multiply hours of peak sun hours (depends on the months) by 1-cloud coverage times by solar panel wattage by number of solar panels to calculate how many KWhs will be generated the next day).

Your monthly averages:

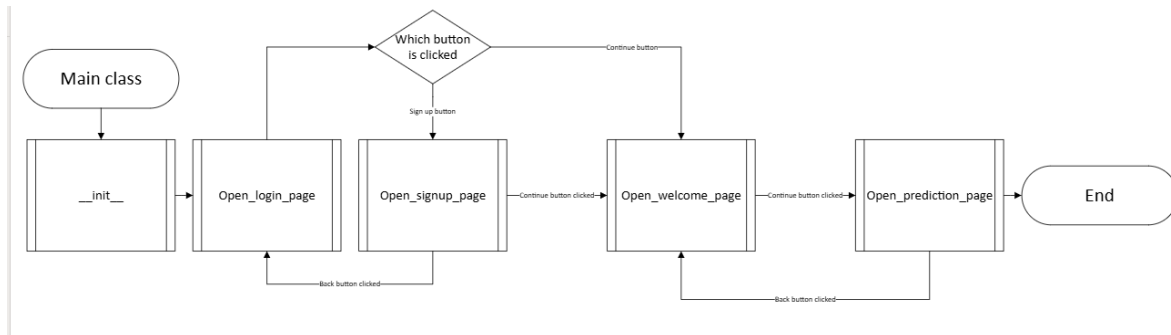
- January: 0.73
- February: 1.64
- March: 2.87
- April: 4.14
- May: 4.56
- June: 5.08
- July: 4.82
- August: 3.89
- September: 3.67
- October: 2
- November: 1.15
- December: 0.72

This is some code that I made whilst experimenting with API.

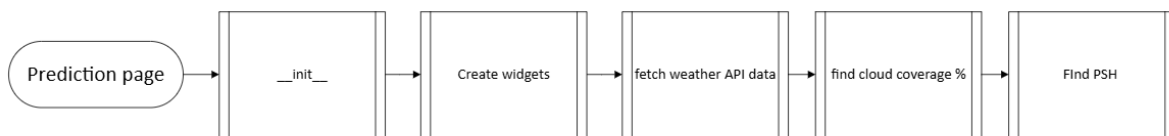
As you can see my code generates a more realistic value for predicted energy generation than when I was using solarcast API. I used printing and screen shots as methods for testing accuracy of formula. Thanks to libraries calendar and datetime, I was able to implement a decision-making process to get a more realistic value for peak sun hours (PSH). It was also easy to find `total_cloud_coverage` as I am experienced with java now.

API code design 4

I decided I'm going to add a back button into predictions page.



I also had to do a large redesign of my predictions page class.



API code development 4

```
def create_widgets(self, email, user_data):
    self.frame.grid_rowconfigure(0, weight=1)
    self.frame.grid_rowconfigure(3, weight=1)
    self.frame.grid_columnconfigure(0, weight=1)
    self.frame.grid_columnconfigure(2, weight=1)

    back_button = tk.Button(self.frame, text="Back", command=lambda: app.open_welcome_page(email, user_data))
    back_button.grid(row=0, column=0, padx=20, pady=20)

    title_label = tk.Label(self.frame, text=f"Predictions for, {email}!")
    title_label.grid(row=1, column=1, padx=20, pady=20)

    WA_data = self.fetch_WA_data()
    forecast_hourly = WA_data['forecast']['forecastday'][0]['hour']
    cloud_coverage_percent = self.cloud_coverage_percent(forecast_hourly)
    print(cloud_coverage_percent)
    PSHs = self.find_PSHs()
    print(PSHs)
    P_KWhs = ((1-cloud_coverage_percent)*PSHs*12*295)/1000

    test_label = tk.Label(self.frame, text=str(P_KWhs))
    test_label.grid(row = 2, column = 1, padx = 20, pady = 20)
```

This screen shot shows my updated create widgets method, includes a back button similar to back button used in sign up class. It also calls fetch weather API data method then extracts necessary data

into forecast_hourly and passes this into cloud_coverage_percent to find average cloud coverage for that day. It also finds Peak Solar Hours using method find_PSHs then calculates power in kilowatts and displays on screen.

```
def fetch_WA_data(self):
    WA_key = "96792c1aa5f743a88be93336252802"
    WA_url = "https://api.weatherapi.com/v1/forecast.json"
    params = {
        'key': WA_key,
        'q': "GL10 3UH",
        'days': 2, # Number of days to forecast
        'aqi': 'no', # Air Quality Index
        'alerts': 'no' # Weather Alerts
    }
    response = requests.get(WA_url, params=params)
    return response.json()
```

My method fetch_WA_data uses code I experimented with previously to extract data. It is also fairly similar to the Givenenergy API syntax so was a quick code with the help of weather API documentation.

```
def cloud_coverage_percent(self, forecast_hourly):
    total_cloud_coverage = 0
    num_entries_is_day = 0
    for entry in forecast_hourly:
        if entry['is_day'] == 1:
            total_cloud_coverage += entry['cloud']
            num_entries_is_day += 1
    cloud_coverage_percent = (total_cloud_coverage/num_entries_is_day)/100
    return cloud_coverage_percent
```

This method makes use of a simple for loop to add up all the cloud coverage for hours that are true for is_day and finds average percent at the end of the looping using the

total_cloud_coverage divided by the amount of num_entries_is_day variable that counts amount of entries that is_day is true for. Looking at this code now, however, this will not work, num_entries_is_day is always going to be 24, needs to be indented. Fixed code:

```
if entry['is_day'] == 1:
    total_cloud_coverage += entry['cloud']
    num_entries_is_day += 1
```

```
def find_PSHs(self):
    now = datetime.now()
    day = now.day
    month = now.month
    year = now.year
    if month == 12:
        next_month = 1
        previous_month = 11
    elif month == 1:
        next_month = 2
        previous_month = 12
    else:
        next_month = month + 1
        previous_month = month - 1
    table = [
        [1,2,3,4,5,6,7,8,9,10,11,12],
        [0.73,1.64,2.87,4.14,4.56,5.08,4.82,3.89,3.67,2,1.15,0.72]
    ]
    _, last_day = calendar.monthrange(year, month)
    if day <= 8:
        PSHs = (table[1][month - 1]*5 + table[1][previous_month - 1]*3)/8
    elif day >= last_day - 8:
        PSHs = (table[1][month - 1]*5 + table[1][next_month - 1]*3)/8
    else:
        PSHs = table[1][month - 1]
    return PSHs
```

This method uses library datetime to find current day, month and year to find peak solar hours, also calculates next/previous month when the current is on the end or start of an iteration(1 and 12). Uses calendar library to find the last day of that month so the PSHs can be calculated to be slightly more variable throughout the month instead of drastic changes when months change, the PSH doesn't actually go from 0.73 in January to 1.63 in February overnight for example.

API code testing 4

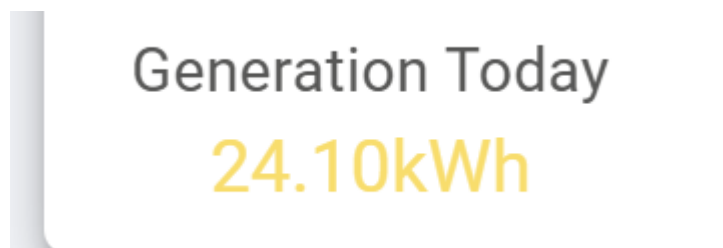
I added in a new model to test multiple at once and decide on which is the most accurate to save time. I also used a separate piece of code to do this for simplicity.

```
87 print("Predicted energy for tomorrow using peak sunlight hours",P_KWhs)
88
89 energy_output = 25 * (1 - average_percent_cloud_coverage)
90 print("Predicted energy for tomorrow using max energy output", energy_output)
91
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
date today 2 - 3
date tomorrow 3 - 3
Predicted energy for tomorrow using peak sunlight hours 8.333180113636365
Predicted energy for tomorrow using max energy output 24.431818181818183
PS C:\Users\wilfw>
```

```
date today 2 - 3
date tomorrow 3 - 3
Predicted energy for tomorrow using peak sunlight hours 8.333180113636365
Predicted energy for tomorrow using max energy output 24.431818181818183
PS C:\Users\wilfw>
```

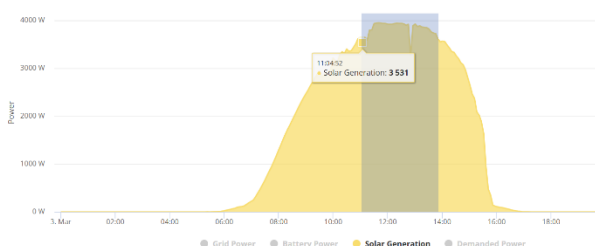


```
date today 3 - 3
date tomorrow 4 - 3
Predicted energy for tomorrow using peak sunlight hours 10.839375000000002
Predicted energy for tomorrow using max energy output 25.0
PS C:\Users\wilfw>
```

API code testing evaluation 4

Thankfully, the new API I am using works well at getting the cloud coverage for the next day and if I feel the need can also get many other variables for tomorrow. For me neither of these calculation models were satisfying me so I decided to do some experimenting. The max energy model does not change

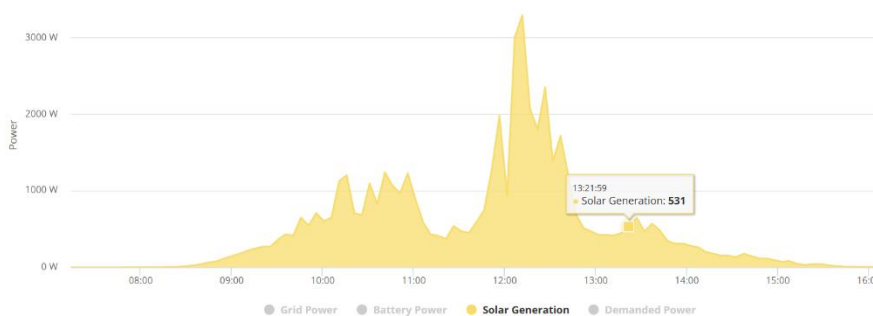
based on months/seasons when the number of daylight hours changes so would not work yearly and the peak sunlight hours only calculated the amount generated during absolute maximum generation hours as displayed by this graph that shows the 2.5 hours peak sunlight hours represents.



Researching calculation model experimenting

In this section I am going to try to deduce a formula to estimate the energy generated the next day. I think the main problems I have identified with the previous peak sunlight hours model is it only calculates energy generated in a specific time frame and not whilst sun is up. I think I need to find a way to calculate predicted energy in other times accurately. I decided to take some generation graphs for various months and see what similarities and differences they have. I will be looking at peak solar hours according to table and graphs, generation hours, when peak solar hours are, rough gradients of hours leading up to peak solar hours and wattage at peak solar hours (PSHs).

January graph



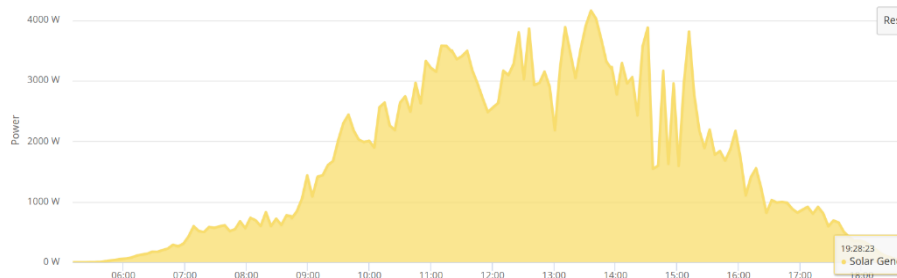
For this month

PSHs according to table: 0.73

PSHs according to graphs:
roughly similar

Peak wattage: 4000ish W

April graph



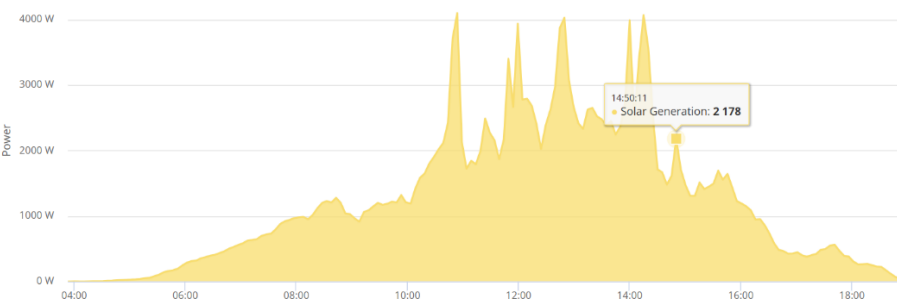
For this month

PSHs according to table: 4.14

PSHs according to graphs:
Very similar

Peak wattage: 4000ish W

July graph



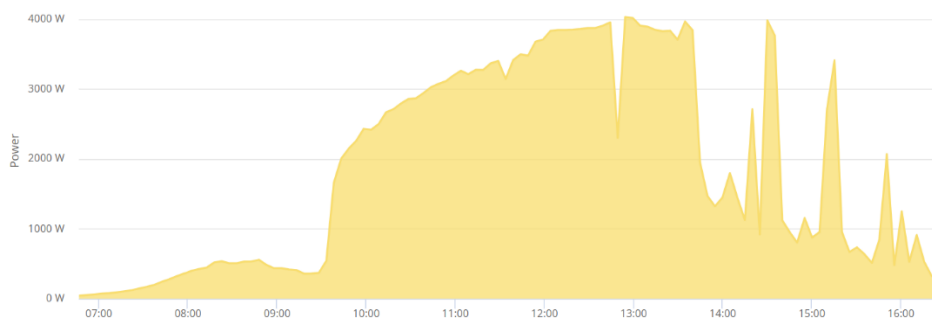
For this month

PSHs according to table: 4.82

PSHs according to graphs:
Very similar

Peak wattage: 4000ish W

October graph



For this month

PSHs according to table: 2

PSHs according to graphs:
slightly similar

Peak wattage: 4000ish W

Finding when centre point of peak sun hours is

January 2024

Day	Peak sun hours centred
06	13:30
23	13:15
24	13:00
25	12:00
30	12:20
Average	12:50

April

Day	Peak sun hours centred
12	13:00
13	12:45
14	13:25
19	14:30
20	13:20
21	12:55
23	14:00
Average	13:25

July

Day	Peak sun hours centred
07	12:45
13	12:40
17	12:50
18	13:30
19	13:30
28	13:20
29	13:20
30	13:00
31	13:20
Average	13:08

Oct

Day	Peak sun hours centred
03	12:50
05	12:45
07	12:30
19	13:45
26	13:00
Average	13:00

To find these I went through the past graphs for every day for that month and when they had a slightly smooth graph with not many spikes and a max of around 4000+watts I assumed that day was not cloudy and could estimate the mid-point of maximum plateau which is the peak sun hours. My results indicate an average of 13:06 which I feel is representative.

Final information

Now I know that peak wattage doesn't change based on the month and the time that PSHs are centred around doesn't change based on the month it makes the calculation easier. The area under the graph leading up to peak sun hours generation time can be modelled as two triangles to find the area underneath as we have the times that the sun is up using the `is_day` weather API variable and can calculate the time it starts being peak sun hours, the change in times is the base of the triangle. The height of the triangle is the cloud cover percentage times by the peak wattage of 4.32 KW (original peak wattage minus 1% per year has been in place). The final area formula is base times height divided by 2, this process is repeated on the later triangle to find generation then. I will need a function to calculate average cloud cover for these periods of start day, peak day and end day generation which should be easy enough.

To find the energy generated in peak sun hours:

Calculate the time period of peak sun hours, this can be done using 13:06 time and taking away half of the Peak Sun Hours and adding the other half to this time to find the start of peak sun hours and end of it. For example, in April this range would be 13:06 takeaway 2.07 hours and add 2.07 hours to get a range of 11:02 to 15:10. Round to the nearest half hour for simplicity. So, Aprils range would be 11:00 to 15:00. For this range find energy generated in each hour by doing $(1 - \text{cloud coverage})$ for the hour times by 4.32 KWs and add all energy generated in each hour up.

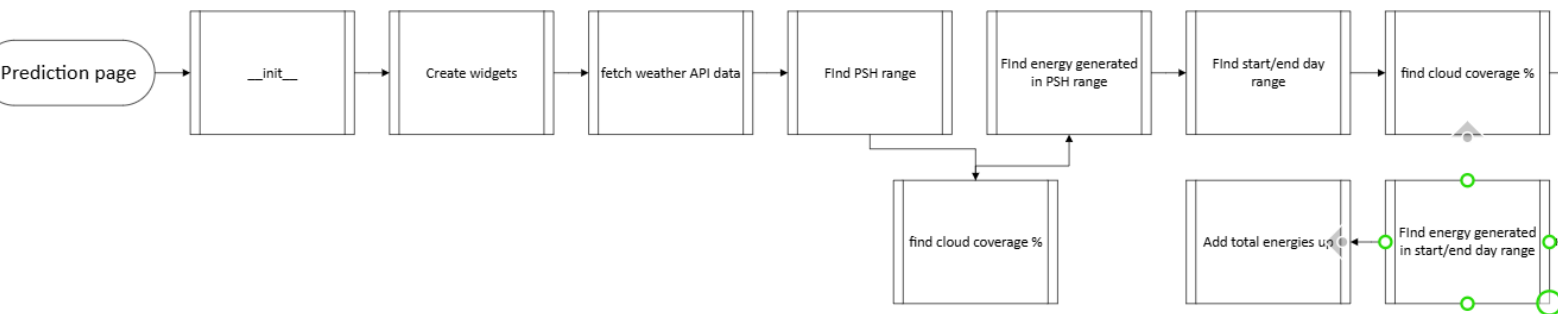
To find the energy generated in start day and end day hours:

Find the hours that are true for `is_day` value and before peak sun hour range times by average $(1 - \text{cloud coverage})$ times by 4.32 KWs times by 0.5. Do the same for after peak sun hours and true for `is_day` values.

API code iteration 5

In this iteration I am going to create lots of methods for the complex calculation I need to carry out including a find PSHs range, find start/end day hours, find PSHs energy generated, and a find start/end day energy generated. I will also need to include other methods including find cloud coverage and PSHs but may need to slightly edit them to fit my needs.

API code design 5



This design reuses modules already created and uses new modules to find prediction for the total energy generated.

API code development 5

```

def create_widgets(self, email, user_data):
    self.frame.grid_rowconfigure(0, weight=1)
    self.frame.grid_rowconfigure(3, weight=1)
    self.frame.grid_columnconfigure(0, weight=1)
    self.frame.grid_columnconfigure(2, weight=1)

    back_button = tk.Button(self.frame, text="Back", command=lambda: app.open_welcome_page(email, user_data))
    back_button.grid(row=0, column=0, padx=20, pady=20)

    title_label = tk.Label(self.frame, text=f"Predictions for, {email}!")
    title_label.grid(row=1, column=1, padx=20, pady=20)

    WA_data = self.fetch_WA_data()
    forecast_hourly = WA_data['forecast'][1]['forecastday'][1]['hour']
    PSHs_range = self.find_PSHs_range()
    print(forecast_hourly)
    print(PSHs_range[0])
    print(PSHs_range[1])

    cloud_coverage_percent_PSHs = self.cloud_coverage_percent_during_period(forecast_hourly, PSHs_range, 1)
    print(cloud_coverage_percent_PSHs, "for PSHs")
    P_KWhs_PSHs = ((1 - cloud_coverage_percent_PSHs) * (PSHs_range[1] - PSHs_range[0]) * 4.5)
    print(P_KWhs_PSHs, "during PSHs")

    cloud_coverage_percent_start = self.cloud_coverage_percent_during_period(forecast_hourly, PSHs_range, 0)
    print(cloud_coverage_percent_start, "for start")
    day_s = self.find_hour(forecast_hourly, 0)
    print(day_s, "day start")
    P_KWhs_start = ((1 - cloud_coverage_percent_start) * (PSHs_range[0] - day_s) * 4.5 * 0.5)
    print(P_KWhs_start, "power start day")

    cloud_coverage_percent_end = self.cloud_coverage_percent_during_period(forecast_hourly, PSHs_range, 2)
    print(cloud_coverage_percent_end, "cc for end")
    day_e = self.find_hour(forecast_hourly, 1)
    print(day_e, "day end")
    P_KWhs_end = ((1 - cloud_coverage_percent_end) * (day_e - PSHs_range[1]) * 4.5 * 0.5)
    print(P_KWhs_end, "power end day")

    test_label = tk.Label(self.frame, text=str(P_KWhs_PSHs + P_KWhs_start + P_KWhs_end))
    test_label.grid(row=2, column=1, padx=20, pady=20)
    
```

The updates to my code use lots of methods that I will show later. First I find the range of hours that are peak solar hours using method `find_PSHs_range()` and print them for de-bugging purposes. Then I find the average cloud coverage for this time using my method `cloud_coverage_percent_during_period` for period and passing in `PSHs_range`, `forecast_hourly` and `1` which correlates to which

function in this method to use. Then I carry out my calculation that I found during research. Then I repeat this for start and end day periods but need a new function `find_hour()` to find the first hour that `is_day` is true and last hour that `is_day` is true, I pass in numbers to specify which I would like which reduces repeated code as I only need one method for finding different values, this is used to find range of start and end day hours, then I carry out the same calculation as before but times by 0.5 as I model these periods as right angled triangles that and base times height times 0.5 is equal to area of right angled triangle. Finally, I produce label and add all individual energies generated in different ranges.

```
def find_PSHs_range(self):
    now = datetime.now()
    day = now.day
    month = now.month
    year = now.year
    if month == 12:
        next_month = 1
        previous_month = 11
    elif month == 1:
        next_month = 2
        previous_month = 12
    else:
        next_month = month + 1
        previous_month = month - 1
    table = [
        [1,2,3,4,5,6,7,8,9,10,11,12],
        [0.73,1.64,2.87,4.14,4.56,5.08,4.82,3.89,3.67,2,1.15,0.72]
    ]
    _, last_day = calendar.monthrange(year, month)
    if day <= 8:
        PSBs = (table[1][month - 1]*5 + table[1][previous_month - 1]*3)/8
    elif day >= last_day - 8:
        PSBs = (table[1][month - 1]*5 + table[1][next_month - 1]*3)/8
    else:
        PSBs = table[1][month - 1]
    PSBs_start = 13.1 - PSBs/2
    PSBs_start = self.round_time(PSBs_start)
    PSBs_end = 13.1 + PSBs/2
    PSBs_end = self.round_time(PSBs_end)
    PSBs_range= [PSBs_start,PSBs_end]
    return PSBs_range
```

This method is similar to my old method find_PSHs() and has identical start but finds the start and end hours of PSBs and returns this as an array. Uses a round time function to round to the nearest half an hour. This uses integer concatenation to determine which half hour time is closest to.

```
def round_time(self, time):
    if time < int(time) + 0.25:
        time = int(time)
    if time > int(time) + 0.75:
        time = int(time) + 1
    else:
        time = int(time) + 0.5
    return time
```

```
def cloud_coverage_percent_during_period(self, forecast_hourly, range, start_PSHs_end):
    #print(forecast_hourly)
    total_cloud_coverage = 0
    num_entries_in_range = 0
    if start_PSHs_end == 0:
        for entry in forecast_hourly:
            hour = int(entry['time'].split()[1].split(":")[0])
            if hour >= range[0] and hour < range[1]:
                total_cloud_coverage += entry['cloud']
                num_entries_in_range += 1

    if start_PSHs_end == 1:
        for entry in forecast_hourly:
            hour = int(entry['time'].split()[1].split(":")[0])
            if entry['is_day'] == 1 and hour < range[0]:
                total_cloud_coverage += entry['cloud']
                num_entries_in_range += 1

    if start_PSHs_end == 2:
        for entry in forecast_hourly:
            hour = int(entry['time'].split()[1].split(":")[0])
            if entry['is_day'] == 1 and hour >= range[1]:
                total_cloud_coverage += entry['cloud']
                num_entries_in_range += 1

    cloud_coverage_percent = (total_cloud_coverage/num_entries_in_range)/100
    #print(num_entries_in_range)
    return cloud_coverage_percent
```

This uses iterations and decisions similar to those that were used in my old cloud coverage percent method but includes a decision to limit the value total cloud coverage to include entities only within range passed into method. The start_PSHs_end variable determines which cloud coverage percent to find and return and uses ranges found in previous methods.

```
def find_hour(self, forecast_hourly, start_end):
    if start_end == 0:
        n = 0
        for entry in forecast_hourly:
            if entry['is_day'] == 1 and n == 0:
                hour = int(entry['time'].split()[1].split(':')[0])
                n += 1

    if start_end == 1:
        n = 0
        for entry in forecast_hourly:
            if entry['is_day'] == 0 and int(entry['time'].split()[1].split(':')[0]) > 12 and n == 0:
                hour = int(entry['time'].split()[1].split(':')[0]) - 1
                n += 1

    return hour
```

My find_hour method uses similar way to decide which output is requested as used in cloud coverage method of passing in integer. It uses a for loop to check through every entity in forecast_hourly to find

when entity is_day is equal to one. The n is used to ensure only first entity found is outputted as otherwise the final entity where is_day is equal to one would be outputted. Finding the end hour was slightly more difficult because I needed to find the entity that changes from is_day equal to one to is_day equal to zero which took a lot of concatenation, and pre-requirements in decision to skip past the early hours of day where is_day is equal to zero and skip past all mid-day hours where is_day is equal to one to find the first hour in the afternoon that is_day is equal to zero and the hour before this is the final day hour and is outputted.

API code testing 5 (used to inform evaluation)

This is the predictions for 10/03/2025 on 09/03/2025.

The terminal window shows the output of a Python script. The output includes a list of weather data points for 2025-03-10, including time, is_day, and cloud coverage. It also shows the execution of a Tkinter window titled 'Predictions for wilf.wallace1@outlook.com!'. The Tkinter window displays a weather forecast interface with a 'Back' button, a 'Predictions for wilf.wallace1@outlook.com!' label, and a numerical value '15.453000000000001'.

This is the actual cloud coverage for 10/03/2025 on 10/03/2025, which is much more cloudy than predicted.

```
{'location': {'name': 'Stonehouse', 'region': 'Gloucestershire14583', 'country': 'UK', 'lat': 51.7430000305176, 'lon': -2.28399991989136, 'tz_id': 'Europe/London', 'localtime_epoch': 1741603235, 'localtime': '2025-03-10 10:40'}, 'current': {'last_updated': '2025-03-10 10:30', 'is_day': 1, 'cloud': 75}, 'forecast': {'forecastday': [{'date': '2025-03-10', 'day': {}, 'astro': {'sunrise': '06:34 AM', 'sunset': '06:05 PM', 'is_sun_up': 0}, 'hour': [{'time': '2025-03-10 00:00', 'is_day': 0, 'cloud': 50}, {'time': '2025-03-10 01:00', 'is_day': 0, 'cloud': 92}, {'time': '2025-03-10 02:00', 'is_day': 0, 'cloud': 66}, {'time': '2025-03-10 03:00', 'is_day': 0, 'cloud': 44}, {'time': '2025-03-10 04:00', 'is_day': 0, 'cloud': 59}, {'time': '2025-03-10 05:00', 'is_day': 0, 'cloud': 99}, {'time': '2025-03-10 06:00', 'is_day': 1, 'cloud': 100}, {'time': '2025-03-10 07:00', 'is_day': 1, 'cloud': 100}, {'time': '2025-03-10 08:00', 'is_day': 1, 'cloud': 100}, {'time': '2025-03-10 09:00', 'is_day': 1, 'cloud': 100}, {'time': '2025-03-10 10:00', 'is_day': 1, 'cloud': 75}, {'time': '2025-03-10 11:00', 'is_day': 1, 'cloud': 57}, {'time': '2025-03-10 12:00', 'is_day': 1, 'cloud': 73}, {'time': '2025-03-10 13:00', 'is_day': 1, 'cloud': 96}, {'time': '2025-03-10 14:00', 'is_day': 1, 'cloud': 100}, {'time': '2025-03-10 15:00', 'is_day': 1, 'cloud': 100}, {'time': '2025-03-10 16:00', 'is_day': 1, 'cloud': 100}, {'time': '2025-03-10 17:00', 'is_day': 1, 'cloud': 100}, {'time': '2025-03-10 18:00', 'is_day': 1, 'cloud': 100}, {'time': '2025-03-10 19:00', 'is_day': 0, 'cloud': 100}, {'time': '2025-03-10 20:00', 'is_day': 0, 'cloud': 100}, {'time': '2025-03-10 21:00', 'is_day': 0, 'cloud': 98}, {'time': '2025-03-10 22:00', 'is_day': 0, 'cloud': 100}, {'time': '2025-03-10 23:00', 'is_day': 0, 'cloud': 100}, {'time': '2025-03-11 00:00', 'is_day': 0, 'cloud': 100}], 'date': '2025-03-11', 'day': {}, 'astro': {'sunrise': '06:32 AM', 'sunset': '06:07 PM', 'is_sun_up': 0}, 'hour': [{'time': '2025-03-11 00:00', 'is_day': 0, 'cloud': 100}, {'time': '2025-03-11 01:00', 'is_day': 0, 'cloud': 100}]}]}
```

This is the actual energy generation for 10/03/2025 which is much less than predicted, hopefully due to wrong predictions from weather API.

Generation Today
4.60kWh

This is the predictions for the 11/03/2025, I slightly altered my calculations to include the decrease in efficiency factor due to the number of years it has been installed in the hope that this is part of what was missing and tomorrows generation will be close to this (I think within 2 KWhs either side should be good enough to calculate how much to charge battery to for this amount of generation.

```

275 class PredictionPage:
285     def create_widgets(self, email, user_data):
286         print(day_e, day_start)
320         P_KWhs_end = ((1 - cloud_coverage_percent_start)*(day_e - PSHs_range[1])*4.5*0.5*0.94)
321         print(P_KWhs_end, "power end day")
322
323         test_label = tk.Label(self.frame, text=str(P_KWhs_PSHs + P_KWhs_start + P_KWhs_end))
324         test_label.grid(row = 2, column = 1, padx = 20, pady = 20)
325

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

<Response [201]>
<Response [201]>
PS C:\Users\wilfw> & C:\Users\wilfw\AppData\Local\Microsoft\WindowsApps\python3.11.exe "c:/Users/wilfw/OneDrive/Documents/Computing A level/Post GUI iteration 4 API code iteration 1.py"

```

[{'time': '2025-03-11 00:00', 'is_day': 0, 'cloud': 100}, {'time': '2025-03-11 01:00', 'is_day': 0, 'cloud': 100}, {'time': '2025-03-11 02:00', 'is_day': 0, 'cloud': 100}, {'time': '2025-03-11 03:00', 'is_day': 0, 'cloud': 100}, {'time': '2025-03-11 04:00', 'is_day': 0, 'cloud': 98}, {'time': '2025-03-11 05:00', 'is_day': 0, 'cloud': 86}, {'time': '2025-03-11 06:00', 'is_day': 0, 'cloud': 60}, {'time': '2025-03-11 07:00', 'is_day': 1, 'cloud': 39}, {'time': '2025-03-11 08:00', 'is_day': 1, 'cloud': 38}, {'time': '2025-03-11 09:00', 'is_day': 1, 'cloud': 40}, {'time': '2025-03-11 10:00', 'is_day': 1, 'cloud': 32}, {'time': '2025-03-11 11:00', 'is_day': 1, 'cloud': 19}, {'time': '2025-03-11 12:00', 'is_day': 1, 'cloud': 100}, {'time': '2025-03-11 13:00', 'is_day': 1, 'cloud': 100}, {'time': '2025-03-11 14:00', 'is_day': 1, 'cloud': 67}, {'time': '2025-03-11 15:00', 'is_day': 1, 'cloud': 76}, {'time': '2025-03-11 16:00', 'is_day': 0, 'cloud': 71}, {'time': '2025-03-11 17:00', 'is_day': 0, 'cloud': 31}, {'time': '2025-03-11 18:00', 'is_day': 0, 'cloud': 62}, {'time': '2025-03-11 19:00', 'is_day': 0, 'cloud': 40}, {'time': '2025-03-11 20:00', 'is_day': 0, 'cloud': 48}]
11.5 PSHs start
14.5 PSHs start
0.344 cc for PSHs
11.09952 KWhs during PSHs
0.98 cc for start
7 day start
0.190350000000000016 power start day
0.76 cc for end
18 day end
0.148050000000000013 power end day

```



Generation Today
12.90kWh

As you can see the prediction was within 1.5 KWhs and after the changes made is therefore a suitable model to use.

```

<Response [200]>
PS C:\Users\wilfw> & C:\Users\wilfw\AppData\Local\Microsoft\WindowsApps\python3.11.exe "c:/Users/wilfw/OneDrive/Documents/Computing A level/Post GUI iteration 4 API code iteration 1.py"
[{'time': '2025-03-12 00:00', 'is_day': 0, 'cloud': 34}, {'time': '2025-03-12 01:00', 'is_day': 0, 'cloud': 87}, {'time': '2025-03-12 02:00', 'is_day': 0, 'cloud': 64}, {'time': '2025-03-12 03:00', 'is_day': 0, 'cloud': 40}, {'time': '2025-03-12 04:00', 'is_day': 0, 'cloud': 34}, {'time': '2025-03-12 05:00', 'is_day': 0, 'cloud': 31}, {'time': '2025-03-12 06:00', 'is_day': 0, 'cloud': 47}, {'time': '2025-03-12 07:00', 'is_day': 1, 'cloud': 80}, {'time': '2025-03-12 08:00', 'is_day': 1, 'cloud': 29}, {'time': '2025-03-12 09:00', 'is_day': 1, 'cloud': 99}, {'time': '2025-03-12 10:00', 'is_day': 1, 'cloud': 90}, {'time': '2025-03-12 11:00', 'is_day': 1, 'cloud': 100}, {'time': '2025-03-12 12:00', 'is_day': 1, 'cloud': 100}, {'time': '2025-03-12 13:00', 'is_day': 1, 'cloud': 100}, {'time': '2025-03-12 14:00', 'is_day': 1, 'cloud': 95}, {'time': '2025-03-12 15:00', 'is_day': 1, 'cloud': 100}, {'time': '2025-03-12 16:00', 'is_day': 1, 'cloud': 100}, {'time': '2025-03-12 17:00', 'is_day': 1, 'cloud': 100}, {'time': '2025-03-12 18:00', 'is_day': 1, 'cloud': 90}, {'time': '2025-03-12 19:00', 'is_day': 0, 'cloud': 80}, {'time': '2025-03-12 20:00', 'is_day': 0, 'cloud': 90}, {'time': '2025-03-12 21:00', 'is_day': 0, 'cloud': 95}, {'time': '2025-03-12 22:00', 'is_day': 0, 'cloud': 87}, {'time': '2025-03-12 23:00', 'is_day': 0, 'cloud': 100}]
11.5 PSHs start
14.5 PSHs start
0.6050000000000000 cc for PSHs
5.0512000000000000 KWhs during PSHs
0.9133333333333333 cc for start
7 day start
0.30405 power start day
0.99 cc for end
18 day end
0.56102 power end day

```



Generation Today
13.50kWh

As you can see the prediction was very far off and in the past two days the value has been too low, therefore I am going to remove my 0.94 factor.

```

C:\Users> will > OneDrive > Documents > Post GUI Iteration 4 API code iteration 1.py > create_widgets.py > create_widgets.py
275 class PredictionPage:
285     def create_widgets(self, email, user_data):
295         cloud_coverage_percent_PSHs = self.cloud_coverage_percent_during_per_100h(forecast_hourly, PSHs_range, 4)
305         print(cloud_coverage_percent_PSHs, "cc for PSHs")
315         P_KWhs_PSHs = ((1 - cloud_coverage_percent_PSHs) * ((PSHs_range[1] + 1) - PSHs_range[0]) * 4.5)
325         print(P_KWhs_PSHs, "KWhs during PSHs")
335
345         cloud_coverage_percent_start = self.cloud_coverage_percent_during_period(forecast_hourly, PSHs_range, 0)
355         print(cloud_coverage_percent_start, "cc for start")
365         day_s = self.find_hour(forecast_hourly, 0)
375         print(day_s, "day start")
385         P_KWhs_start = ((1 - cloud_coverage_percent_start) * (PSHs_range[0] - day_s) * 4.5 * 0.5)
395         print(P_KWhs_start, "power start day")

```

```

0', 'is_day': 0, 'cloud': 100}, {'time': '2025-03-13 04:00', 'is_day': 0, 'cloud': 100}, {'time': '2025-03-13 05:00', 'is_day': 0, 'cloud': 100}, {'time': '2025-03-13 06:00', 'is_day': 0, 'cloud': 77}, {'time': '2025-03-13 07:00', 'is_day': 1, 'cloud': 80}, {'time': '2025-03-13 08:00', 'is_day': 1, 'cloud': 97}, {'time': '2025-03-13 09:00', 'is_day': 1, 'cloud': 88}, {'time': '2025-03-13 10:00', 'is_day': 1, 'cloud': 71}, {'time': '2025-03-13 11:00', 'is_day': 1, 'cloud': 68}, {'time': '2025-03-13 12:00', 'is_day': 1, 'cloud': 80}, {'time': '2025-03-13 13:00', 'is_day': 1, 'cloud': 90}, {'time': '2025-03-13 14:00', 'is_day': 1, 'cloud': 100}, {'time': '2025-03-13 15:00', 'is_day': 1, 'cloud': 100}, {'time': '2025-03-13 16:00', 'is_day': 1, 'cloud': 100}, {'time': '2025-03-13 17:00', 'is_day': 1, 'cloud': 100}, {'time': '2025-03-13 18:00', 'is_day': 1, 'cloud': 100}, {'time': '2025-03-13 19:00', 'is_day': 0, 'cloud': 64}, {'time': '2025-03-13 20:00', 'is_day': 0, 'cloud': 38}, {'time': '2025-03-13 21:00', 'is_day': 0, 'cloud': 35}, {'time': '2025-03-13 22:00', 'is_day': 0, 'cloud': 28}, {'time': '2025-03-13 23:00', 'is_day': 0, 'cloud': 32}]
11.5 PSHs start
14.5 PSHs start
0.8220000000000001 cc for PSHs
3.2039999999999999 KWhs during PSHs
0.9 cc for start
7 day start
1.0124999999999997 power start day
1.0 cc for end
18 day end
0.7874999999999999 power end day
percent ish 0.854237288135593226
percent 0.7810576271186443
<Response [201]>
{'data': {'value': 78, 'success': True, 'message': 'Written Successfully'}}
{'data': {'value': 78, 'success': True, 'message': 'Written Successfully'}}

```

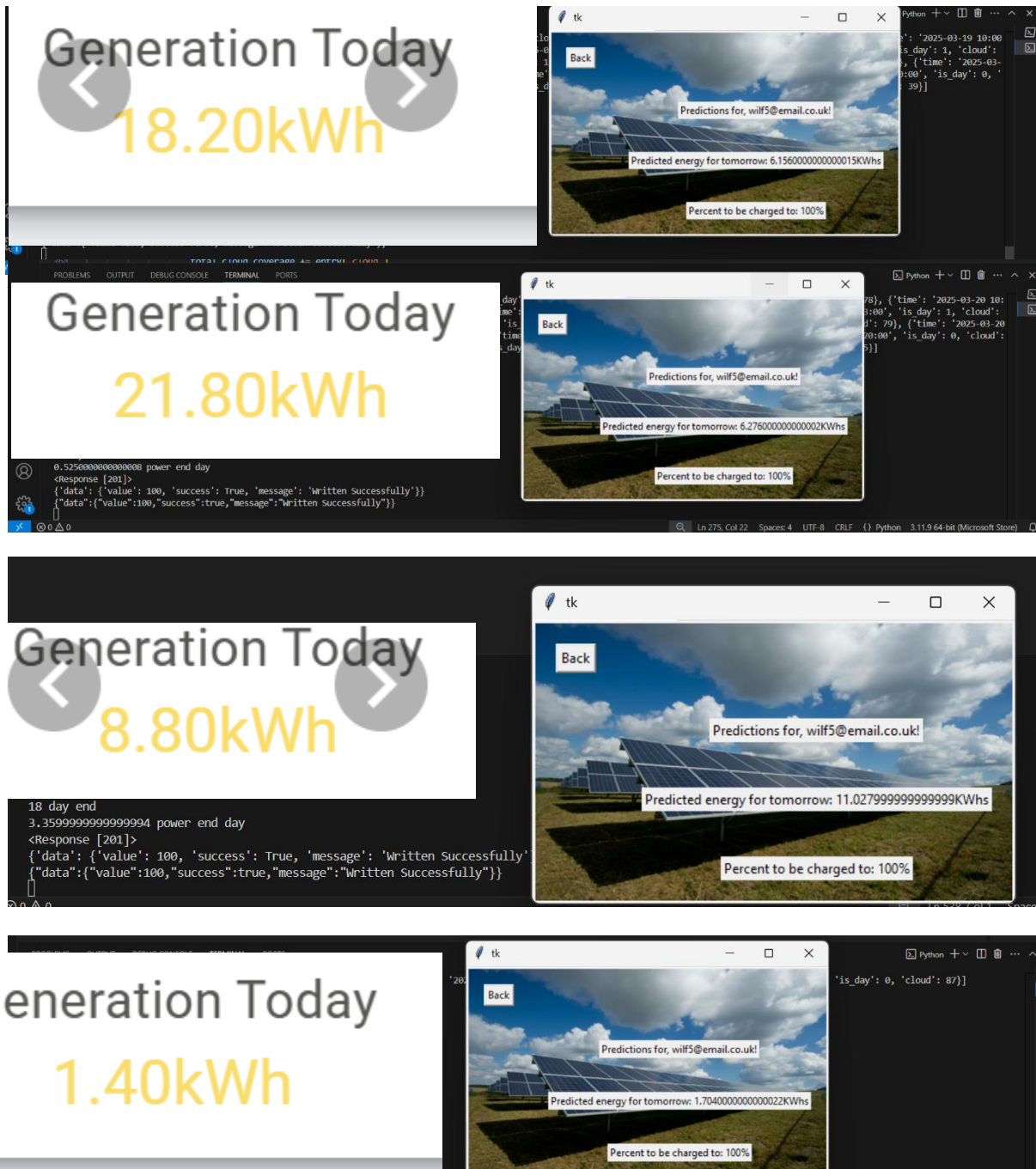
Generation Today
9.60kWh

As you can see the generation for today was a fair bit off predicted value, about 1.4 KWhs, today was due to weather predictions not being accurate as was more sunny than anticipated.



Today's energy generation prediction was a fair bit off, due to poorly predicted weather conditions, but still a good enough prediction that a good estimate for battery percentage can be found.





API code evaluation 5

My code works fairly well but is often in-accurate, usually to changing weather conditions than previously predicted. I think it is often wrong due to the rounding of peak solar hours which means there are sometimes hours unaccounted for or over accounted for.

API code iteration 6

In this iteration I will clean up the display on predictions page, add more descriptive labels and begin writing to givenenergy.

```
> Post GUI iteration 4 API code iteration 1.py string.delete.py x Basecodepython
```

```
C> Users\wilfw > OneDrive > Documents > Computing A level > string.delete.py ...
```

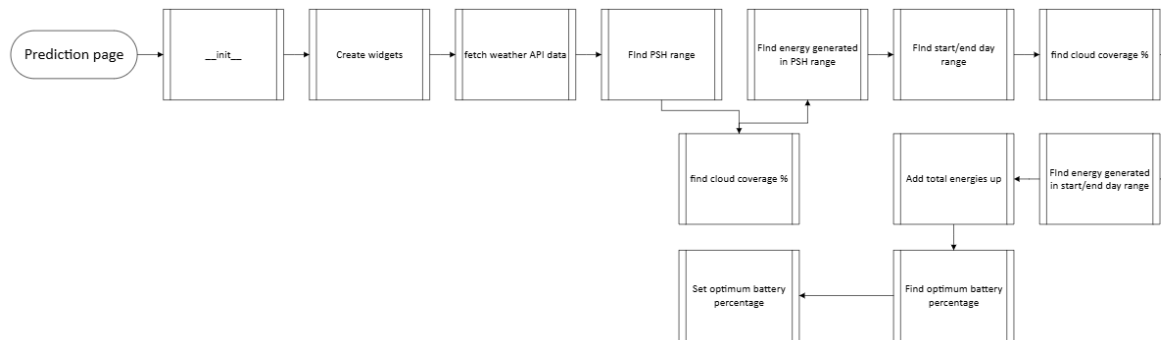
```
1 import requests
2 import json
3 id = "SD2245G010"
4 url = f'https://api.givenenergy.cloud/v1/inverter/{id}/settings/77/write'
5 GEKey = "eyJ0eXAiOiJKV1QiLCJhbGciOiJSUzI1NiJ9.eyJhdWQiOiJIOTc3MDIXOS1jYWE2LTFlOTUyLWZlbnRpb24iLCJyb2wiOiJ0bm9keSBkaXNpdCIsImF1dG8iOiJ1b250cm9udGVudCIsImF1dG8iOiJ1b250cm9udGVudC"
6 percent = "50"
7 payload = {
8     "value": percent,
9 }
10 headers = {
11     "Authorization": f'Bearer {GEKey}',
12     "Content-Type": 'application/json',
13     "Accept": 'application/json'
14 }
15
16 response = requests.request('POST', url, headers=headers, json=payload)
17 print(response)
18 response.json()
19 print(response.json())
20 print(response.text)
```

```
"message": "Not Found"
```

```
PS C:\Users\wilfw> C:\Users\wilfw\AppData\Local\Microsoft\WindowsApps/python3.11.exe "<br></div>"</pre>
```

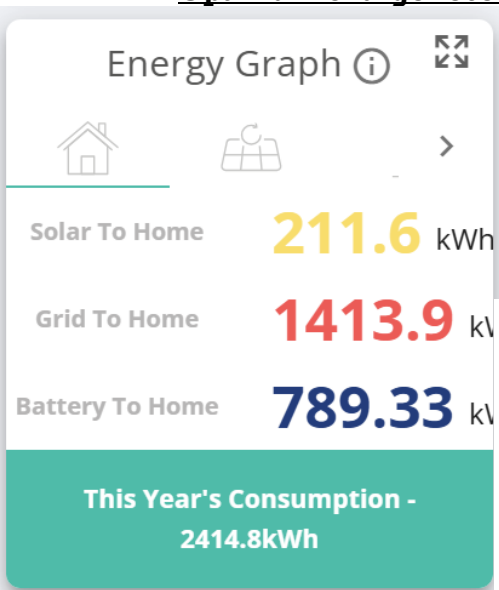
API code design 5

This code updates the value stored in givenergy that determines how much percentage to charge the battery to during period from 00:00 to 05:00 in the morning which is the desired outcome of my project. I now need to add this code into a method in predictions page and a method to calculate the optimum battery charge to start the day with. This was quite easy to code thanks to the givenergy API documentation and the teravolt givenergy API documaentation page. The 77 in url specifies where value is going and what value is.

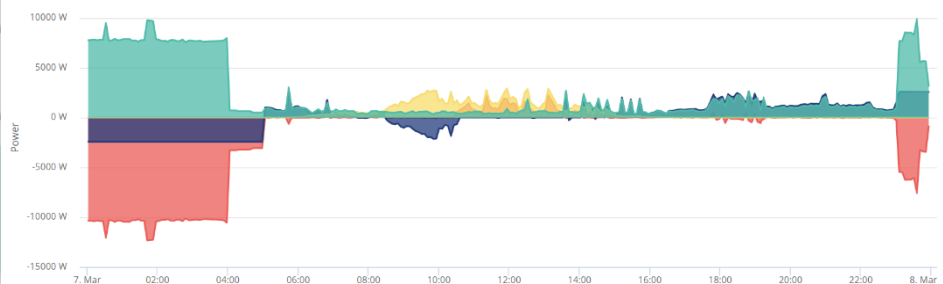


This new design of class prediction page shows my two new methods.

Optimum charge research



This shows the energy used so far this year, using this I can find the average energy consumed per day. $2414.8/67$ is equal to 36 KWhs a day. This value is similar to average values for energy consumed per day for last week and last month so is a good value to use. My battery maximum capacity is 14.75 KWhs and therefore on a day where sub 10 KWhs will be generated should be charged to 100%



After looking at past energy graphs I realised the majority of the demanded energy was coming from the times when battery was being charged and that during the day the real demanded energy is much lower. In most graphs the demanded power varies from about 500 to 1100 watts with the occasional random spike in demand. So I decided to do 900 times the usual time periods of this to find the “real” demanded energy. 800 times 17 is equal to 15.3 KWhs which will be the value used from now on when referring to energy demand. I think I will need another formula that will determine how much to charge battery to. This will include variables such as the average “real” demanded energy and

how much solar energy will be generated. I think we need enough battery to last until solar energy starts generating which is until `is_day` is equal to one times by maybe 800 watts based on previous graphs. This is the minimum battery will be charged too called “pre-day battery”. Will also add

on how much to charge based on how much solar energy will be generated, if solar energy predicted is too big and percentage is negative, should charge to 4%, if solar energy predicted is too small, and percentage is bigger than 11 should charge to 100%.

API code development 5

This is the code I added into my class create widgets that calls my two new methods created in this iteration and displaying the percent it will charge battery to as a label.

```

328         percent = self.find_optimum_percent(total_P_KWhs, day_s)
329         self.write_percent(percent, email, user_data)
330         percent_label = tk.Label(self.frame, text=str(percent))
331         percent_label.grid(row = 3, column = 1, padx = 20, pady = 20)

```

```

def find_optimum_percent(self, predicted_energy, day_s):
    pre_day_energy = (int(day_s) - 6)*0.8
    if pre_day_energy < 0:
        pre_day_energy = 0
    if 14.75 <= predicted_energy:
        percent = pre_day_energy/14.75
        print("percent",percent)
    else:
        energy_needed_after_pre_day = 14.75 - predicted_energy
        percent_ish = pre_day_energy/14.75
        percent = (energy_needed_after_pre_day/14.75)*1.1 + percent_ish
        print("percent_ish",percent_ish)
        print("percent",percent)
    if percent >= 1:
        percent = 1
    if percent <= 0.04:
        percent = 0.04
    return percent*100

```

This method in my code is responsible for calculating the optimum battery percentage, not too high that user ends up selling energy to the grid because of no where to store excess but also not too low that user buys energy at peak rates instead of at lower rates from midnight to five. It works by finding the demanded energy before solar panels start generating and then finding the battery that will be needed to store the predicted energy and ensuring there is space for this it then totals them. I added in a factoring tool as part of battery will be used up before

it starts generating. I also added if statements to ensure the percent is in the valid range of 0 to 100.

```

433     def write_percent(self, percent, email, user_data):
434         id = self.binary_search(email, user_data, 'Solar ID')
435         GKey = self.binary_search(email, user_data, 'API Key')
436         url = f'https://api.givenergy.cloud/v1/inverter/{id}/settings/77/write'
437         payload = {
438             "value": percent,
439         }
440         headers = {
441             'Authorization': f'Bearer {GKey}',
442             'Content-Type': 'application/json',
443             'Accept': 'application/json'
444         }
445
446         response = requests.request('POST', url, headers=headers, json=payload)
447         print(response)
448         print(response.json())
449         print(response.text)

```

This code is very similar to the code experimented with in the recent research section, only difference is it searches for the user's id and key through the Edexcel database using the binary search method used multiple times previously which saves me much time thanks to how re-usable my code is.

API code testing 5

My code works well and updates the value held in givenergy to the new percent, the only doubt is whether or not the percentage is a valid percentage. This can be tested by finding out whether or not the energy graph for that day uses energy from the grid when percentage is not charged to 100. If battery is not charged to a hundred should not import from the grid at all as there must be enough solar energy to meet battery and household usage needs, may export but not import.

This mode will trigger your battery to charge at the times you specify

```

279 class PredictionManager:
280     def create_widgets(self, email, user_data):
281         cloud_coverage_percent_PSHs = self.cloud_coverage_percent_during_period(
282             P_KWhs_PSHs = ((1-cloud_coverage_percent_PSHs)*(PSHs_range[1] + 1)-PSHs_range[0])
283             print(P_KWhs_PSHs, "KWhs during PSHs")
284
285         cloud_coverage_percent_start = self.cloud_coverage_percent_during_period(
286             print(cloud_coverage_percent_start, "cc for start")
287             day_s = self.find_hour(forecast_hourly, 0)
288             print(day_s, "day start")
289             P_KWhs_start = ((1 - cloud_coverage_percent_start)*(PSHs_range[0] - day_s)*
290             print(P_KWhs_start, "power start day")

```

Slot 1

Start Time: 00:00 End Time: 05:00

Enabled ☒

Charge To 78%

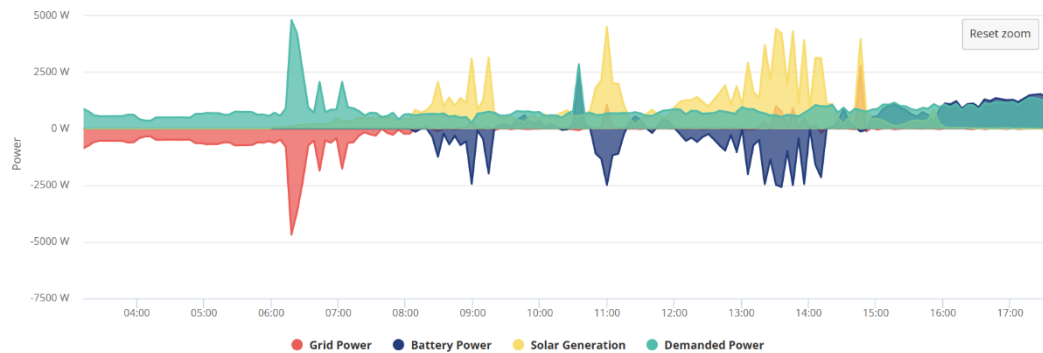
SUBMIT



Date	Battery% charged to	Exported to grid (on day) estimate	Times exported to grid	Imported from grid (peak hours) (on day)	Times imported from grid
13/03/2025	78	0.6KWhs	11:00,14:00	2.4 KWhs	5:00-8:00
14/03/2025	100	0 KWhs	n/a	4.5KWhs	13:00-24:00
17/03/2025	100	0 KWhs	n/a	6-7KWhs	23:00-24:00

13/03/2025 overview

Graph:



After looking over my code I realised that the calculations didn't take into account demanded energy from household usage which is likely to take up energy from solar generation and battery storage.

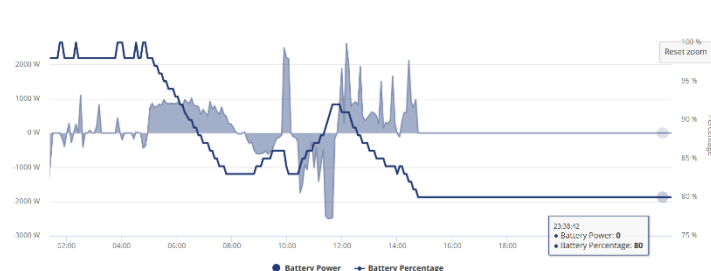
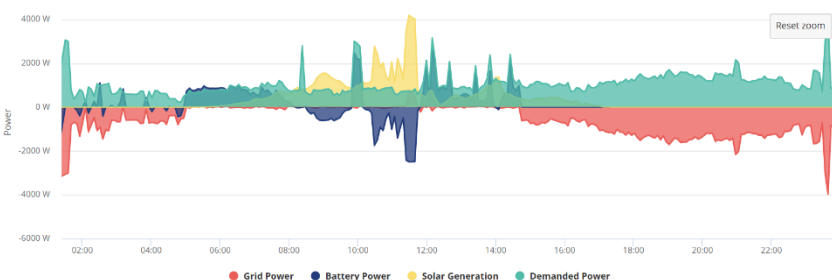
Fixed code: I needed to edit my code to only not charge to 100% if excess energy was going to be generated, this required including energy demanded after solar panels had started generating, if it is going to be excess, should charge to the highest values whilst leaving room for excess energy to be stored in battery. If the excess energy is greater than battery capacity charge to the amount of battery that will be used up whilst solar panels are not generating (pre_day energy + 0.04) the (+0.04) part is because battery won't discharge below 4 percent so this needs to be accounted for in this scenario.

```
def find_optimum_percent(self, predicted_energy, day_s):
    pre_day_energy = (int(day_s) - 6)*0.8
    energy_needed_after_pre_day = 15.3 - pre_day_energy
    if pre_day_energy < 0:
        pre_day_energy = 0
    if predicted_energy - energy_needed_after_pre_day > 14.75:
        percent = pre_day_energy/14.75 + 0.04
    elif predicted_energy > energy_needed_after_pre_day:
        percent = 1 - (predicted_energy - energy_needed_after_pre_day)/14.75 + pre_day_energy/14.75
    elif energy_needed_after_pre_day > predicted_energy:
        percent = 1
    if percent > 1:
        percent = 1
    if percent < 0.04:
        percent = 0.04
    return percent*100
```

For some reason, as displayed on the graph, my inverter refuses to use battery charge until after 8:00, need to check if this is a re-occurring theme and if so, can it be fixed and how to fix it.

14/03/2025 overview

Graphs:

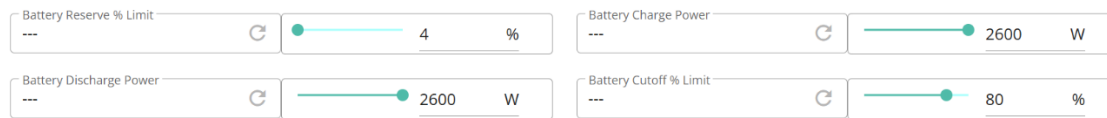


The graphs for this day show that the battery was charged to 100% correctly as there was little solar generation, however the inverter decided to not use most of battery and used none of battery past 15:00/once battery hit 80%.

I located the source of the problem the battery cutoff percentage was set to 80%

which means would not use battery charge once reaching this limit.

Percentage Limits



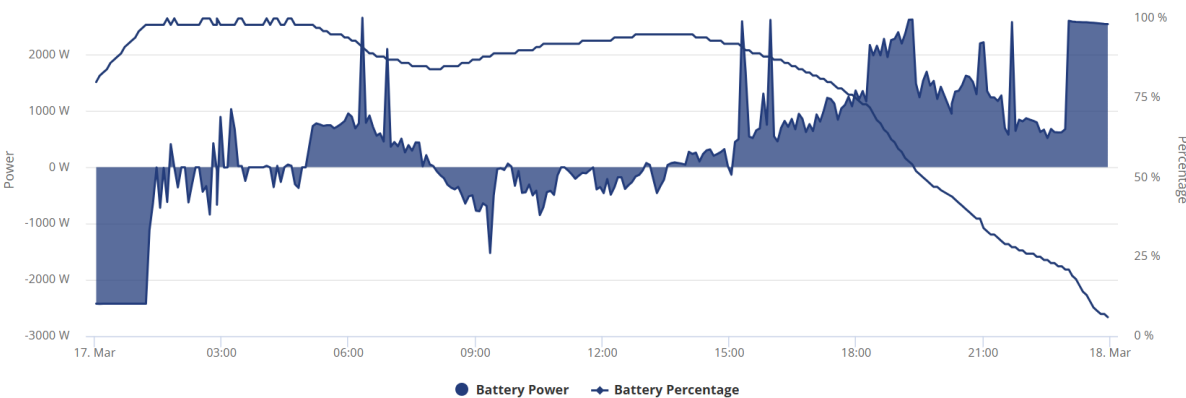
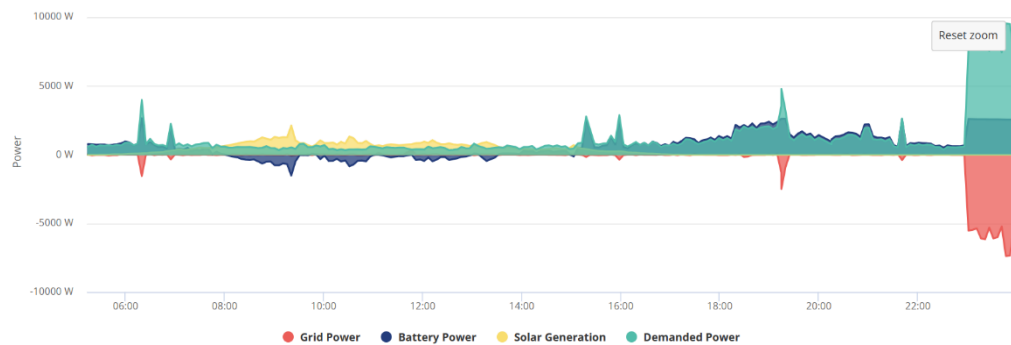
17/03/2025

Graph:

My graph indicates that during peak hours nearly no energy was imported from the grid and no energy was exported to the grid.

During hours from 23:00

to 24:00 there was a large spike which demanded more power than battery can output (over 2600W) where grid was needed. This was a succesful battery percentage to be charged to.



Testing using physical inputs to inform evaluation

```

1 def find_optimum_percent(predicted_energy, day_s):
2     pre_day_energy = (int(day_s) - 6)*0.8
3     energy_needed_after_pre_day = 15.3 - pre_day_energy
4     if pre_day_energy < 0:
5         pre_day_energy = 0
6     if predicted_energy - energy_needed_after_pre_day > 14.75:
7         percent = pre_day_energy/14.75 + 0.04
8     elif predicted_energy > energy_needed_after_pre_day:
9         percent = 1 - (predicted_energy - energy_needed_after_pre_day)/14.75 + pre_day_energy/14.75
10    elif energy_needed_after_pre_day > predicted_energy:
11        percent = 1
12    if percent > 1:
13        percent = 1
14    if percent < 0.04:
15        percent = 0.04
16    return percent*100
17
18 percent = find_optimum_percent(13.2, 5)
19 print(percent)

```

To test for a range of data (sort of white-box testing) I needed to choose the data inputted so I copied the program and edited it slightly to check if inputs aligned to desired outputs.

Pre_day_energy = p_d_e energy_needed_after_pre_day = e_n_a_p_d

Test #	Test name/purpose	Test data (predicted energy)	Test data (day start)	Expected result	Actual result
1	Test sub-6-hour input to check it deals with it appropriately, (no syntax error)	13.2	5	% = 100, p_d_e = 0	% = 100 p_d_e = 0
2	Test over 6-hour input to check it calculates p_d_e correctly	13.2	8	% = 100, p_d_e = 1.6	% = 100, p_d_e = 1.6
3	Test p_e just under e_n_a_p_d to ensure it makes right decision	13.6	8	% = 100, p_d_e = 1.6	% = 100, p_d_e = 1.6
4	Test p_e just over e_n_a_p_d to ensure correct decision is made	13.9	8	90 < % < 100 P_d_e = 1.6	% = 100 P_d_e = 1.6
5	As above. Forgot to account for fact the correct decision adds p_d_e/14,75 onto percentage which can cause to be over 100%.	15.5	8	90 < % < 100 P_d_e = 1.6	% = 98.6 P_d_e = 1.6
6	Test the lowest p_e generated that would lead is larger than e_n_a_p_d	28.4	8	10 < % < 15 P_d_e = 1.6	% = 11.2 P_d_e = 1.6
7	Test that higher than 28.4 p_e percentage is equal to p_d_e/14.75 which is 10% for 8 day_s	31	8	10 < % < 11 P_d_e = 1.6	% = 10.8 P_d_e = 1.6
8	Test that when p_d_e is zero and p_e is higher then 31	31	6	4 < % < 6 P_d_e = 0	% = 4.0 P_d_e = 0

Test 1

```
30 percent = 0.04
31 print("p_d_e is, ",pre_day_energy)
32 return percent*100
33
34 percent = find_optimum_percent(13.2, 5)
35 print("percent is, ",percent)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
100
PS C:\Users\wilfw> & C:/Users/wilfw/AppData/Local/Micro
p_d_e is, 0
percent is, 100
PS C:\Users\wilfw> 
```

Test 2

```
34 percent = find_optimum_percent(13.2, 8)
35 print("percent is, ",percent)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
percent is, 100
PS C:\Users\wilfw> & C:/Users/wilfw/AppData/Local/Micro
p_d_e is, 1.6
percent is, 100
PS C:\Users\wilfw> 
```

Test 3

```
34 percent = find_optimum_percent(13.6, 8)
35 print("percent is, ",percent)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
percent is, 100
PS C:\Users\wilfw> & C:/Users/wilfw/AppData/Local/Micro
p_d_e is, 1.6
percent is, 100
PS C:\Users\wilfw> 
```

Test 4

```
33
34 percent = find_optimum_percent(13.9, 8)
35 print("percent is, ",percent)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\wilfw> & C:/Users/wilfw/AppData/Local/Micro
p_d_e is, 1.6
percent is, 100
PS C:\Users\wilfw> 
```

Test 5

```
34 percent = find_optimum_percent(13.9+1.6, 8)
35 print("percent is, ",percent)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\wilfw> & C:/Users/wilfw/AppData/Local/Micro
p_d_e is, 1.6
percent is, 98.64406779661017
PS C:\Users\wilfw> 
```

Test 6

```
34 percent = find_optimum_percent(28.4, 8)
35 print("percent is, ",percent)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\wilfw> & C:/Users/wilfw/AppData/Local/Micro
p_d_e is, 1.6
percent is, 11.18644067796612
PS C:\Users\wilfw> 
```

Test 7

```
34 percent = find_optimum_percent(31, 8)
35 print("percent is, ",percent)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\wilfw> & C:/Users/wilfw/AppData/Local/Micro
p_d_e is, 1.6
percent is, 10.847457627118645
PS C:\Users\wilfw> 
```

Test 8

```
35 percent = find_optimum_percent(31, 6)
36 print("percent is, ",percent)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
p_d_e is, 0.0
percent is, 4.0
PS C:\Users\wilfw> 
```

Evaluation

My code works very well and calculates the optimum percentage correctly and checks that percentage is within $4 < \% 100$ and if not changes it to the correct upper or lower bound.

Testing to inform the final evaluation + usability testing

Test/ vid #	Test name/purpose	Test data	Test data	Expected result	Actual result
1	Is the login interface/sign up interface functional and can be traversed as intended.	n/a	n/a	Can traverse in between login and sign-up page as please. Background changes size to fit size of frame. Doesn't allow for no input to be attempted. Can type in all entry boxes.	Successful bar the background gets a bit blurry if change size of frame at all.
2	Does login page allow an email that is stored in database to be inputted with correct password and proceed accordingly.	Email: BOMP@NEA.co.uk	Password: 1234	Valid, displays givenenergy data	Valid, displays givenenergy data
3	Does login page correctly deny access to invalid attempts.	Emails: BOMP@NEE.co.uk BOMP@NEA.co.uk No input	Password: 1234 1233 No input	Invalid. No account detected Invalid password Enter valid email	Invalid. No account detected Invalid password Enter valid email
4	Does sign up page function as designed to with valid inputs.	Email: wilf5@email.co.uk Usual API key and solar id	Password: 1234 Confirm password: 1234	Goes to welcome page, displays data, can go to prediction page which also displays data, users' details are stored in data base. Can log in with correct details	Goes to welcome page, displays no data, can't go to prediction page, users' details are stored in database, can login with correct details.
5.1	Does sign up page reject invalid inputs. Email is not a real email.	Email: wilf@notemail Normal API key Normal solar id	Password: 1234 Confirm Password: 1234	Please enter valid email message displayed	Please enter valid email message displayed

5.2	Does sign up page reject invalid inputs. Passwords don't match.	Email: wilf@email.co.uk Normal API key Normal solar id	Password: 1234 Confirm password: 123	Please make sure passwords match error message displayed	Please make sure passwords match error message displayed
5.3	Does sign up page reject invalid inputs. Presence checks for API key and solar id.	Email: wilf@email.co.uk No API key No solar id	Password: 1234 Confirm password: 1234	Doesn't let through to welcome page	Proceeds to welcome page
6.1	Does sign up page reject invalid inputs. Presence checks for email.	Email: None Normal API key Normal solar id	Password: 1234 Confirm password: 1234	Doesn't let through when there is nothing in email entry box	Denies access when no email inputted
6.2	Does sign up page reject invalid inputs. Presence checks for passwords.	Email: Wilf5@email.co.uk Normal API key Normal solar id	Password: None Confirm: None	Doesn't let user into welcome page	Lets user into welcome page
7	Robust test.	Changing size of window quickly	n/a	Quality of picture is unaffected and is not laggy.	Quality of picture is affected (slightly in x direction, greatly in y direction) quality resets when change pages. Is not laggy.
8	Robust test.	Spamming of buttons	n/a	There is no difference in speed to access different pages. Doesn't crash.	There is no difference in speed to access different pages. Doesn't crash.

Not successful

Very successful

See pages 55 and 63 the testing of energy predictions and percentage optimisation using physical inputs. I didn't retest these two sections of code as it would be overly repetitive because they both already have been tested with the program and function as they should (ish, I will talk about this later).

See file full of video tests for evidence.

I didn't include the testing of parts of programs that have not been created (graph display and calendar).

Final testing (robustness and usability) evaluation

This testing helped point out many mistakes that had been overlooked in previous testing during the iterative development stage. The majority of the algorithm works as it should, but certain aspects didn't, luckily these aspects don't have a great effect on the algorithm.

Tests that didn't go as planned

Bold: User (Dave) answer

Italic: Question

Normal: My comments

4: Does sign up page function as designed to with valid inputs

The reason for this red rating is because when valid inputs were entered, they were correctly stored in database, but the algorithm couldn't access either API, meaning that neither the welcome page or prediction page had any data displayed on it.

How much of an inconvenience does this error pose to you?

This error seems (at first glance) to be a major problem as I already hate the shenanigans of signing up to things already, this may put me off the algorithm all together. However, as it is only really a one time error I have to put up with, I think I could get over it.

I suspect that the reason for this is an error when accessing the database as the user's email was displayed correctly, this is because it is passed into other classes physically and not accessed through database, this leads me to suspect that the database is the source of the problem. I think the writing to database then immediately reading from database has an error somewhere, or maybe they are done in the wrong order. This has been overlooked because when originally testing the sign-up page, data did not need to be read from database, so this wasn't tested.

However, as Dave said, after signing in and not having any data displayed, when the user logs in they see all data as expected.

5.3: Presence check for API key and solar id in the sign-up page

This was a failure because when nothing was submitted to both fields the user was shown the welcome page when should be asked to input values into these fields.

How major of a problem do you feel this error is?

I think this error is fairly unproblematic as I am obviously going to input values into the fields. However, I worry if I don't see these fields and continue without entering details there, there is nowhere else I can input them, and I would probably have to sign-up with a different email.

I suspect the reason for this error is I added in these entry fields after the email entry field which I did create a presence check function for. There was also a lapse in judgement when testing these, focusing on the email and password inputs and not the API key or solar id input.

I initially thought this error to be a “no biggie” but on reflection I realise the user is right and this could be a major inconvenience. Dave also helped me to realise that if the user mis-inputs their API key or Solar id which are both very easy to do, I have left no room for error, and they can't re-enter this anywhere which could be very problematic.

6.2: Presence check for passwords in sign-up page

This failed the test because although when either confirm password or password fields were left blank user couldn't get through to welcome page when both were left blank, welcome page was displayed to user.

How big of an issue do you think this error is?

I think it is possibly the smallest of the three errors outlined in this testing because I would always enter a password, and the chances that I don't see both password and confirm password are next to zero.

I think this error stems from once again a lapse in judgement in previous testing, I tested that the two password inputs had to match but not that they had to contain values.

I think it would be an easy fix as it would take maybe 3 lines and an if statement to do a presence check.

I agree with the user and think this is a minor problem.

Test 7: Robust test (changing size of window)

This was not a complete failure as it wasn't laggy, quality was always optimum when opening up a new window and only partly reduced quality when changing in the x-axis. However, quality was so greatly reduced when y axis changed fast that picture was not only unrecognisable but was just thick strips of colour. When switching from small window to large window quality also diminished.

Is this fail a big issue?

No, I see no reason for me to be constantly changing window size and even if I were to change size slightly, there is only a minor change in quality for small changes. What did annoy me was the change in quality when changing from small window to full window.

I think this error occurs somewhere in my background helper class and this would be a hard fix as it is hard to pinpoint the exact problem, I think the easiest fix is having a default quality lower than the current one and locking out y axis movement to a certain extent.

I don't think this would be a necessary fix though, as does the user as it has no real effect on the program and minimal effect to aesthetics.

Testing energy predictions in earlier testing

The energy predictions were often accurate (within 2-3KWhs) of actual value generated that day but was too often very far off and very inconsistent. In my eyes I considered this a partial failure.

How much of an issue is this for you?

This is a major problem for me because it is one of my top priorities and is realistically half the purpose of the program.

These results indicated not a systematic error but a random error which indicates that my formula is actually correct, but the weather predictions or weather API is to fault.

I feel it is a necessary fix but a hard one, my only solution would use multiple weather predicting API and find averages of cloud coverage.

Final testing to inform evaluation conclusion

In conclusion the testing was successful, the pages all function together well aside from a few missing presence checks and the sign-up to welcome page transition. None of the errors found actually meant the algorithm couldn't be used or function. There is a slight lack of usability but once past the sign-up step it is smooth sailing. My code also proved to be very robust nearly passing both tests and didn't crash or lag during tests. There is a slight aesthetic issues when changing size of window but in total is quite robust. The lack of consistency of energy predictions is also disappointing and out of all the failed tests is the one I would like to fix the most as I feel it is the furthest up the user's priority list.

Success criteria

This is the success criteria determined in the research section of the code.

No.	Criteria	Justification
1	Be automatic , once set up, user does not need to interact with given energy, the algorithm should make changes to how much to charge battery to during off-peak hours on its own.	To be convenient for the user, the user needs to trust that the algorithm is saving them money without their input or/and them constantly checking it is working.
2	Reduce energy sold by user back to the grid , especially in the summer when the sun is more intense and the user is producing more energy than they are using which leads to excess energy sold to grid, should accomplish this by ensuring battery is only charged enough at start of the day to get through demanded energy before solar panels start generating energy. If there is battery left over from the day that is not predicted to be needed the next	The grid buys energy from homeowners for a much lower price than the price that the grid sells it back to them, therefore by reducing the energy sold the grid and storing it for later use the user spends less on the higher rates charged by the grid as they need to buy less.

	day use it in the user's washing machine and dishwasher energy peak during off-peak hours.	
3	Reduce energy bought from the grid , buys all/most energy during off-peak hours in midnight to 5AM hours, use this energy on washing machines and dishwashers during these times as well.	This means the user saves on average 20p per KWh which will save the user lots of money.
4	User should be able to input when they are away, and the algorithm should aim to buy no energy from the grid to keep up with low demand from an empty house. The algorithm should do this by storing solar energy in battery and use battery/solar energy at any point there is demanded energy because the solar panels should be generating more than the house is using apart from maybe in winter months.	This will cost the user the least money whilst they are away from home (hopefully cost them only the standing fees).
5	User interface.	For user to interact with.
6	User can view what is happening with their energy at any given time.	So the user can feel reassured that the algorithm is working.
7	Be reliable , consistently makes roughly accurate predictions and meets other specifications no matter the month.	So that user is confident enough to not check up on their givenenergy system for months and feel confident in the algorithm to do its job.
8	Saves user above minimum 4 pounds a month regularly, above 6 pounds a month in the summer.	This is so that the user feels it is worthwhile to implement this algorithm into their givenenergy portal.
9	Uses past data to influence the algorithms decisions (might be a bit of a stretch).	This should result in more savings as the algorithm will be able to see what did work and what didn't work by how much is saved.
10	A way to calculate how much energy has been saved via tariff rates, total energy bought off the grid, total energy produced by solar panels, the same calculations assuming no changes were made that month by the algorithm, this may be quite difficult.	So that the user can see the effects of the algorithm and the algorithm can use the data to make decisions in the future.
11	User enters their credentials into website including API key and Solar Panels ID.	So the website can access the user's givenenergy account.
12	Accessible to the public.	Anyone can access/download/use the app.
13	Estimates next day's energy production.	To predict how much to charge battery to.

Not successful				Very successful
-----------------------	--	--	--	------------------------

Success criteria evaluation

In conclusion I think there were a lot of criteria missed but I think the main criteria (apart from being automatic) were met and has helped solve a part of the user's program. Based on the user interview the criteria missed that they are most disappointed about is the automatic criteria not being met.

How to complete the criteria not met and evaluation of criteria not met

1: Be automatic

Whilst researching my givEnergy API documentation in teravolt I came across this useful piece of information. I felt it is the easiest solution as all modern computers have some

sort of task manager/scheduler that can be used to automatically open app/website/algorithm to be ran at specific time (23:30 or later) to change the battery percentage with the most possible accurate weather forecast before midnight. This also needs the python library "schedule".

Finally

Hopefully this has been a fairly useful introduction to the Givenergy API and how to use it's basic calls.

Have a play with some of the other calls from the API by changing the URLs requested to get a feel for how it behaves. **Remember you can also trigger these scripts up from Windows Task Scheduler, so you can run them at specific times of the day if you leave your PC running.**

4: User able to input when they are away

I feel that this is not a major criterion that needed to be hit as Andrew Sainsbury only mentioned it once whilst in the original client interview. This could be implemented fairly easily as I feel I have experience handling dates using the datetime library. This, however, could be a useful method as it would help to save user energy by reducing battery charged to as there is less demand for energy. I think it would be a fairly implementation.

I would create a new class for calendar, make a new page, allow user to select days from display that are in two states either home or away. This would indicate to algorithm what to set energy needed before generation variable and energy needed after pre-day variable in predictions class to.

9: Use past data to influence decisions

This could be used to improve the accuracy of energy predictions as it can find out if predicted energy for that day was correct or not, if not was it too large or too little and edit formula to be more accurate in the future.

I would do this by getting energy generated from givEnergy using its API and comparing to the value predicted the other day, I would then perform a calculation that finds the ratio I was off by and times the predicted energy for tomorrow by this ratio. This will hopefully, over time home in onto the perfect constant.

10: A way to calculate how much energy has been saved

This could act to help user gauge how effective my algorithm is performing and if not saving much energy can indicate that there is a problem with my algorithm.

This could be done by finding out how much energy was avoided being imported from the grid at peak hours which will likely have been generated using fossil fuels or nuclear power stations which are both considered damaging to the environment. I can find out how much energy was avoided being imported at

peak times by finding how much of the battery that my algorithm decided to charge to was used during peak times.

12: Accessible to the public

This will end up saving more energy in total and help reduce carbon emissions and save more solar panels users more money which could make solar panels a more viable option and more economically efficient.

I can do this once other criteria is taken care of by publishing on either app stores or as a website online so that anyone can use it.

7: Be reliable

I gave this criteria a rating of dark orange as I felt and the user felt that the algorithm was not accurate enough. However, this is not to the fault of my calculations or algorithm as if this was the case my predictions would be consistently over or under the real value by a fairly consistent value – a systematic error would occur whereas a random error is occurring – this lack of accuracy is therefore due to poor weather predictions, this indicates my weather API is not of a high standard.

To solve this, I should use multiple APIs maybe 3 and find average predicted cloud coverage. I feel this would give a much closer value to actual energy prediction.

8: Save user minimum £4 a month and £6 in summer

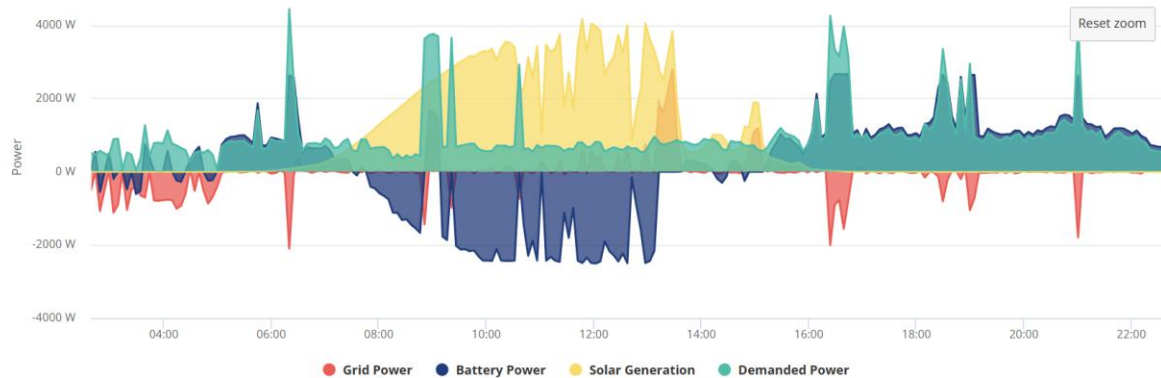
I gave this a yellow rating as it is not that my algorithm doesn't, just that I never actually made a method of calculating this or looked into this as much as I could have, luckily this did not seem to be one of the end-user's top priorities.

To accomplish this criteria, I would use the users tariff rates and how much energy has been saved. First, I would have to fulfil criteria of finding out how much energy has been saved.

Evaluation of the criteria met

2: Reduce energy sold back to grid from user

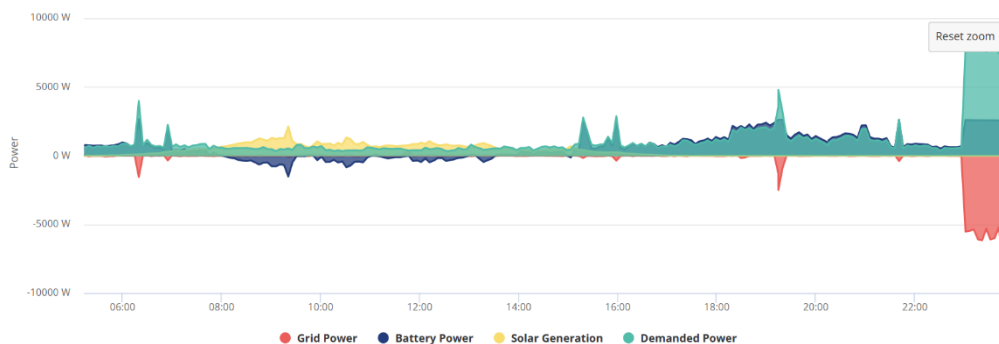
I gave this a green as when predicted energy has been within 3KWhs the optimum charge method seems to work perfectly, and the user can go nearly the whole day without exporting to the grid on a sunny day as shown in API code testing 5.



As you can see from this graph the solar panels generated lots of energy with minimal export to the grid.

3: Reduce energy bought from the grid

I gave this green because I feel even when energy predictions are 4 KWhs off the accuracy is just good enough to (not completely) decrease the energy imported from the grid. This is shown in this graph on a day when not much energy was predicted, the software still managed to minimise the energy imported from grid.



5: User interface

I gave this light green because I feel my interface is extremely usable apart from some small errors which do not greatly affect the usability of the user interface (I will address errors in later section: evaluation of tests). The database works well and for the most part the user interface is extremely effective.

6: the user can view what is happening with their energy

I gave this a light green because I felt that it showed a great deal of data on the welcome page and more data on the predictions page, but never actually shows

the current flow of energy at any point. Despite the user not mentioning this in the interview, I feel that this would have been a good addition.

11: The user enters their credentials into the website

This was given light green as the user enters all credentials into the website in the sign-up page including email, making passwords, API key and solar id. I think a good addition would have been for user to enter their postcode in this section as this would improve the accuracy of the energy predictions as it can then be used in the weather API being passed in as a variable.

13: Predict the next day's energy generation

This was given light green because it often did this but is never usually accurate, I suspect due to changing weather conditions or due to error in my rounding class making the total hours predicted for not add up to the expected amount.

Usability evaluation

No.	Usability features	Justification
1	Straightforward and simple design	User can easily interpret where to find certain information and where to access different features. This will ensure they are not overwhelmed with numbers, words and images and are not put off my algorithm.
2	Different pages for different functions	So that the user can see and do all they need to for the task that they intend to do and no more. This can make it easier for user to navigate interface. For example: having separate pages for today's data, tomorrow's predictions and actions to be carried out and a calendar page for user to input when they are away.
3	Accurate and informative page titles/labels	So that the user can understand easily what different pages do, why they are needed. Accurate label names help user understand the data displayed to them. This is necessary because my algorithm will be dealing with energy in lots of different forms so will need to be clear labels so that the user doesn't get confused.
4	Graphs to display the flow of energy for that day as are shown on givenenergy portal	This is a good visualisation as to what happened to energy at certain times of the day.
5	Users can switch between pages without delay	This will increase the convenience of using the algorithm as waiting times will be reduced.

1: Straightforward and simple design

I gave this a green rating because it was a simple design with not too many hubs and pages that it got confusing. There were clear buttons that directed to different pages with a simple background.

2: Different pages for different functions

There were 4 main functions at the end of my algorithm: Log in, Sign in, displaying data from today and current data and making predictions for tomorrow. I gave green because there were different pages for these different functions.

3: Accurate and informative page titles/labels

I only gave this a yellow rating because I felt that some of the titles and labels were misleading. For example: the welcome page was really a page that displayed energy usage, generation and charge for that day/currently and could have been named better. Also, in the predictions page I felt that “Predicted energy for tomorrow” was extremely misleading. The final inaccurate title/label was “Initial battery charge” this was misleading because it didn’t specify what was meant by initial, did it mean at 00:00 in the morning or at 5:00 after charging (it meant 00:00 but the user doesn’t know that).

4: Graphs to show flow of energy on that day

I gave this red because it wasn’t implemented, the reason for this is despite possibly being a useful implementation, the user did not mention it and is not a necessary part of the solution and due to the amount of work needed to implement this, I decided to have this near the bottom of my priorities list.

5: Users can switch between pages without delays

As you can see in the testing, my algorithm passed robustness tests with flying colours, so I gave this a green.

Usability evaluation

Based on the usability features defined in analysis and the usability testing I can draw up the conclusion that my algorithm is usable but to the bare minimum. The reason for this is because the majority of usability features were met but many of the usability testing was a failure. However, these failures did not greatly influence the algorithm and could still be used in the intended use.

User interviews after David has used the algorithm himself

Italic: Me

Bold: David (end user)

Has my program fixed your problem?

I would say that it has fixed parts of it, my problem was that I didn’t know what to charge the battery to during off-peak hours and thanks to this program, I don’t need to think about this. However, another part of my problem is that I often forget to check the weather for the next day and forget to change the battery percentage which nearly always costs me money and this part of the problem hasn’t been fixed.

What are your favourite parts of my algorithm?

I like the predictions page as this is the page that tells me what to charge battery to and actually does it for me which is helpful. I also feel having the information on the welcome page is also helpful as it shows me any data I may need to see in one location.

What specific aspects would you change?

I would like it to run every day at 23:30 to make sure it updates my givEnergy settings. I also think that the user should be able to input their postcode so that specific weather conditions can be fetched based on the user's location as I think this would improve the accuracy of the energy predictions which could potentially save the user money.

Do you think the energy predictions are accurate enough?

I think that when weather predictions don't dramatically change the energy predictions are accurate enough, but I think the weather predictions are too often wrong and can cause some big differences in predicted and actual energy predicted. This is a slight problem to me and would like more accurate weather predictions as this will lead to more money being saved. However, I still feel like these energy predictions will be more accurate and helpful for finding the optimum charge than me just eyeballing the weather for tomorrow and guessing. The more accurate the better but not a great concern.

What comments do you have on the usability?

I think the usability of the program is poor because of the amount of interface issues that occurred in the final testing. I think I would find myself getting annoyed at the difficulty of the interface. However, I was impressed with the robustness of the program. I think that the names for the labels on welcome page and prediction pages should be more specific.

Italic: Me

Bold: Andrew

Do you feel my algorithm was successful?

I think that it has solved some of the problems I defined at the start but missed the major criteria of being automatic which would have made owning solar panels much more convenient and efficient.

Do you think my algorithm is usable?

I think the GUI is a bit of a let-down given the number of errors and lack of presence checks but these do not actually impact the functionality of the

program so if you were to ask me yes or no, I'd say yes but not to the standard that I am used to.

If released in the form of a website or app, would you use my algorithm?

I think that I would, despite not being automatic, it saves me the trouble of looking at a weather forecast and deciding what to charge my battery to.

Do you think the energy generation predictions are accurate enough?

Yes, in my opinion the predictions don't have to be perfect as with the optimum percentage, I would just like it to be accurate enough to consistently save me money at my convenience. I was surprised there was nowhere for me to input my post code or how many solar panels and what their wattage was. I think if I were to use this algorithm it would be extremely poor predictions as it doesn't know any of my details and there is no-where to input them.

User interview evaluation

From the comments that the user had my algorithm is functional, but they don't seem overly enthusiastic about this and seem to have many issues with the algorithm. They seem to indicate that my algorithm does the "bare minimum" and their biggest problem is the lack of automacy and in close second the inaccuracy of the energy predictions. Andrew helped to point out a big error in my program that there was nowhere to input the user's postcode or their solar panels. This may have been overlooked as I was only using Dave's API and only Dave's inverter. This means the energy generation predictions will be even more inaccurate than they already are for David. I now realise this is also a problem as the user cannot input how much energy their battery can store which will determine the optimum charge to be charged to. This means that the users optimum charge calculations will also be in accurate and not actually optimum.

Limitations/Insurmountable problems and how I dealt with them

How I dealt with/got round limitations mentioned in analysis:

Creating a user interface alongside the algorithm proved to be not very difficult, the difficulty I had was implementing the automatic aspect as well as the other two aspects. In the future I would likely use task scheduler for a small scale/local scale but for larger scale would probably need to use either part of a dedicated server or a raspberry pi to iterate through a database containing user details and running program using each user details.

The accuracy of the program ended up being an insurmountable problem and is talked about further in that section.

The usefulness of the algorithm couldn't actually be tested during the winter due to prediction module being developed just after winter really and is hard to model. The testing of program became a newfound problem and is talked about in the next section.

The last limitation was not addressed, I did not get onto developing different modules for different inverter suppliers. In the future I would likely develop a selection screen at sign up page for them to select their inverter provider and that be stored in dataset, I would then use if statements to decide which API module to read/write to inverter. Luckily most modules wouldn't need to be changed at all/only changed a little.

New limitations I encountered:

Most API's cost money to use especially at the frequency I was using them at for testing, this meant that I had a limited selection of which APIs I could use. For example, the Met Office was the API I wanted as it is one of the most accurate forecasters out there, but they charged a price for their API. Therefore, I was forced to use less established APIs that were free. This meant that I had an inaccurate energy predictor.

As my code has only really been tested in the months of February and March, it cannot be said that my code works well or not because it hasn't been tested in any other months as the predictions will vary fairly greatly per month. This is due to changing peak solar hours. It was very difficult to test my program as a whole due to the fact I needed to wait a day to test one piece of data. The best solution to this model was to pass physical data in separately that hasn't been fetched from API to test quickly.

Insurmountable problems:

The changing weather conditions seemed to constantly play a big part in how accurate the energy predictions are, this is because the cloud cover is a very unpredictable thing and in my case the cloud cover predictions shifting either way one or two hours from what was predicted could completely throw off my prediction because of the difference in peak solar hours generation and other parts of the day generation. Luckily, the accuracy of this didn't matter too much as the algorithm does not need to be correct to the nearest percent of optimum charge but within maybe 10% of the optimum charge, this will still save the user money and will allow an allowed uncertainty of about 2 KWhs. I think this uncertainty is attainable if multiple APIs are used to find an average of the predicted energy generated by the solar panels.

Maintenance

Due to the second limitation mentioned, the accuracy of predictions will have to be monitored throughout the year to ensure a level of accuracy has been maintained no matter the changing seasons and if not calculations should be edited and monitored again to test if problem has been fixed, this will likely be a iterative process that repeatably takes place over a year or a few years.

Evaluation final summary and next steps to make

In complete summary, I think that the program has been successful from the point of view of Dave because of the slightly but not completely positive reviews from my end users and the fact that Andre's energy predictions are likely to be extremely far off. The steps I would take to further improve my program is:

Fix the many bugs discovered in the final testing. As these are quick fixes and define the usability of my algorithm these should be tackled first. I would also change some of the label names in the welcome page and prediction page (including the welcome page). I would also add more comments throughout my code as there was a severe lack of comments through my code which makes it less understandable for other programmers.

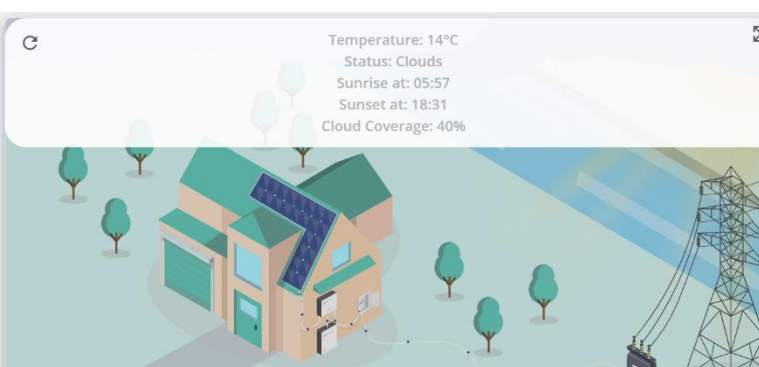
Include entry boxes for user to enter their postcode to be used and passed into weather API, solar panel wattage and number of solar panels and their battery capacity to be used in calculations.

Include a scheduling possibility where my algorithm runs in the background of the user's computer. This should be tackled second as I think from the user interviews it is a big issue and is a necessary fix. This seems to me like it would take a lot of time but after this has been completed, I feel the user feedback will be much more positive so is the next logical step. This would only work on the small scale however and think if I receive lots of demand for program, would possibly need part of a server to run program at certain times or a raspberry pi.

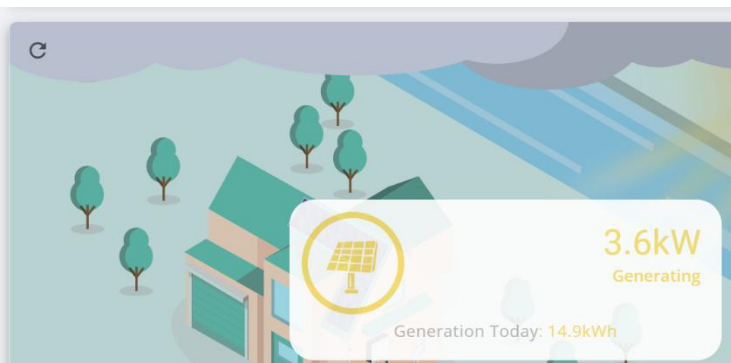
I would then create more validating techniques for things like solar id and API key to check they are in the correct form. I would also create a method that sends a confirmation email to the users email they inputted to check the email they existed exists. This would make my program one step closer to being ready to be released to the public.

I would then start developing other read/write to/from using API functions for different inverter providers and alter other modules and database to support new changes. This would greatly expand my "target market".

Finally, I would use more weather APIs to further improve the energy generation predictions. This is more of a perfecting step, not completely necessary but has been commented on by the one of the users in interview. This is a step that may need to be repeated throughout the year as seasons and months change as the



lengths of days and Peak solar hours change. I would also fix the error due to rounding PShs to the nearest 0.5 that I suspect is happening. Or, alternatively, I could use a completely different method for estimating energy generated, this includes using the givEnergy website to calculate create a table for the cloud coverage against the energy being



generated by solar panels at this cloud coverage at certain hours in the day. Then when each hour has a cloud coverage and is_day value is true, I can find the energy generated by solar panels using the table. This could also be implemented using machine learning to save me the time of creating the table.

References

Page one

<https://www.greenmatch.co.uk/average-electricity-cost-uk> 14/06/2024

<https://www.otovo.co.uk/blog/photovoltaic-systems/how-to-sell-your-surplus-solar-energy-back-to-the-grid/> 14/06/2024

Page two

[Google Maps](#) 15/06/2024

Page three to five

<https://www.goodenergy.co.uk/how-do-solar-panels-work/> 15/06/2024

<https://www.performanceservices.com/resources/how-do-solar-panels-work/> 15/06/2024

Page six

Solar Diverters | Costs & Benefits 2024 (theecoexperts.co.uk) 20/06/2024

Solar panel grants: Government funding explained - Which? 01/07/2024

https://www.deegesolar.co.uk/voltage_optimisers/ 01/07/2024

Page eight

[GivEnergy portal | Advanced home energy monitoring | GivEnergy](#) 11/07/24

[Smart, clean energy storage for your home or business | GivEnergy](#) 11/07/24

[Login | givenergy.cloud](#) 03/10/24

Page nine

[GivEnergy API Documentation \(v1.25.0\)](#) 15/07/2024

<https://terravolt.co.uk/solar-battery/givenergy-api/> 15/07/2024

[How To Use an API? The Complete Guide - GeeksforGeeks](#) 22/10/2024

Page thirty six

Picture is from [Are Solar Panel Fields Bad For The Environment? 7 Super Tips](#) 04/03/2025

Page fifty

[Peak Sun Hours Calculator \(with Map\) - Footprint Hero](#) 05/03/2025

Page seventy-six

<https://terravolt.co.uk/solar-battery/givenergy-api/> 15/03/2025